

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统



第三章：主存储器

第四章：数据的表示与运算

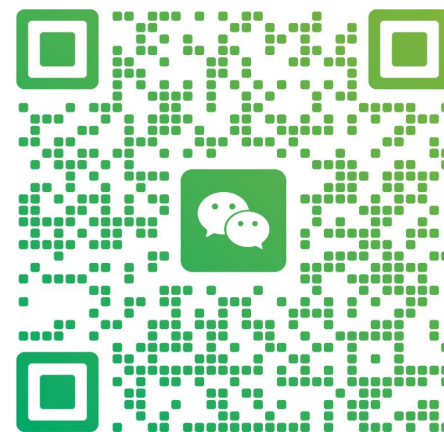
第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统

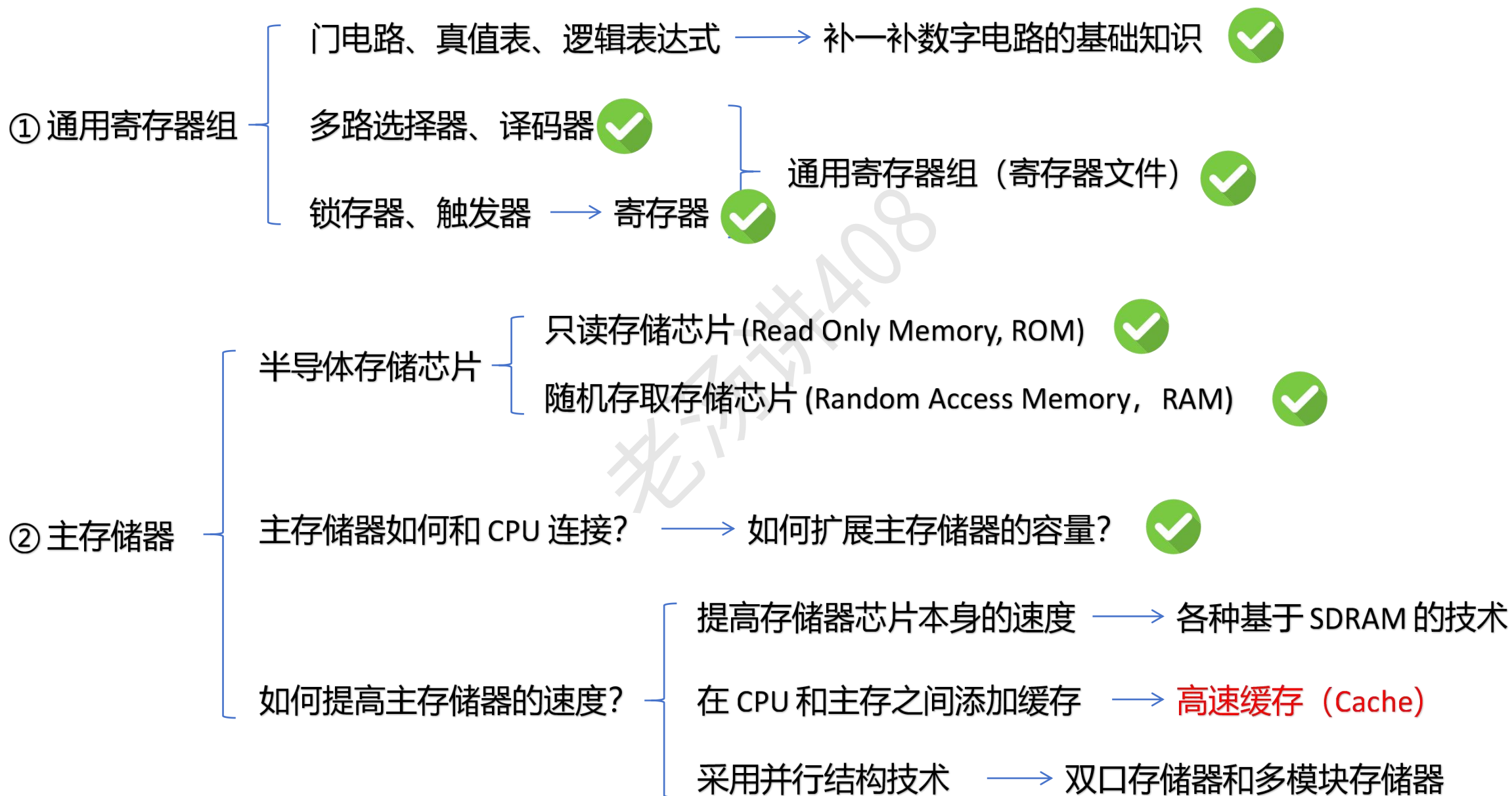


老汤
浙江 杭州

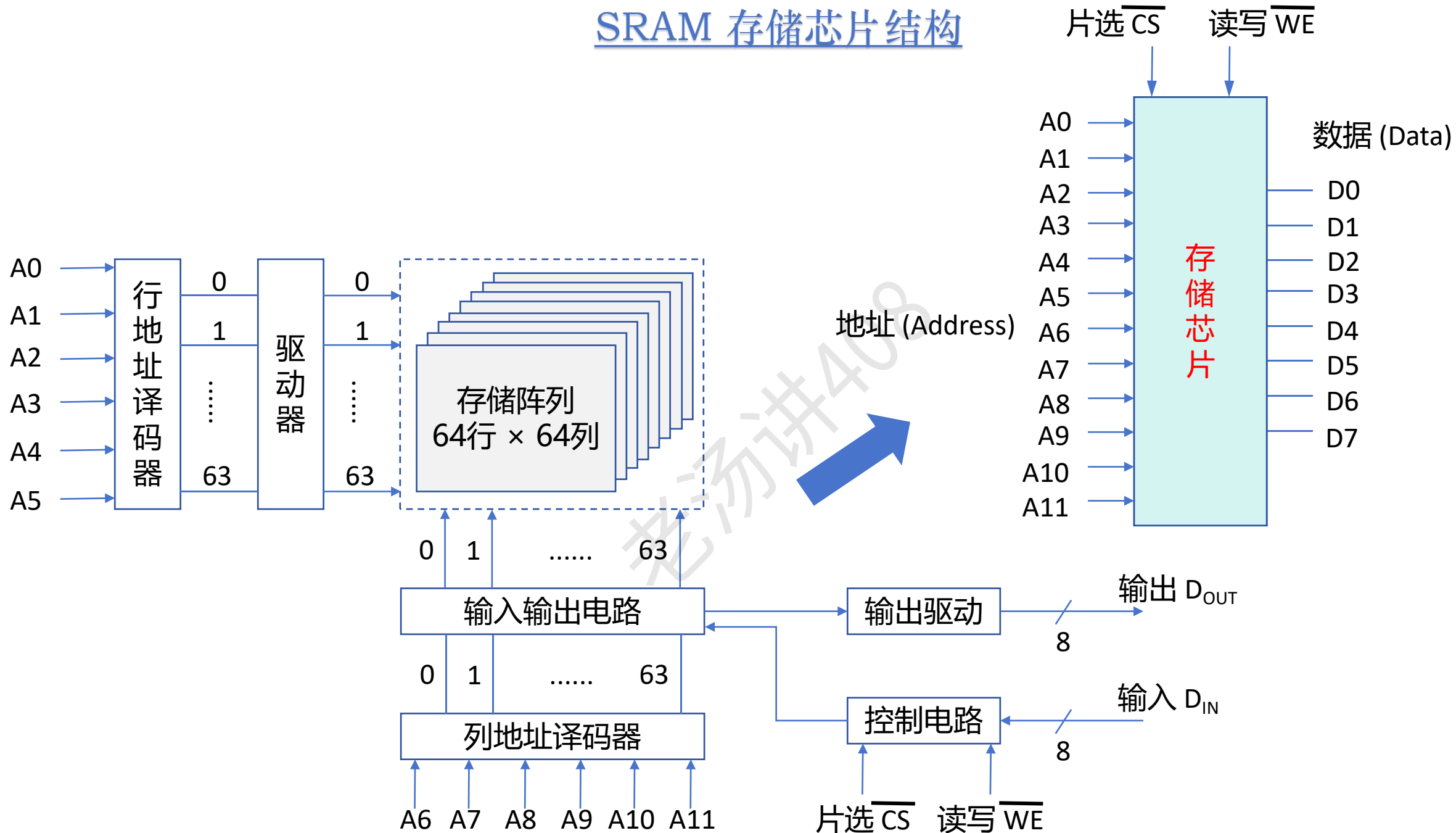


扫一扫上面的二维码图案，加我为朋友。

【第三章：主存储器】内容



SRAM 存储芯片结构

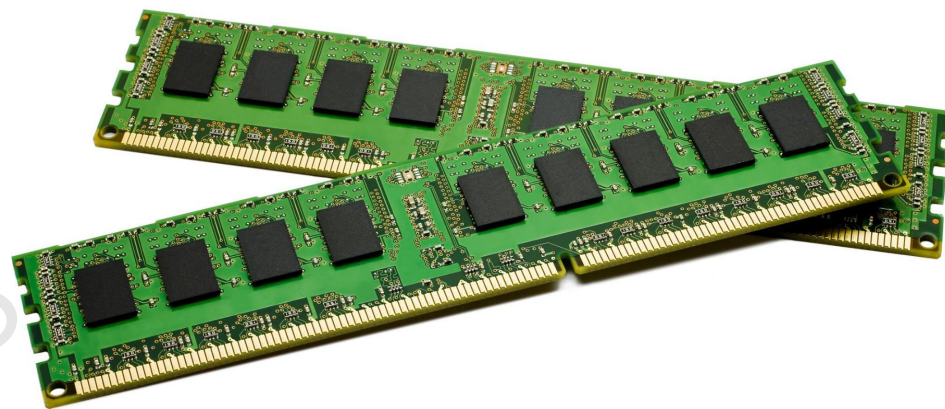


主存储器容量的扩展

受集成度和功耗等因素的限制，单个存储芯片容量不可能很大

主存储器容量大小，比如 8GB、16GB、32GB 等等

存储芯片的扩展技术



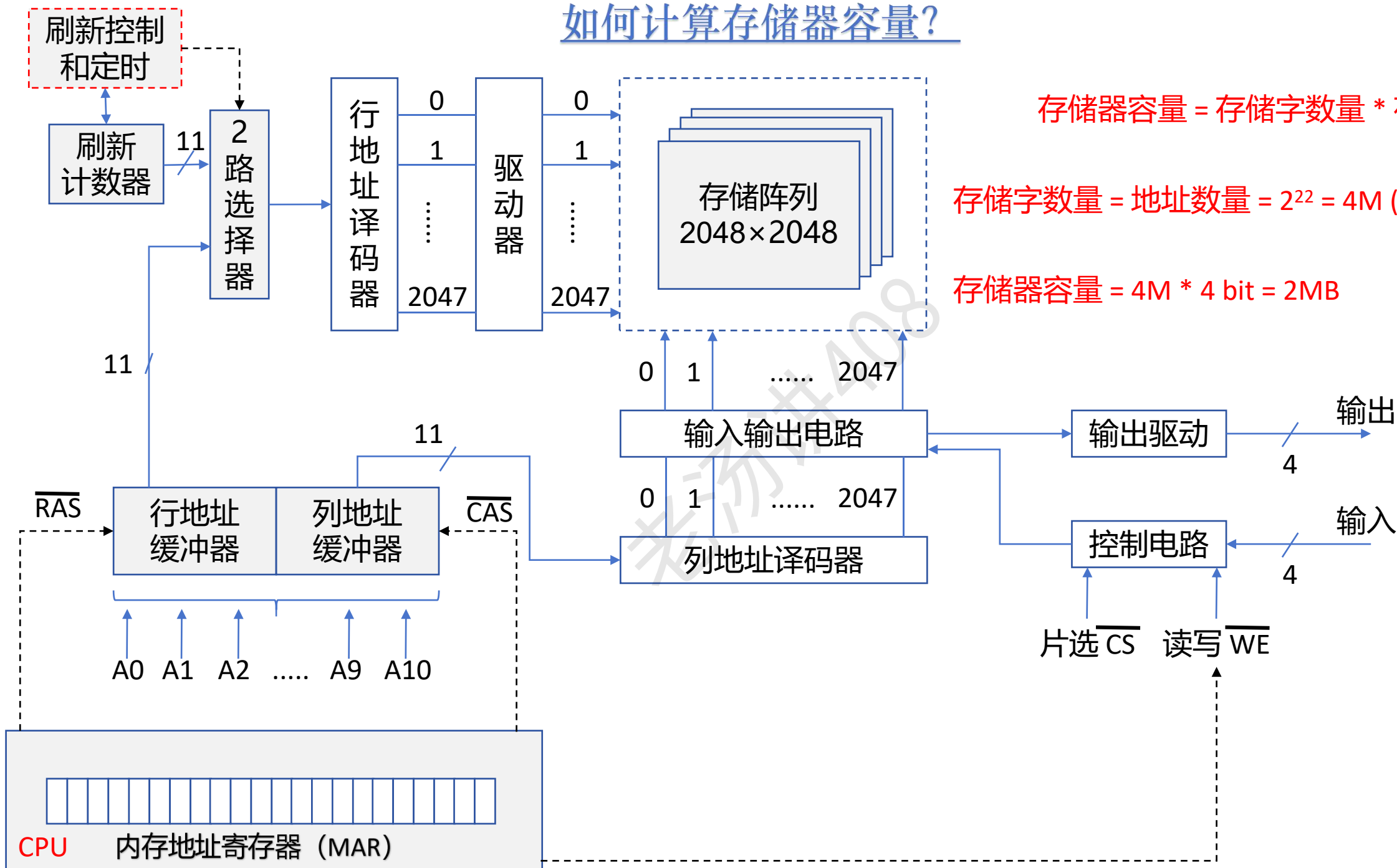
主存模块（内存条）

① 如何计算存储器容量？

② CPU 和主存储器之间如何连接？

③ 主存储器容量如何扩展？

如何计算存储器容量？

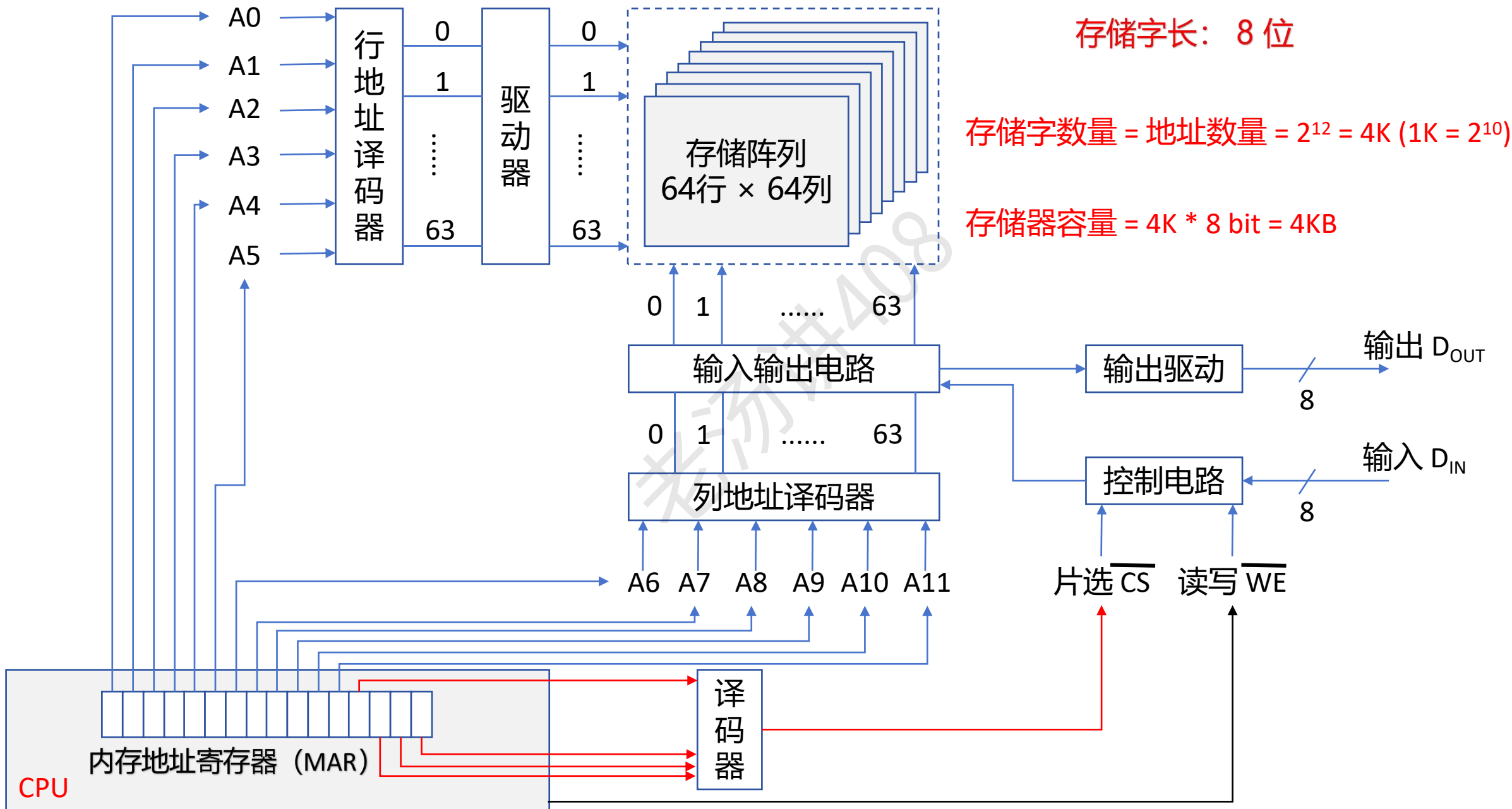


存储器容量 = 存储字数量 * 存储字长

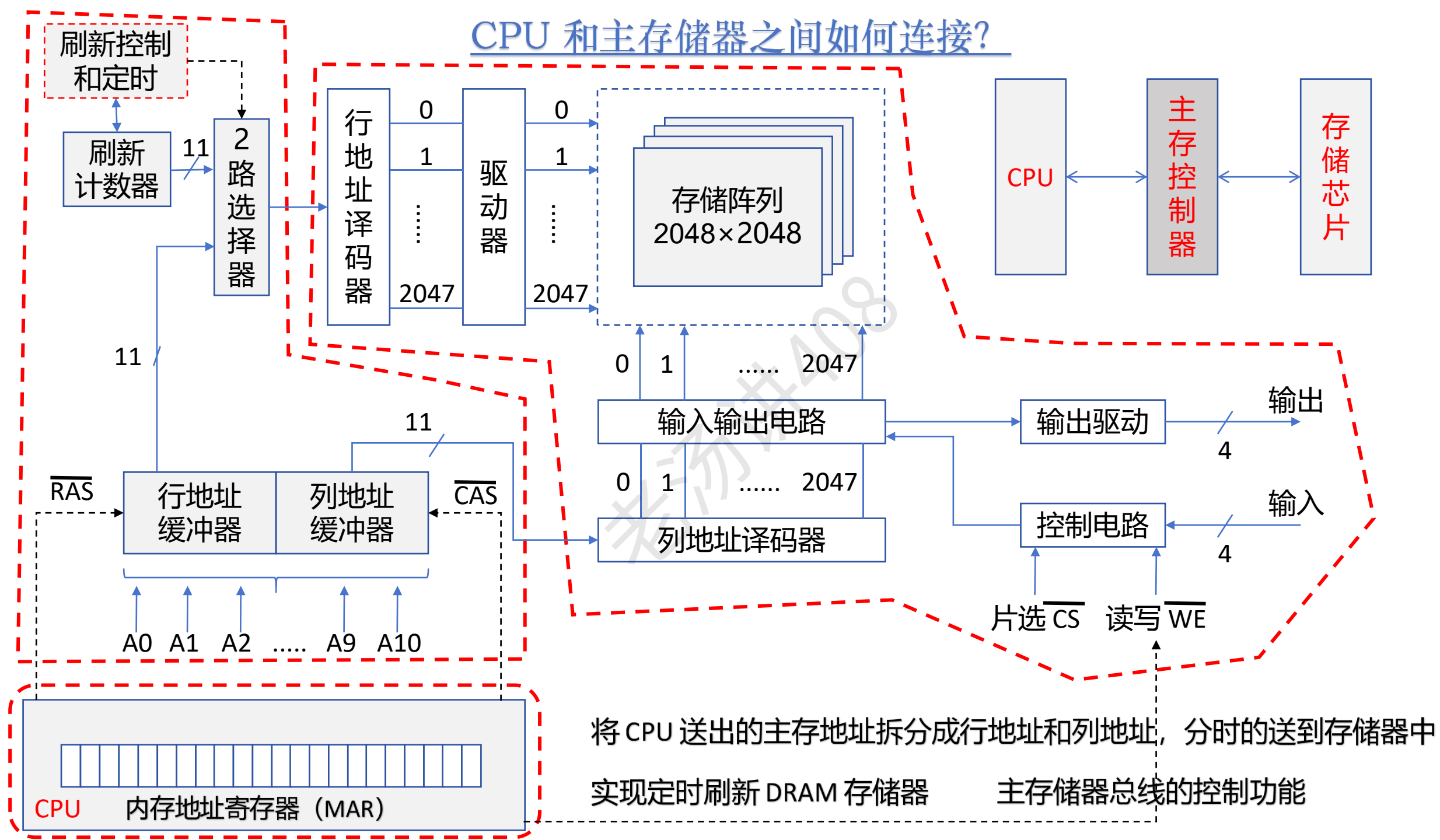
存储字数量 = 地址数量 = $2^{22} = 4M$ ($1M = 2^{20}$)

存储器容量 = $4M * 4 \text{ bit} = 2MB$

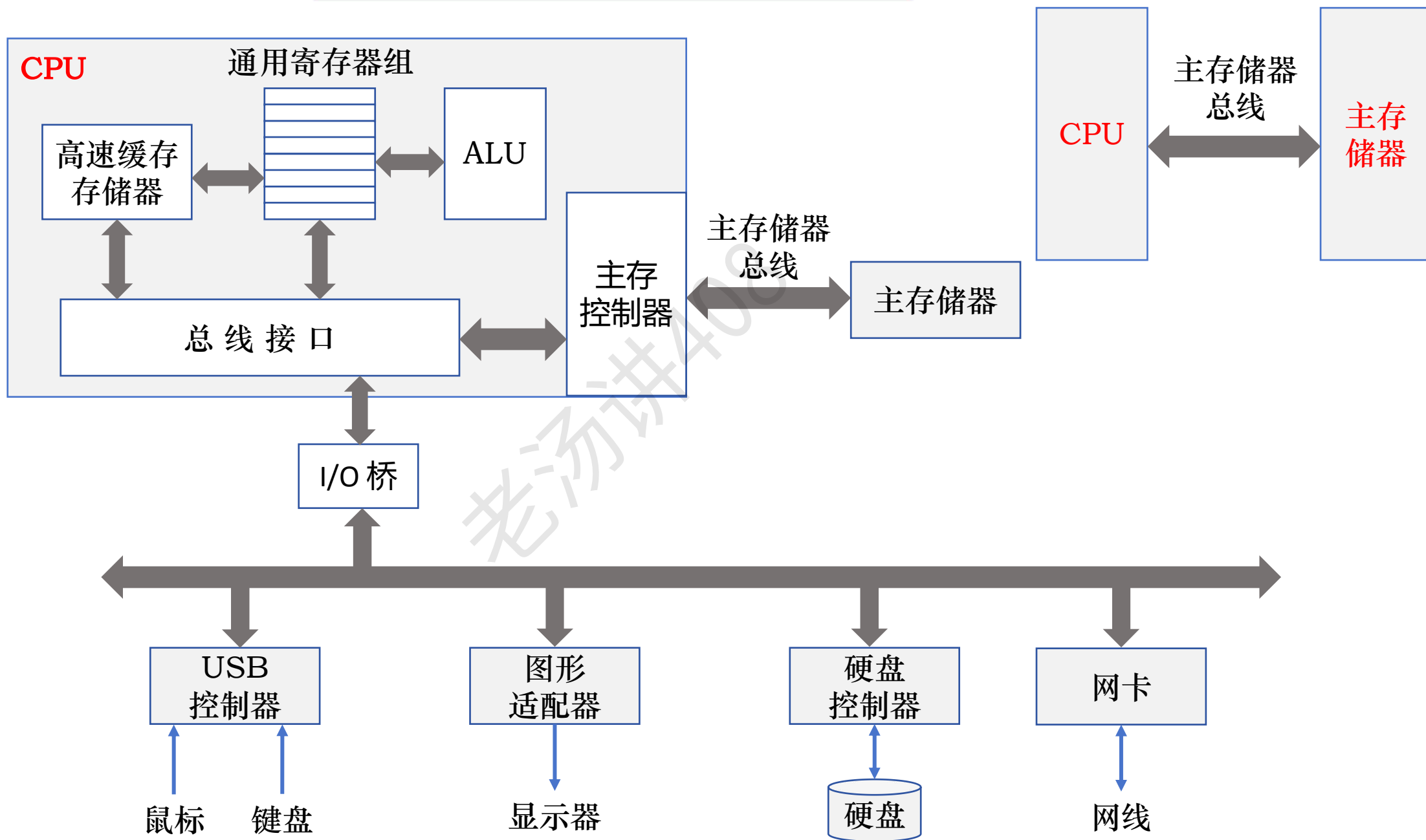
如何计算存储器容量？



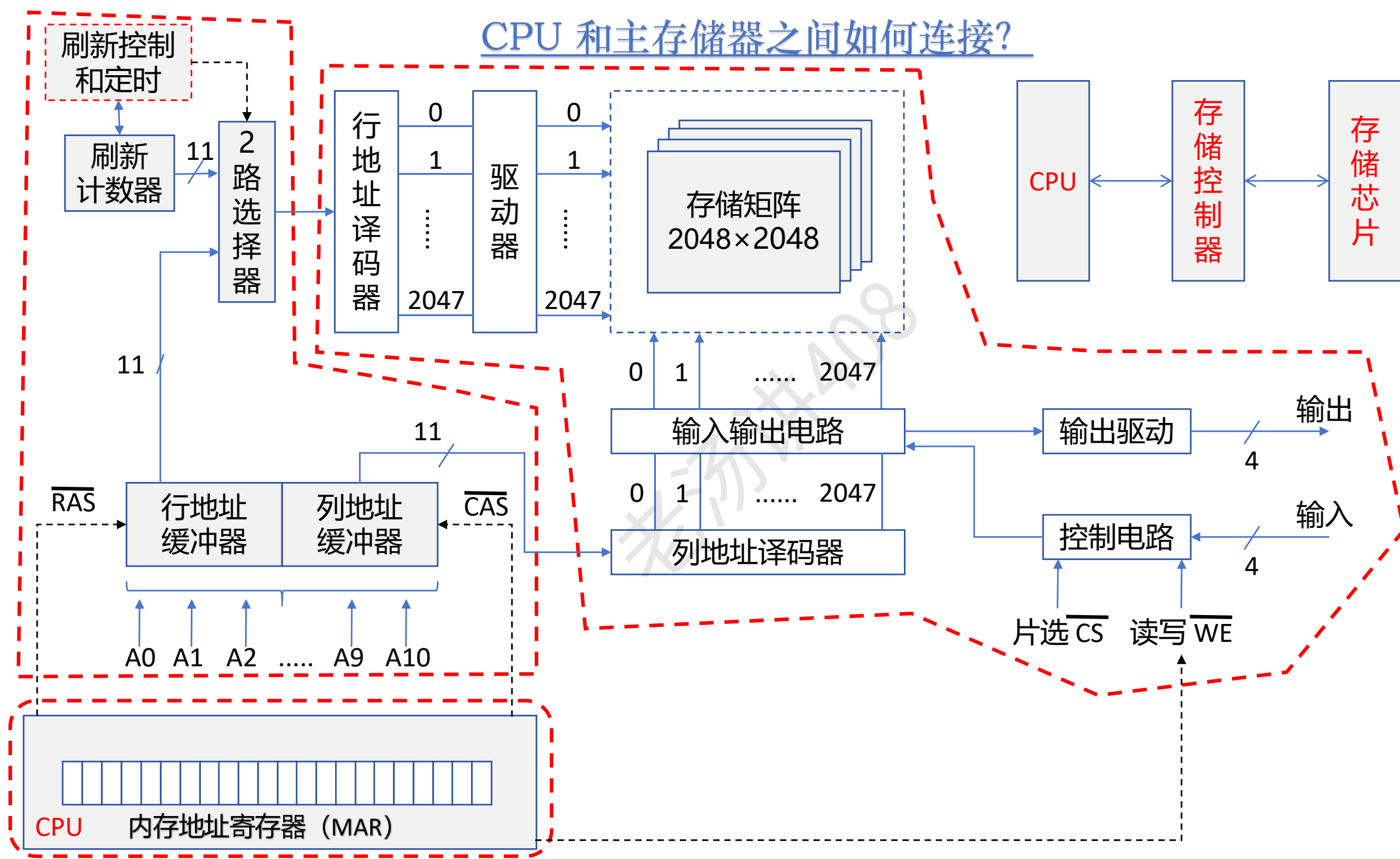
CPU 和主存储器之间如何连接？



CPU 和主存储器之间如何连接？

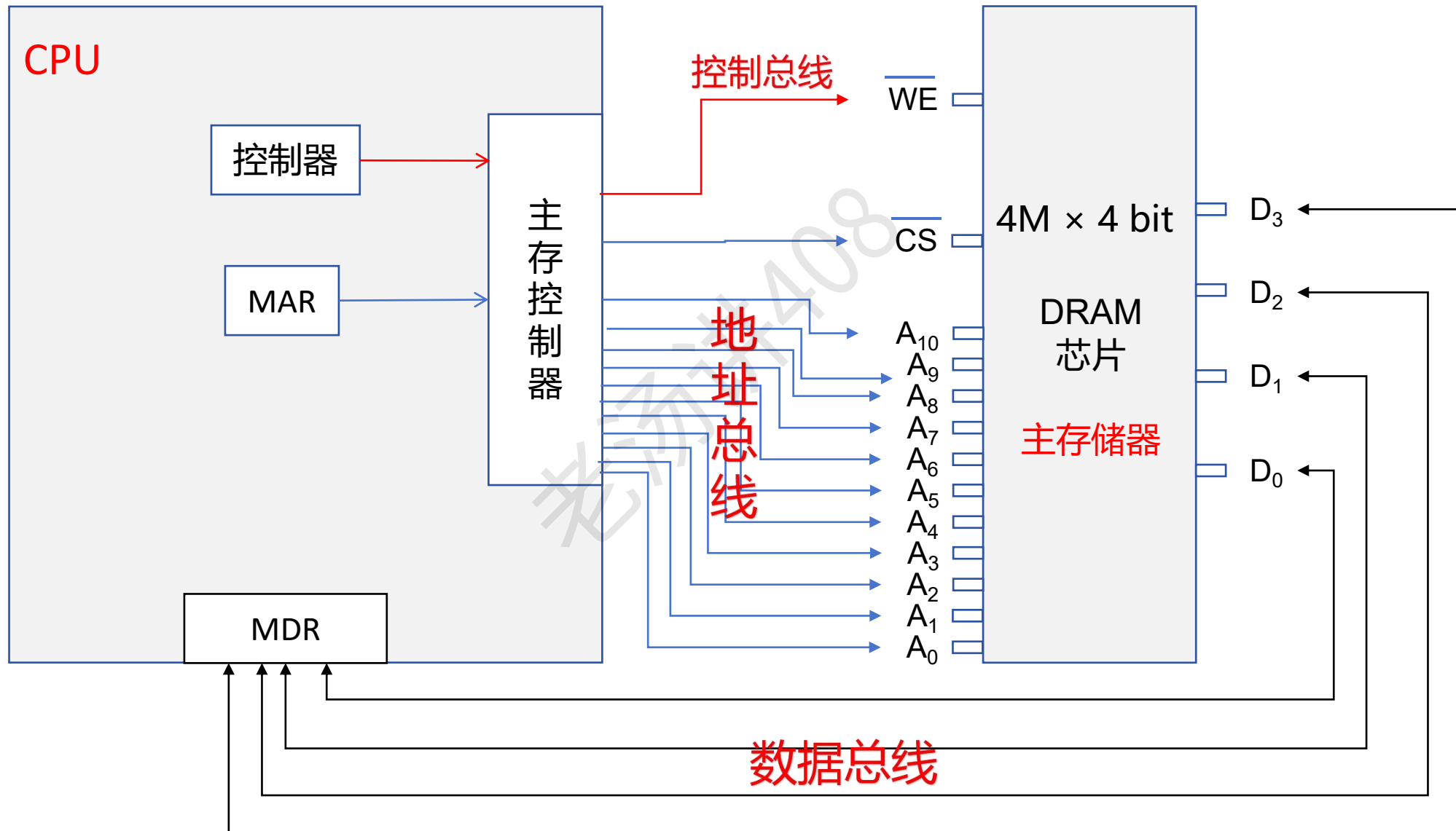


CPU 和主存储器之间如何连接？



CPU 和主存储器之间如何连接?

DRAM 采用地址引脚复用技术

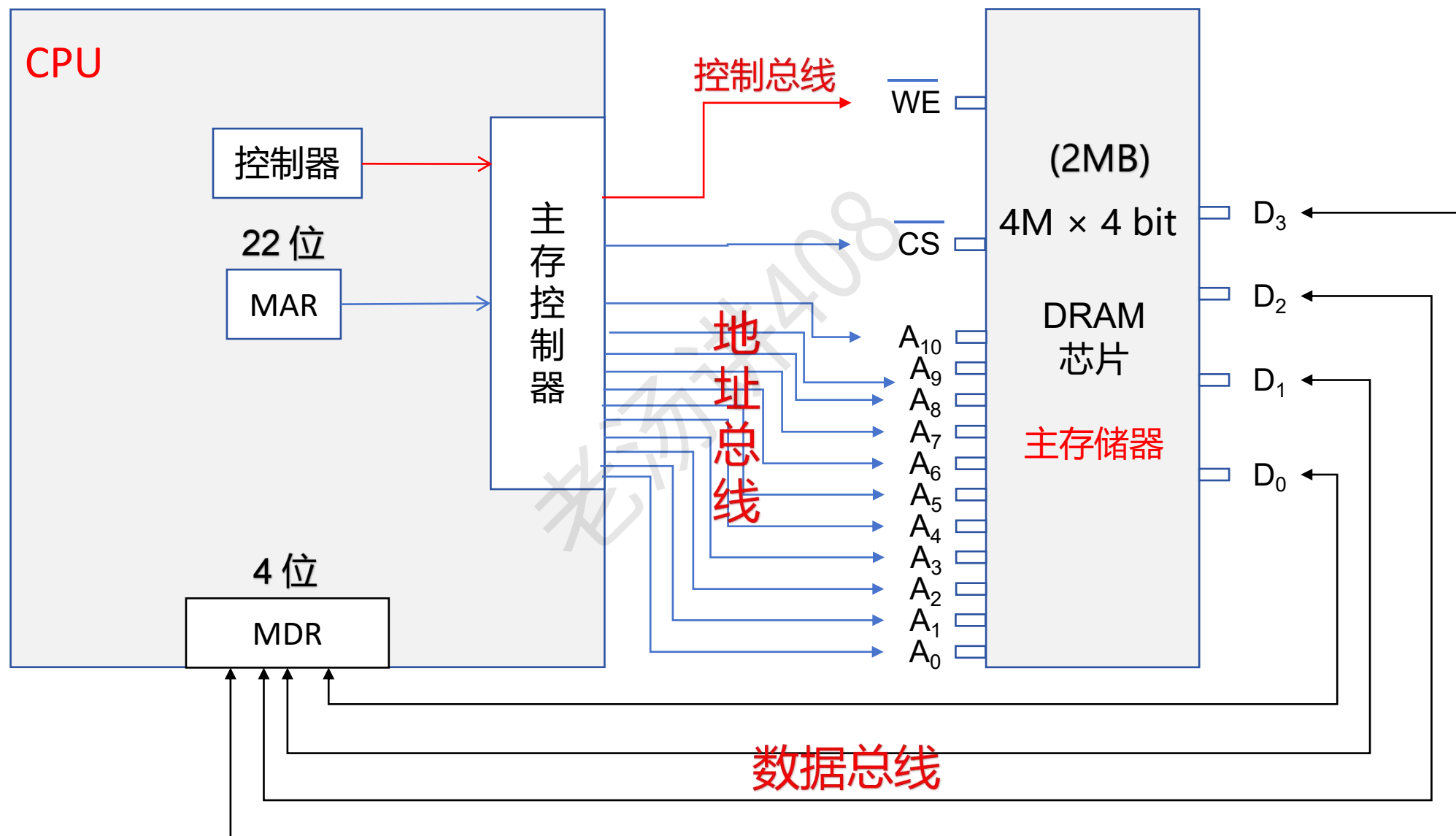


CPU 和主存储器之间如何连接?

MDR 的位数就是存储字长

根据 MAR 的位数, 可以计算得到主存地址的数量

主存容量 = $m * 2^n$



主存储器容量的扩展

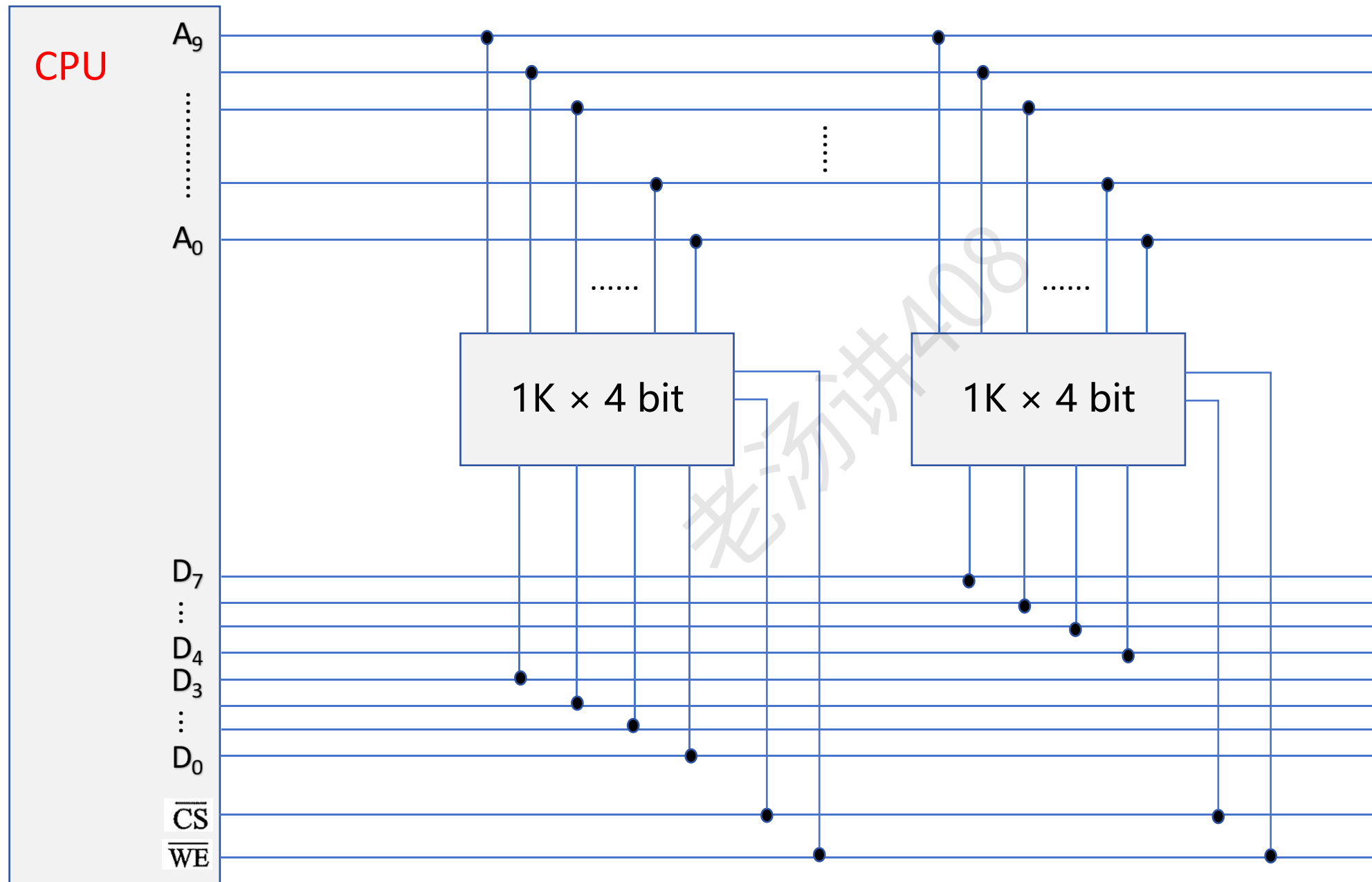
存储器容量 = 存储字数量 * 存储字长

① 位扩展 → 增大存储字

② 字扩展 → 增加存储字数量

③ 位、字扩展 → 同时增大存储字和增加存储字数量

主存储器容量的扩展：位扩展



扩展前:

存储字数量: 1K

存储字长: 4 位

主存容量: 512B

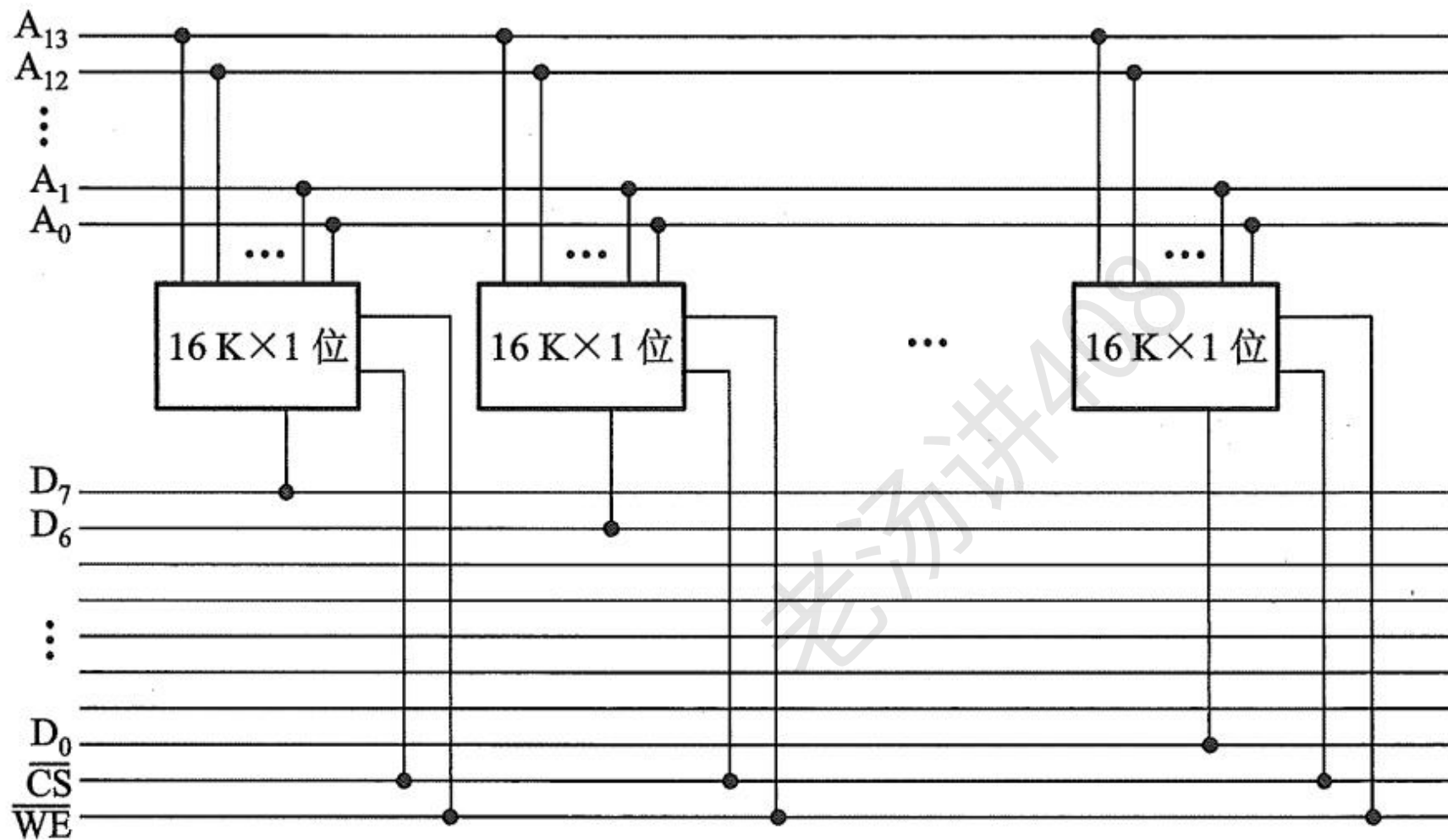
扩展后:

存储字数量: 1K

存储字长: 8 位

主存容量: 1KB

主存储器容量的扩展：位扩展



扩展后：

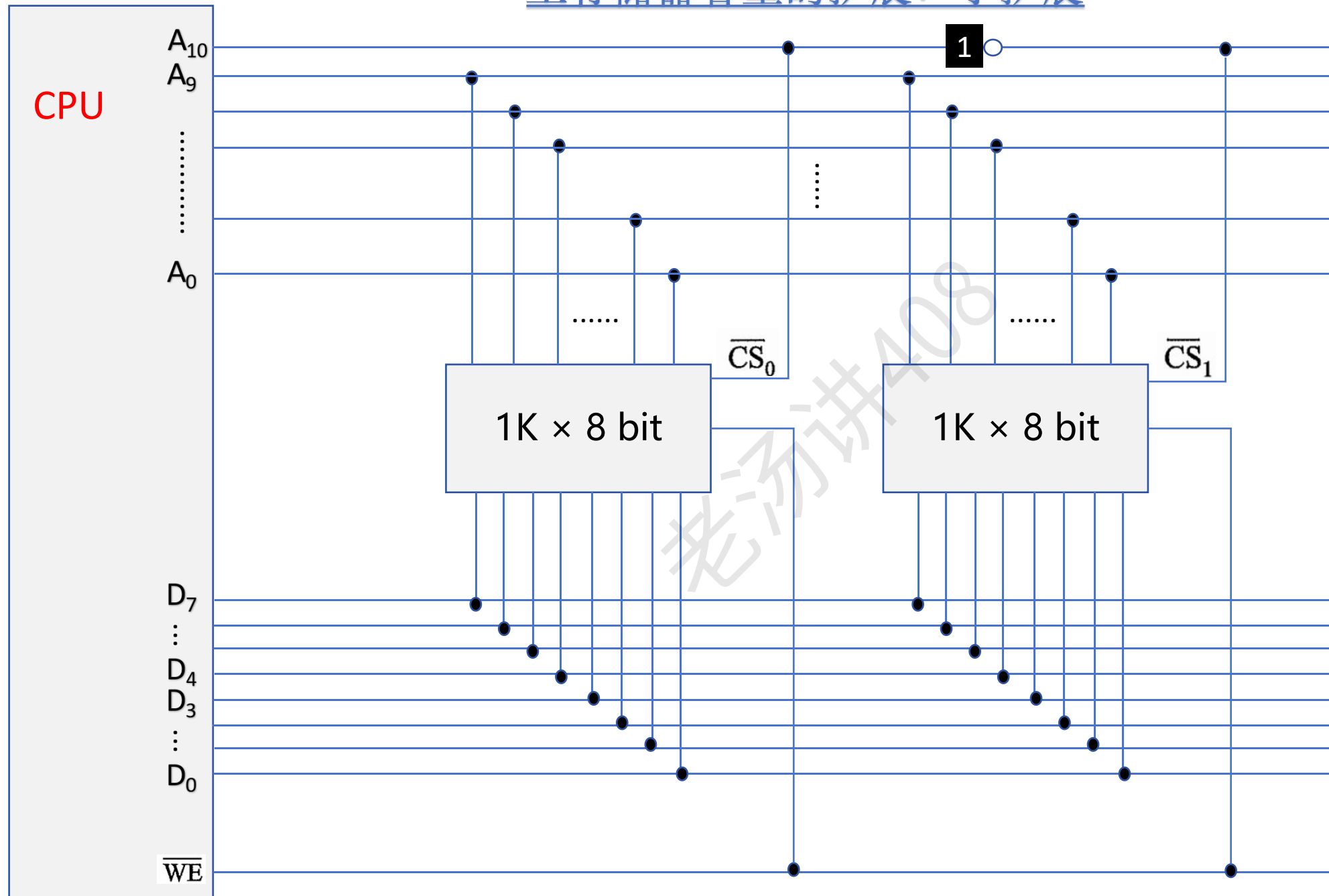
存储字数量：16K

存储字长：8 位

主存容量：16KB

$$16K = 2^4 * 2^{10} = 2^{14}$$

主存储器容量的扩展：字扩展



扩展后：

存储字数量：2K

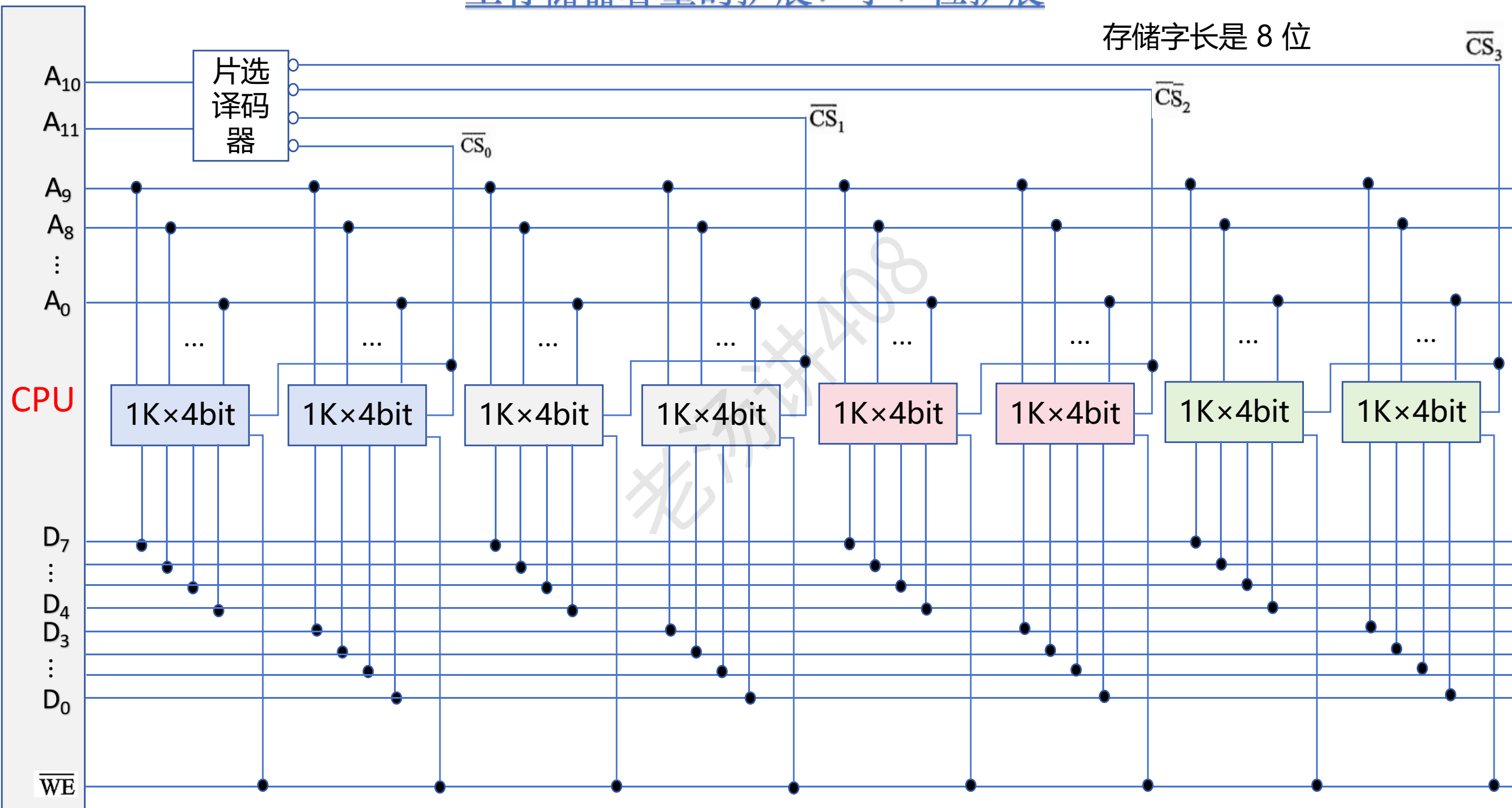
存储字长：8 位

主存容量：2KB

主存储器容量的扩展：字、位扩展

存储字的数量是 $2^{12} = 4K$

存储字长是 8 位



主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片
- ③ 详细画出存储芯片的片选逻辑图

1K×4 位 RAM

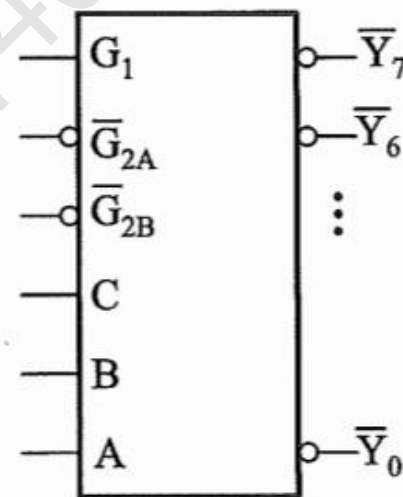
4K×8 位 RAM

8K×8 位 RAM

2K×8 位 ROM

4K×8 位 ROM

8K×8 位 ROM

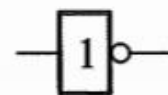


74138 译码器

G_1 、 $\overline{G}_{2A}$ 、 $\overline{G}_{2B}$ 为控制端

C 、 B 、 A 为变量输入端

$\overline{Y}_0, \dots, \overline{Y}_7$ 为变量输出端



主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片
- ③ 详细画出存储芯片的片选逻辑图

1K×4 位 RAM

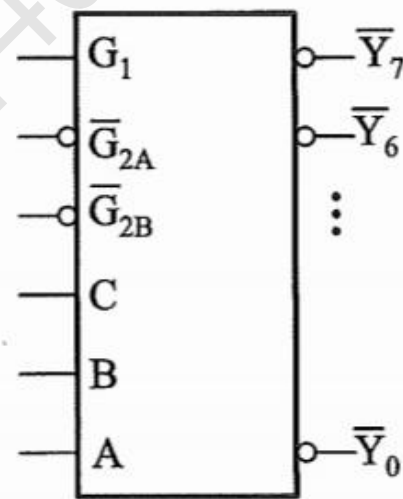
4K×8 位 RAM

8K×8 位 RAM

2K×8 位 ROM

4K×8 位 ROM

8K×8 位 ROM

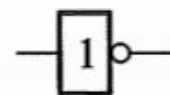


74138 译码器

G_1 、 $\overline{G}_{2A}$ 、 $\overline{G}_{2B}$ 为控制端

C 、 B 、 A 为变量输入端

$\overline{Y}_0, \dots, \overline{Y}_7$ 为变量输出端



非门

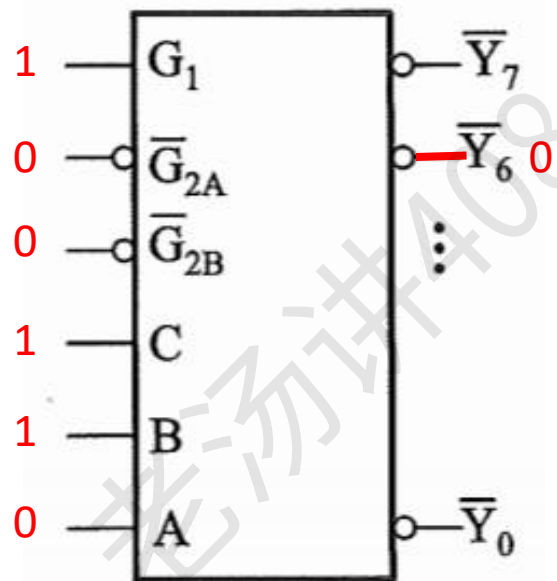


与非门



非与非门

74138 译码器



74138 译码器

G_1 、 $\overline{G}_{2A}$ 、 $\overline{G}_{2B}$ 为控制端

C、B、A 为变量输入端

$\overline{Y}_0, \dots, \overline{Y}_7$ 为变量输出端

主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片
- ③ 详细画出存储芯片的片选逻辑图

第一步：将十六进制地址范围写成二进制地址，并确定其总容量

		A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
系统程序区 (2K * 8 位)	6000H	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	...																
	67FFH	0	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1

$$67\text{FFH} - 6000\text{H} + 1 = 800\text{H} = 1000\ 0000\ 0000\text{B} = 2^{11} = 2\text{K}$$

主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片
- ③ 详细画出存储芯片的片选逻辑图

第一步：将十六进制地址范围写成二进制地址，并确定其总容量

[illegible]

$$6\text{BFFH} - 6800\text{H} + 1 = 400\text{H} = 0100\ 0000\ 0000\text{B} = 2^{10} = 1\text{K}$$

主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

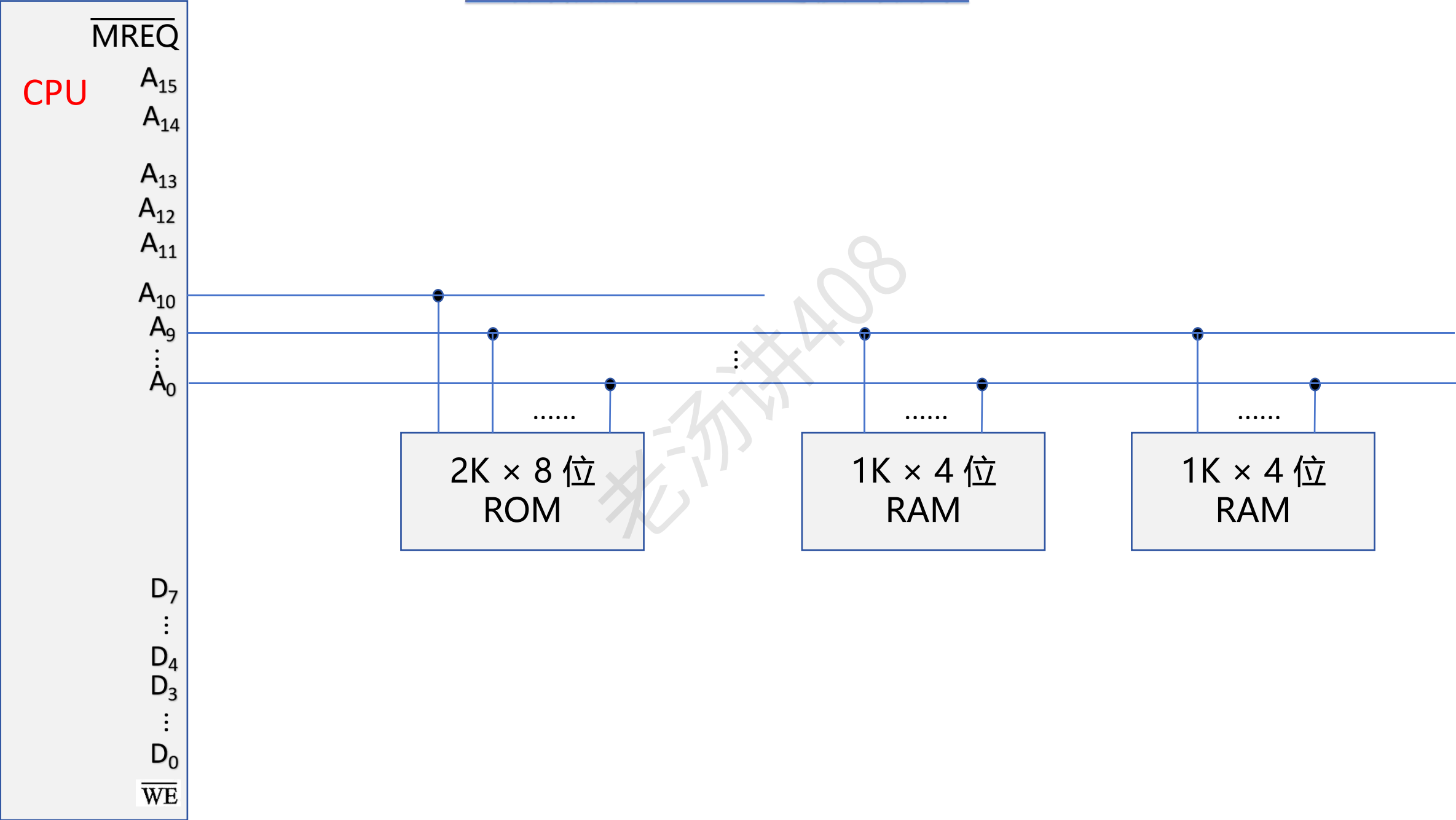
- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片 $67FFH - 6000H + 1 = 800H = 1000\ 0000\ 0000B = 2^{11} = 2K$
- ③ 详细画出存储芯片的片选逻辑图

第二步：根据地址范围的容量以及该范围在计算机系统中的作用，选择存储芯片

[illegible]

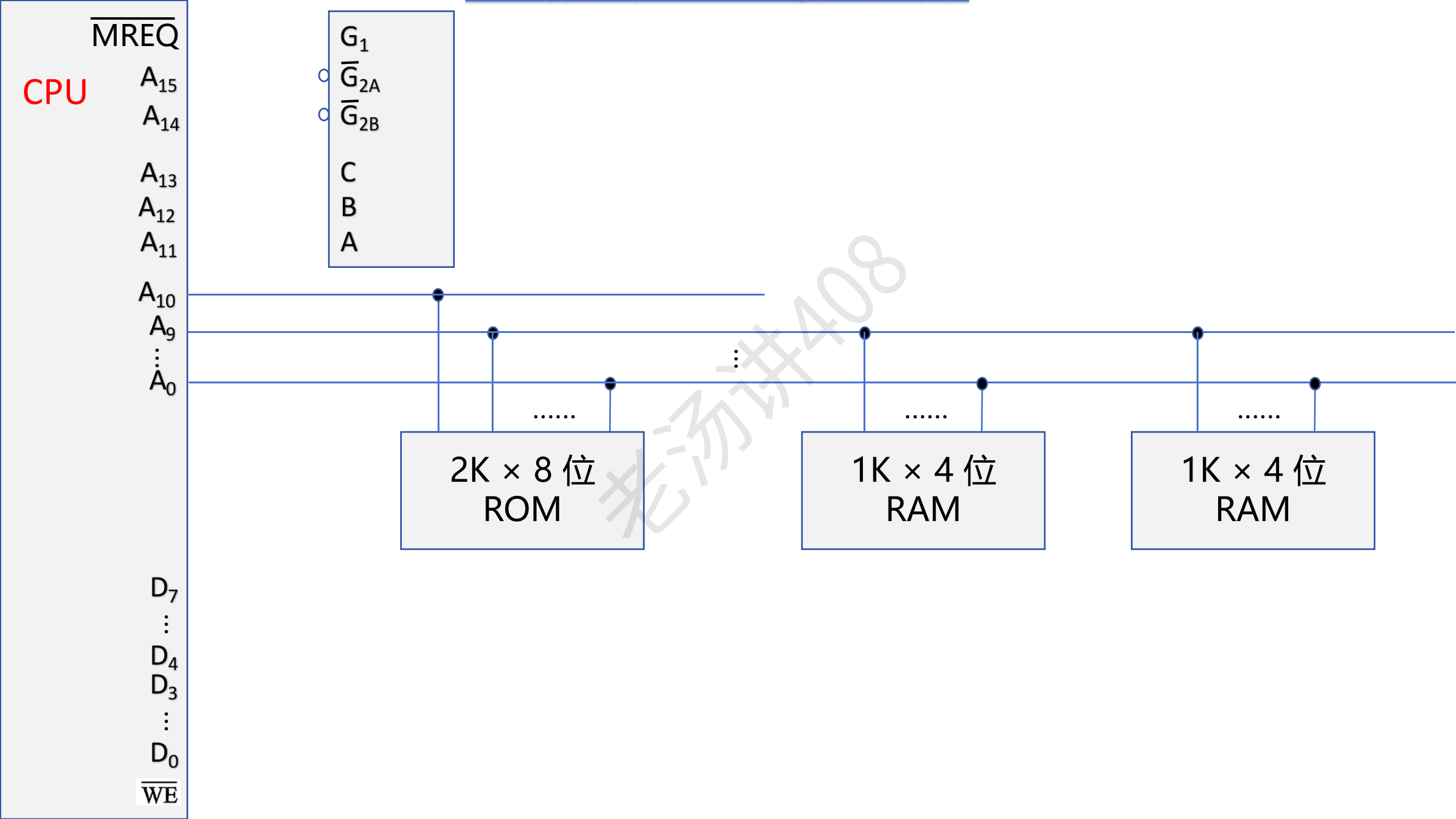
第三步：分配 CPU 的地址线

主存储器和 CPU 连接的例子



第四步：片选信号的形成

主存储器与 CPU 连接的例子



主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

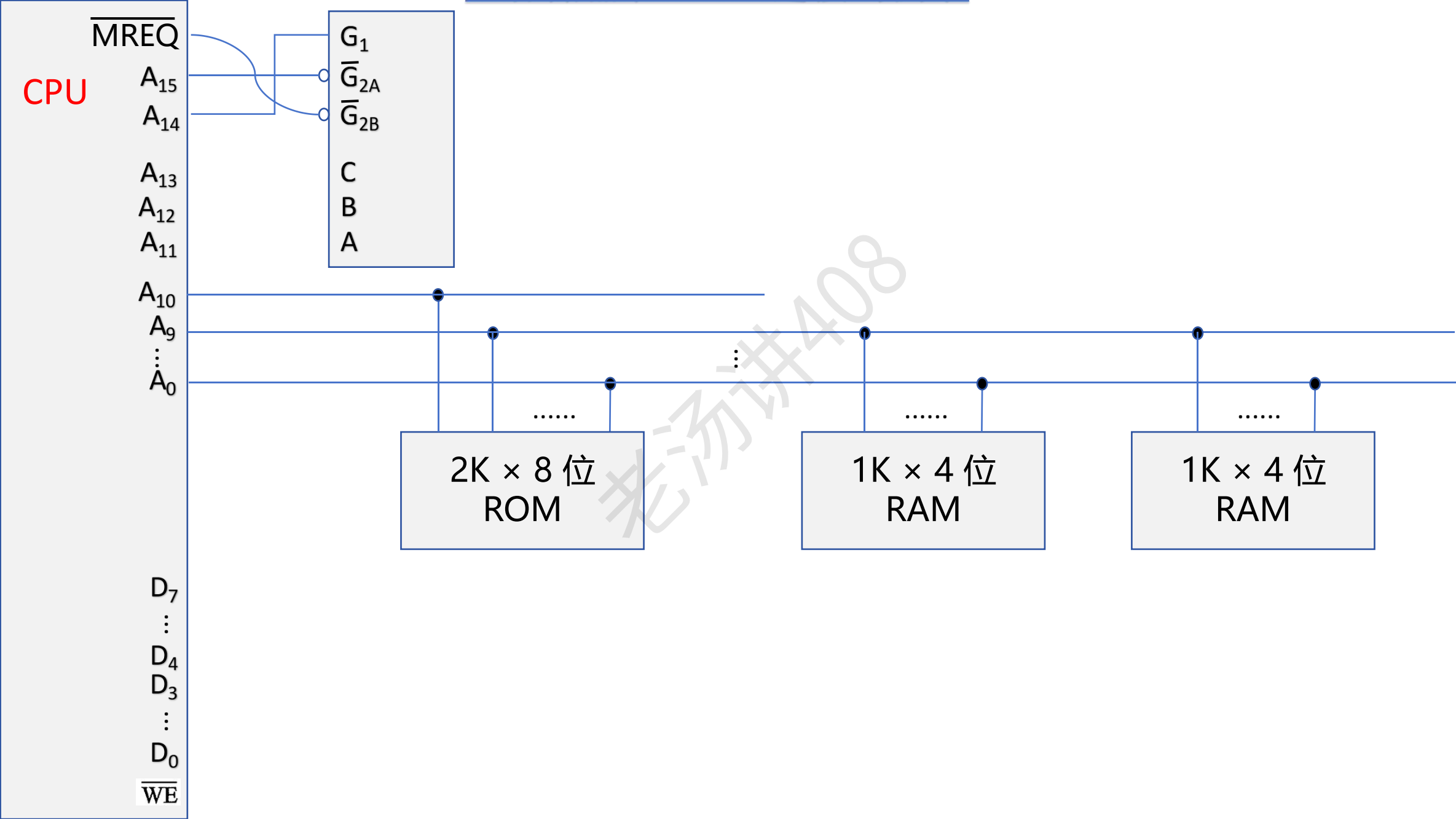
- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片 $67FFH - 6000H + 1 = 800H = 1000\ 0000\ 0000B = 2^{11} = 2K$
- ③ 详细画出存储芯片的片选逻辑图

第二步：根据地址范围的容量以及该范围在计算机系统中的作用，选择存储芯片

[illegible]

第四步：片选信号的形成

主存储器和 CPU 连接的例子



主存储器和 CPU 连接的例子

设 CPU 有 16 根地址线、8 根数据线，并用 $\overline{\text{MREQ}}$ 作为访存控制信号 (低电平有效)，用 $\overline{\text{WE}}$ 作为读/写控制信号 (高电平为读，低电平为写)。现有下列存储芯片：1K×4 位 RAM、4K×8 位 RAM、8K×8 位 RAM、2K×8 位 ROM、4K×8 位 ROM、8K×8 位 ROM 及 74138 译码器和各种门电路，如下图所示。画出 CPU 与存储器的连接图，要求如下：

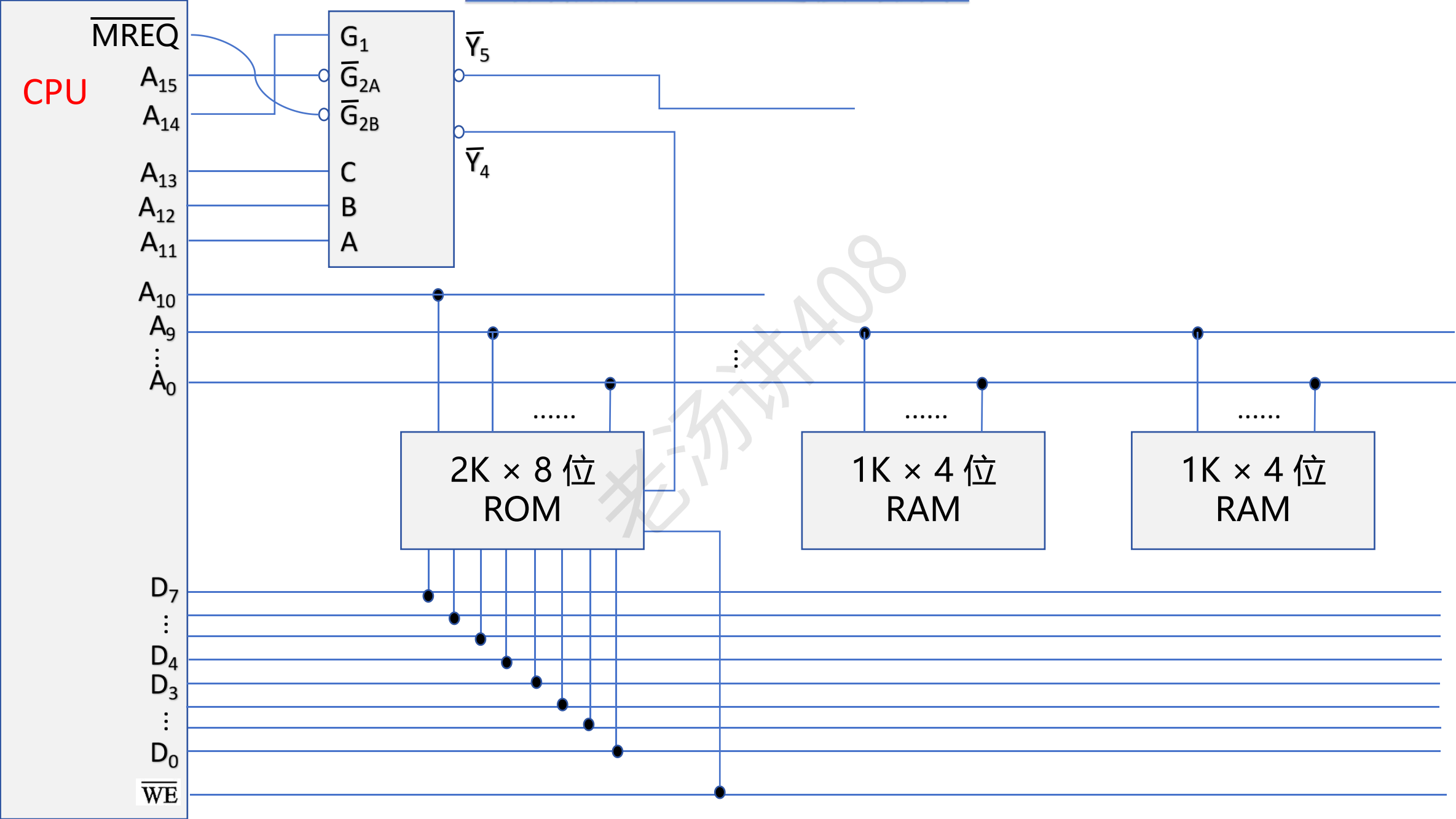
- ① 主存地址空间分配：6000H~67FFH 为系统程序区；6800H~6BFFH 为用户程序区
- ② 合理选用上述存储芯片，说明各选几片 $67FFH - 6000H + 1 = 800H = 1000\ 0000\ 0000B = 2^{11} = 2K$
- ③ 详细画出存储芯片的片选逻辑图

第二步：根据地址范围的容量以及该范围在计算机系统中的作用，选择存储芯片

[illegible]

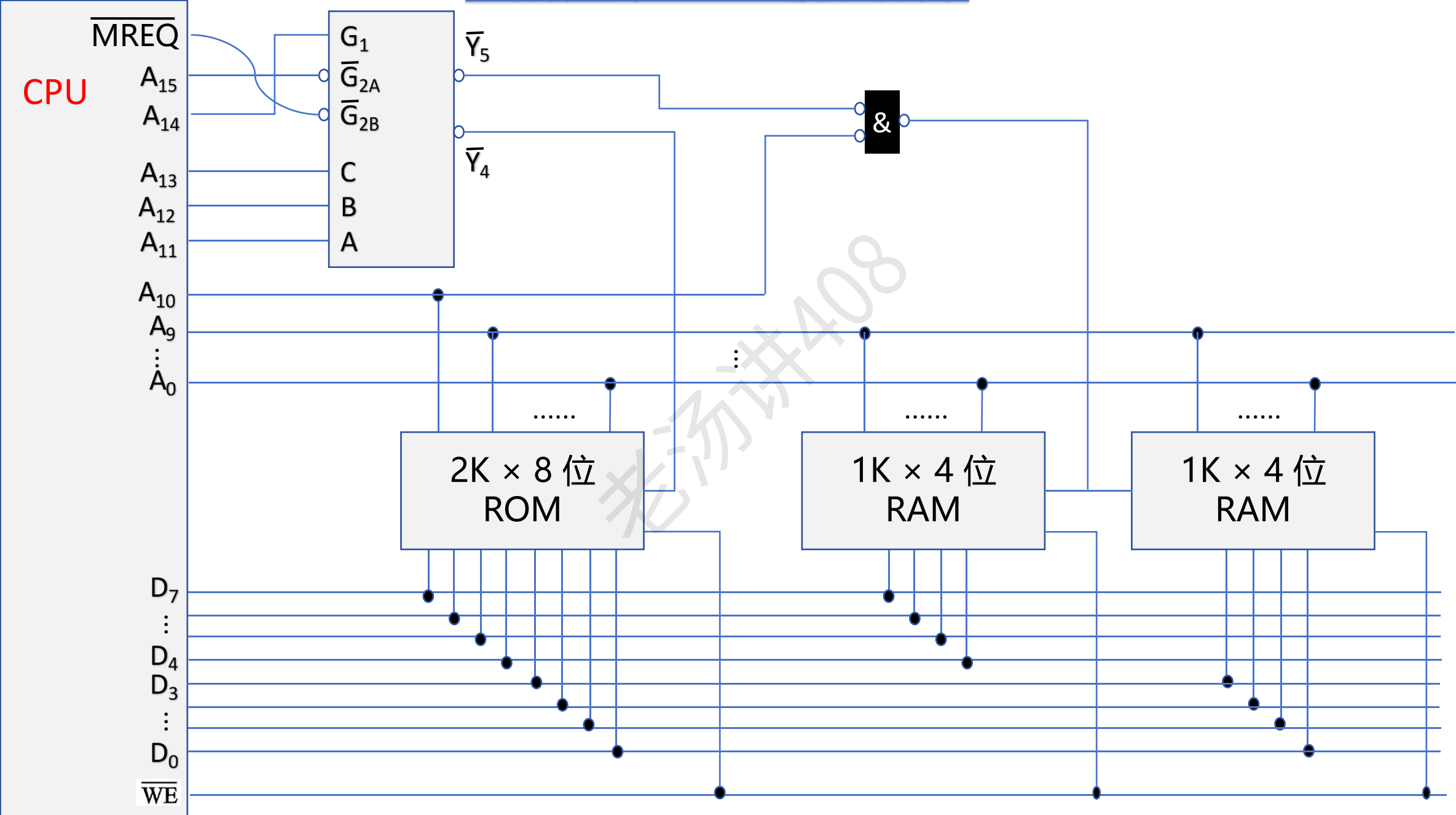
第四步：片选信号的形成

主存储器和 CPU 连接的例子

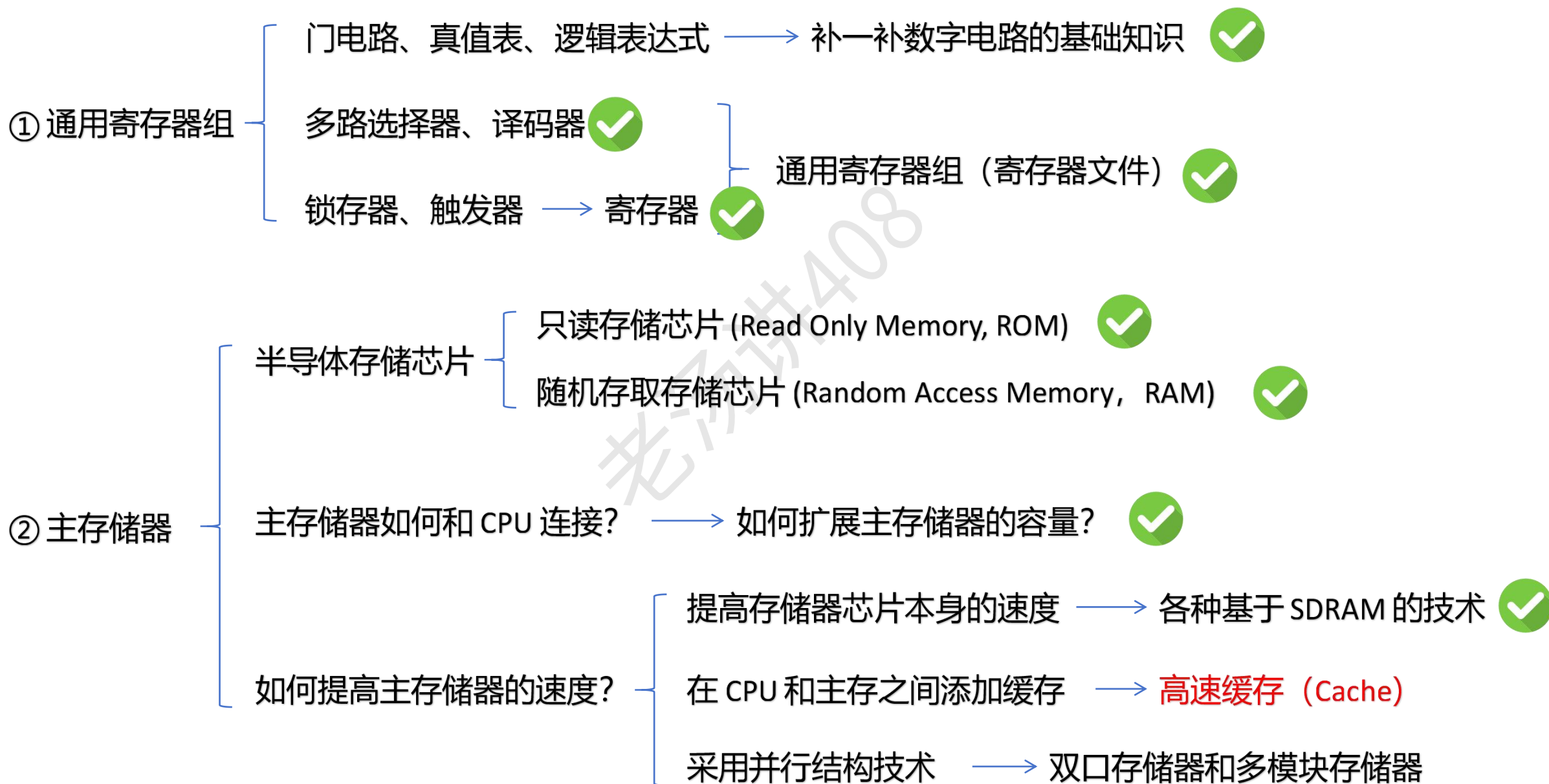


第四步：片选信号的形成

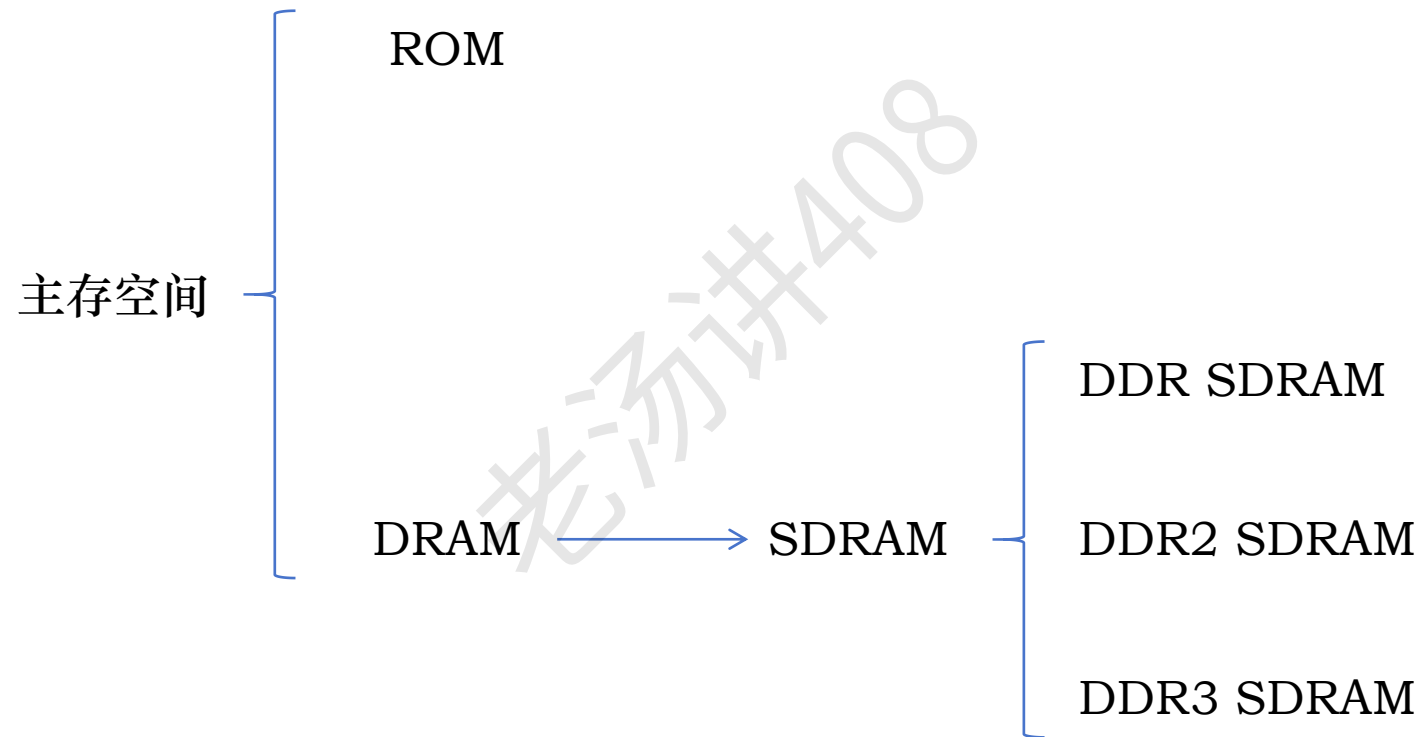
主存储器和 CPU 连接的例子



【第三章：主存储器】内容

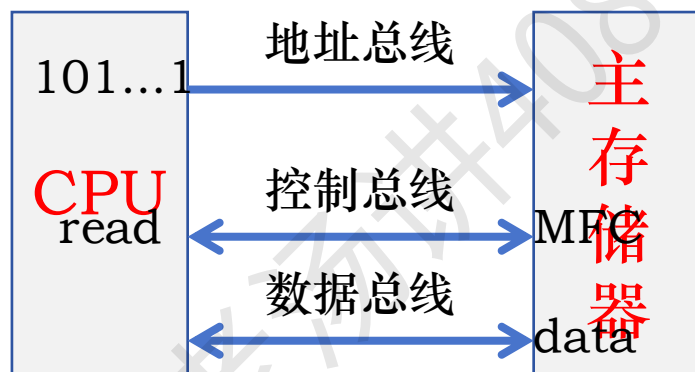


DDR SDRAM 内存条



同步 DRAM (Synchronous DRAM, SDRAM)

早期的计算机，CPU 和主存之间就是采用**异步**的方式进行通信

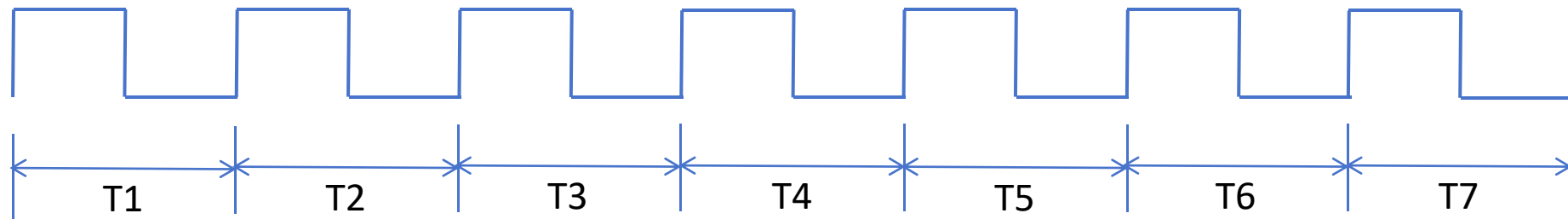


MFC，是 Memory Function Completed 的缩写，意思是存储功能完成

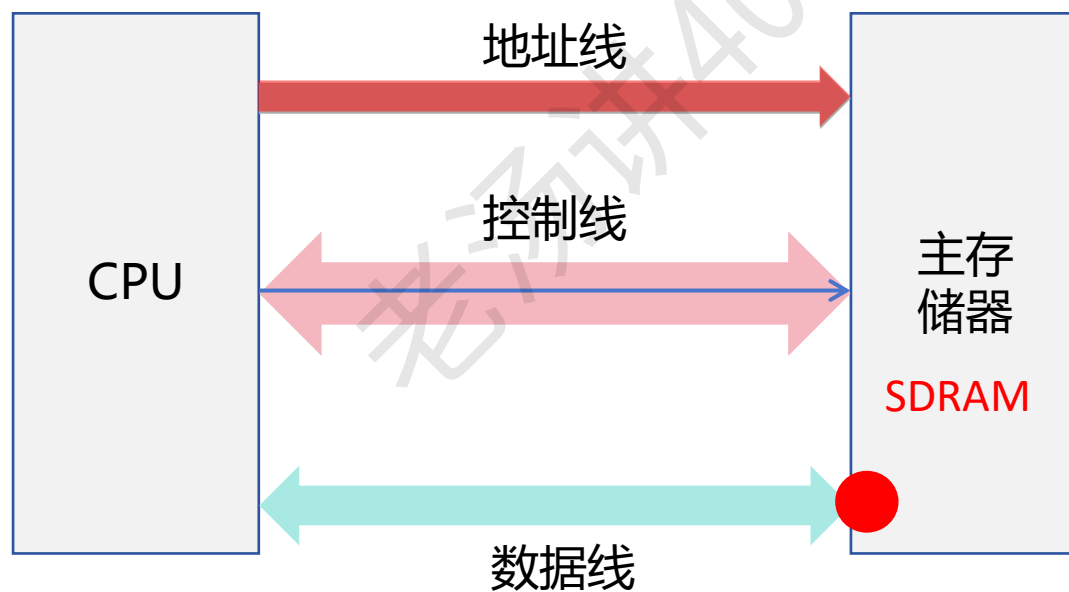
浪费了 CPU 的时间，实际上在存储器读写数据区间，CPU 完全可以做其他的事情

同步 DRAM (Synchronous DRAM, SDRAM)

存储总线时钟信号:

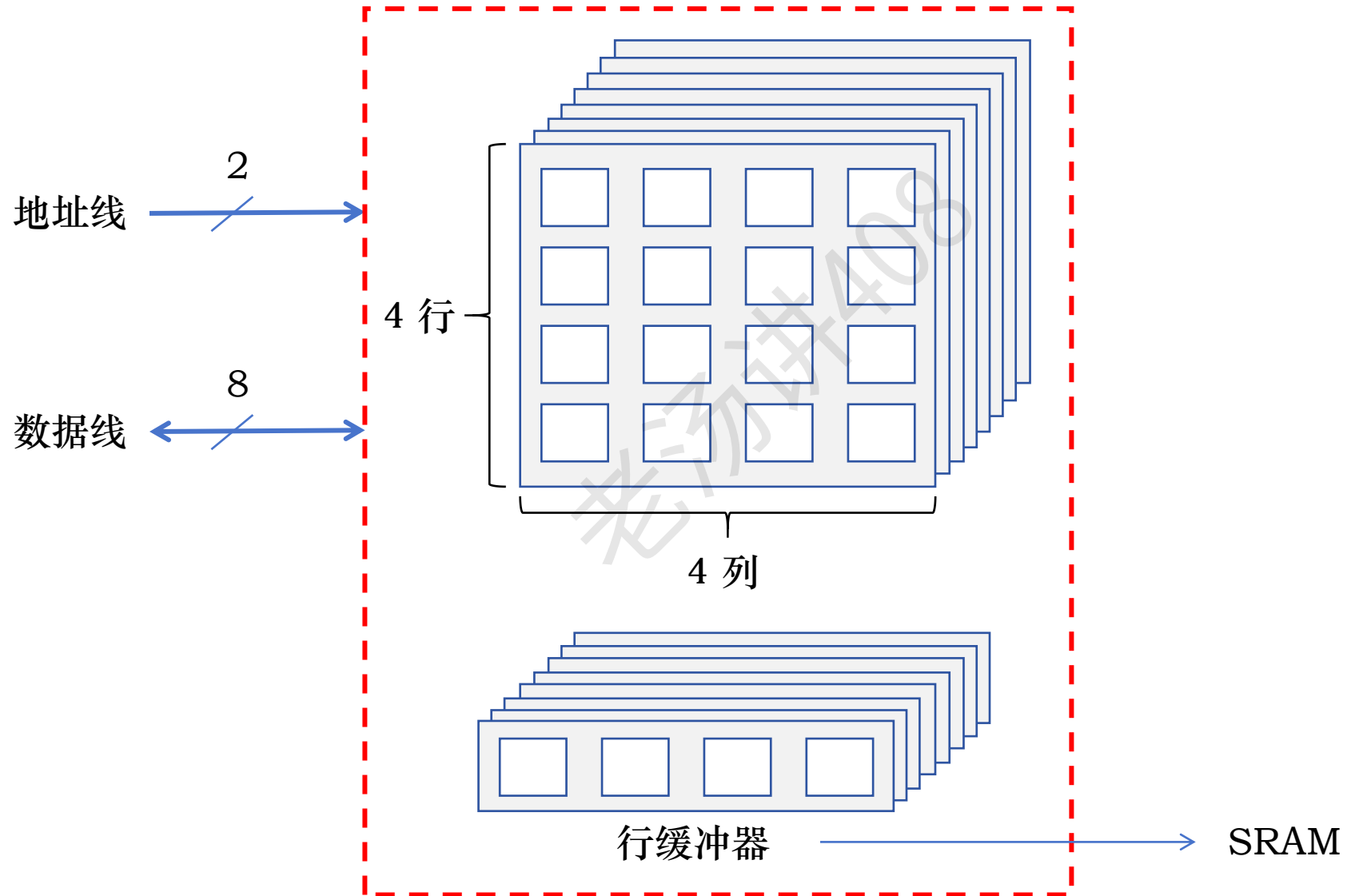


存储器总线



同步 DRAM (Synchronous DRAM, SDRAM) : 行缓冲器

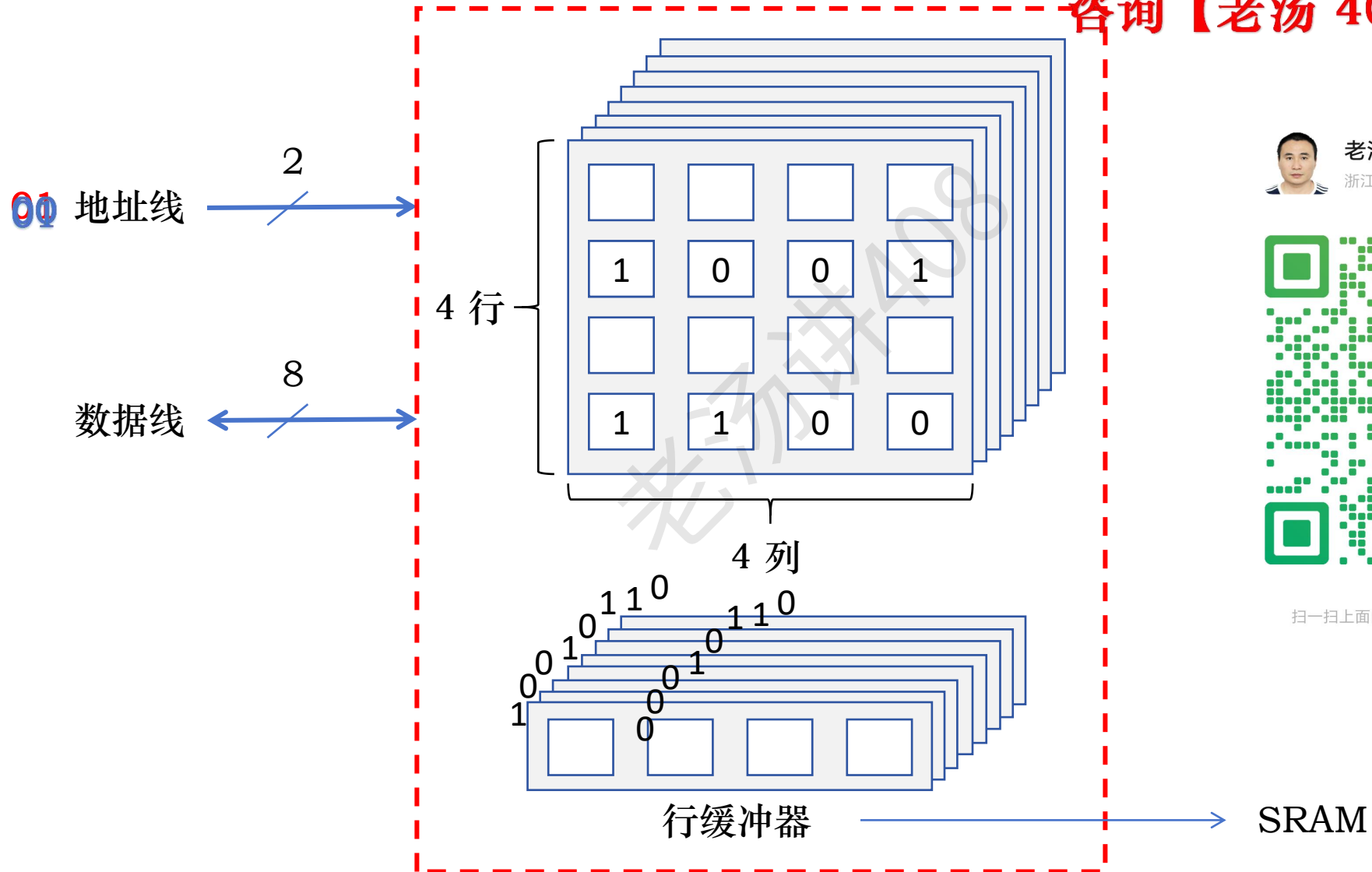
主存大小: 16×8 位



同步 DRAM (Synchronous DRAM, SDRAM) : 行缓冲器

主存大小: 16×8 位

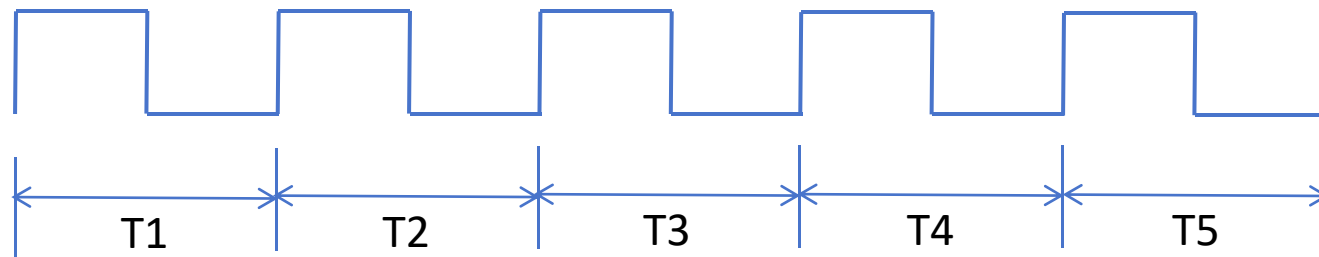
加老汤微信, 帮你 408 冲 120+
咨询【老汤 408 高分特训营】



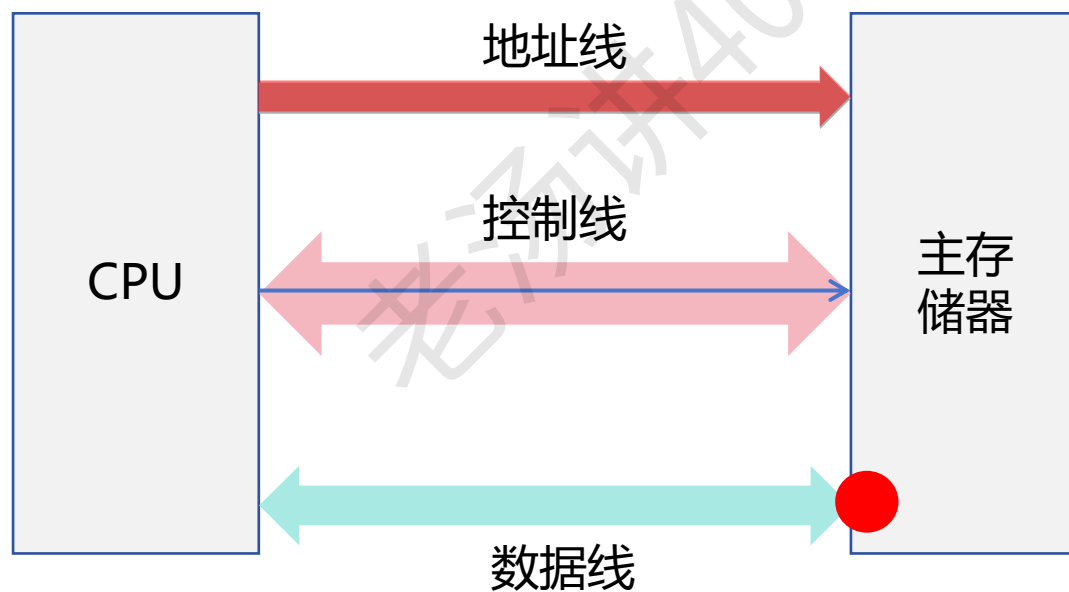
扫一扫上面的二维码图案, 加我为朋友。

同步 DRAM (Synchronous DRAM, SDRAM) : 突发传输

存储总线时钟信号:

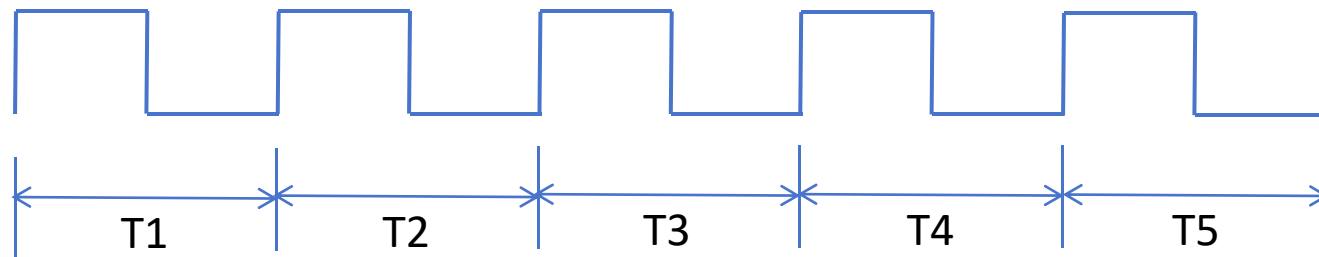


存储器总线

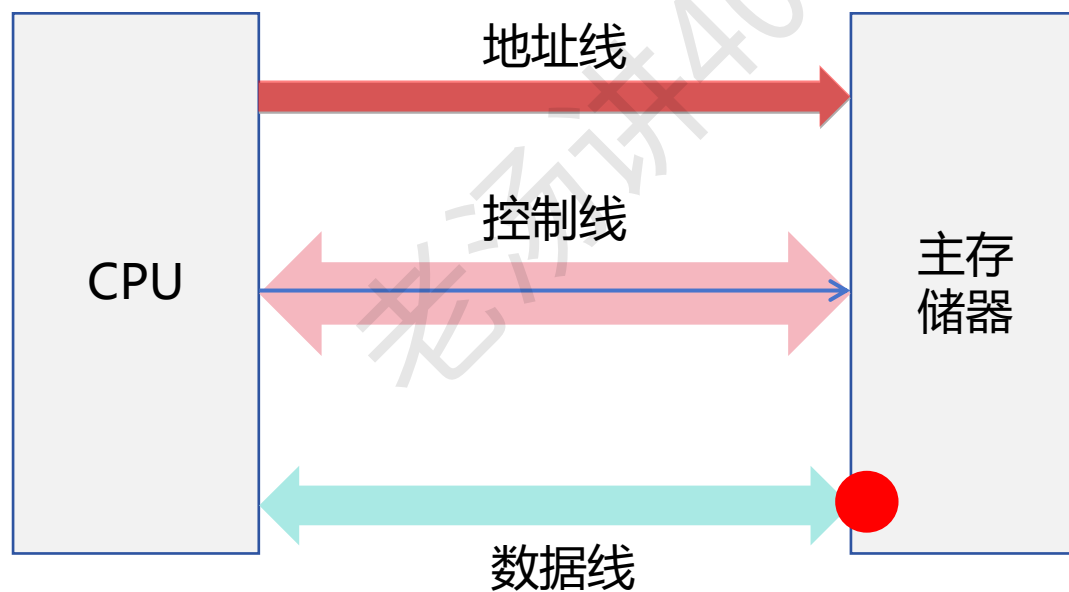


同步 DRAM (Synchronous DRAM, SDRAM) : 突发传输

存储总线时钟信号:

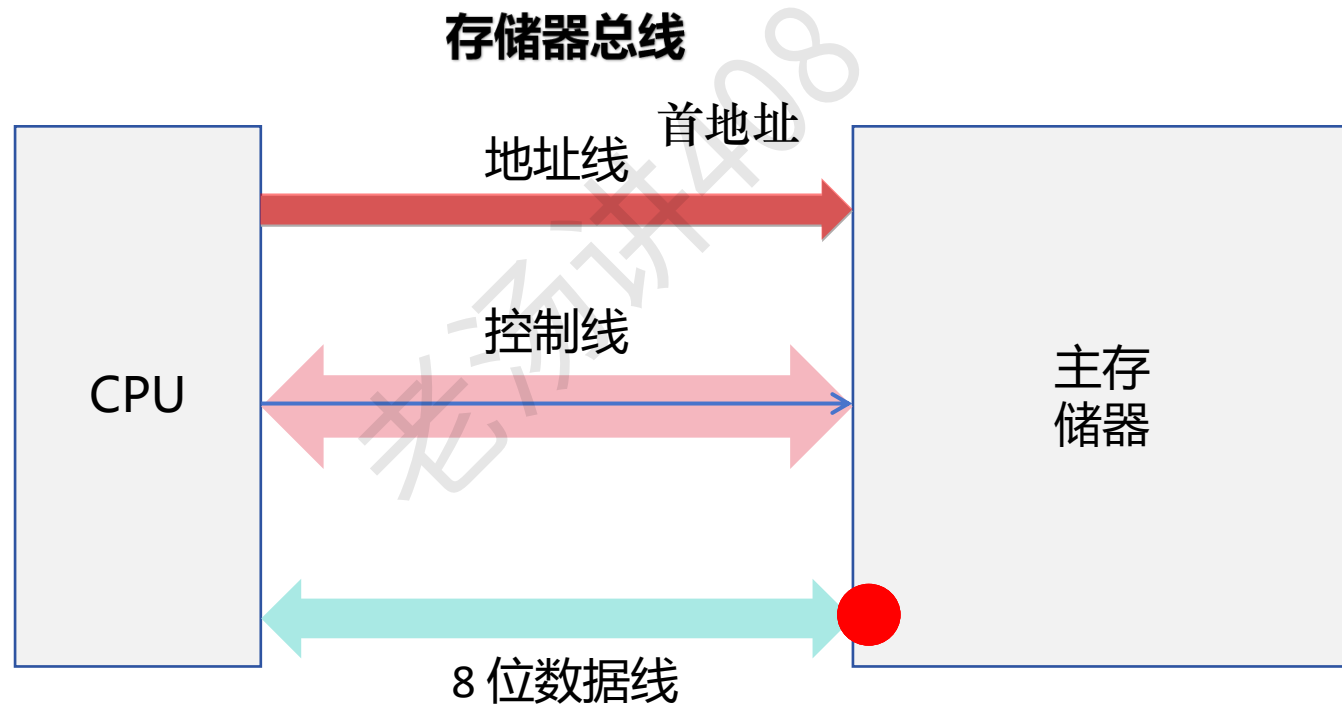
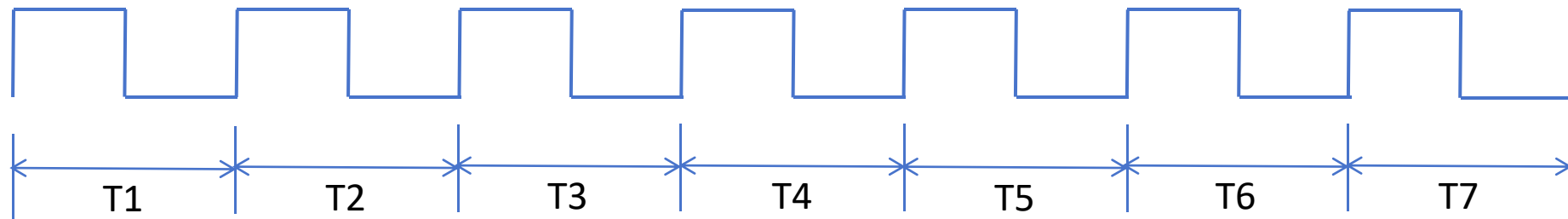


存储器总线

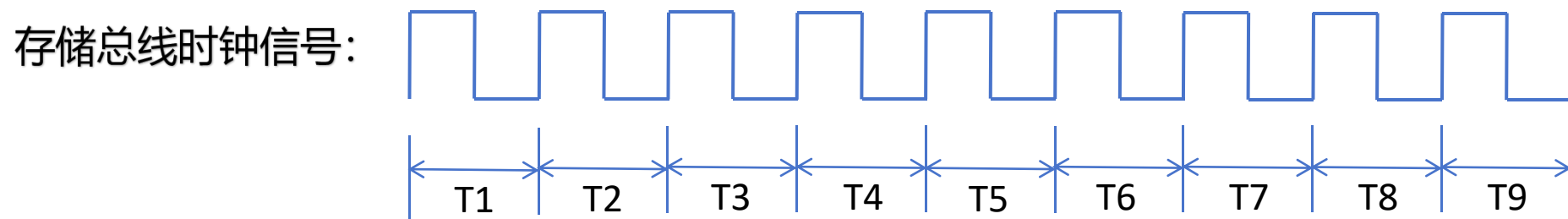


同步 DRAM (Synchronous DRAM, SDRAM) : 突发传输

存储总线时钟信号:



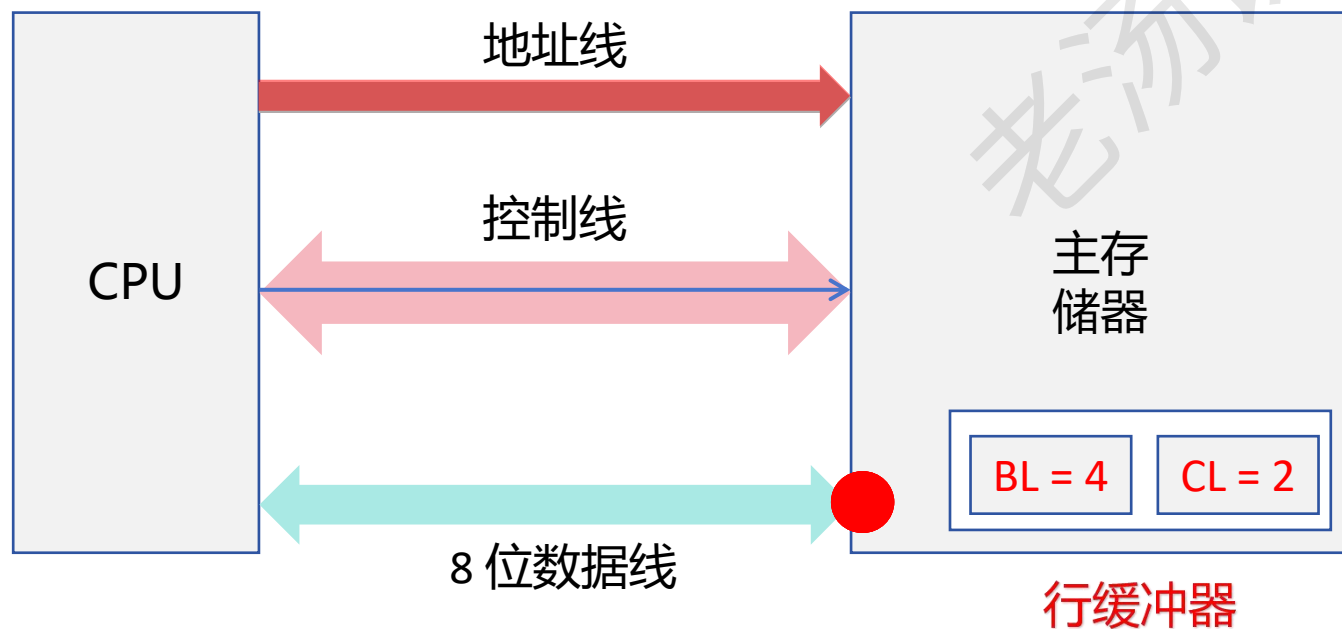
同步 DRAM (Synchronous DRAM, SDRAM) : 突发传输



CS (片选信号) CAS (列地址信号)

RAS (行地址信号) 读命令信号

存储器总线

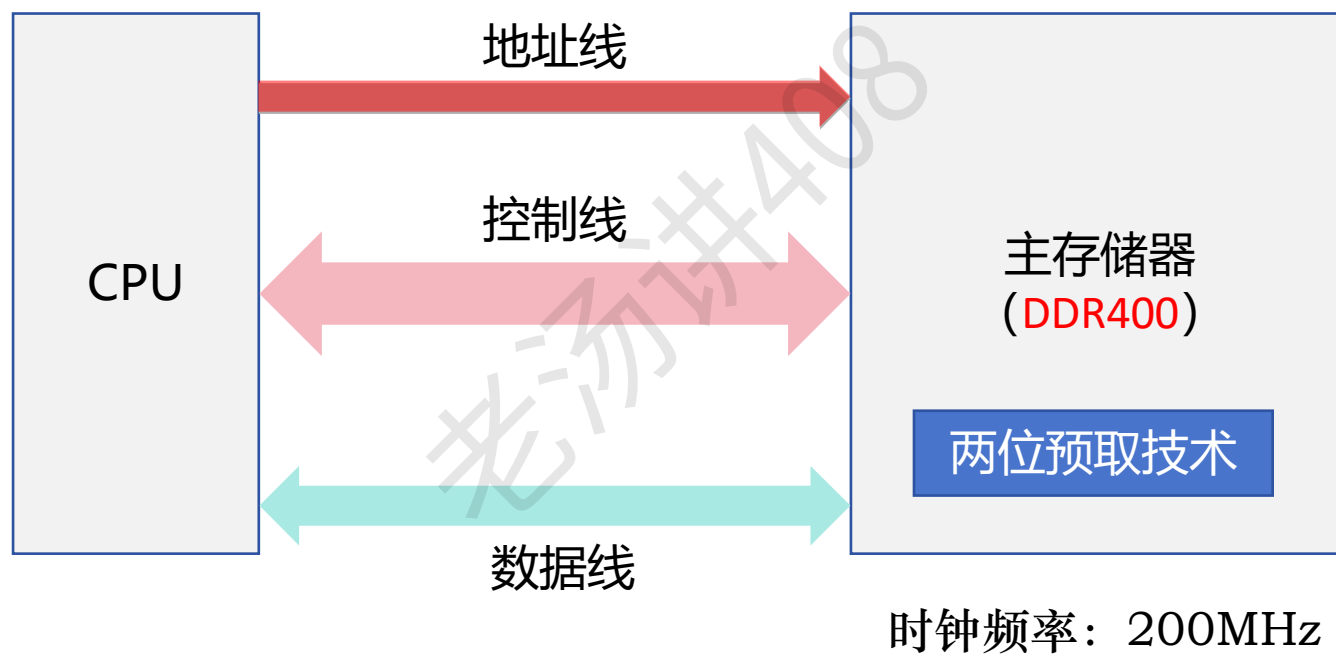


BL: 突发长度 (Burst Length)

CL: CAS (列地址信号) 潜伏期 (CAS Latency)

DDR (Double Data Rate) SDRAM

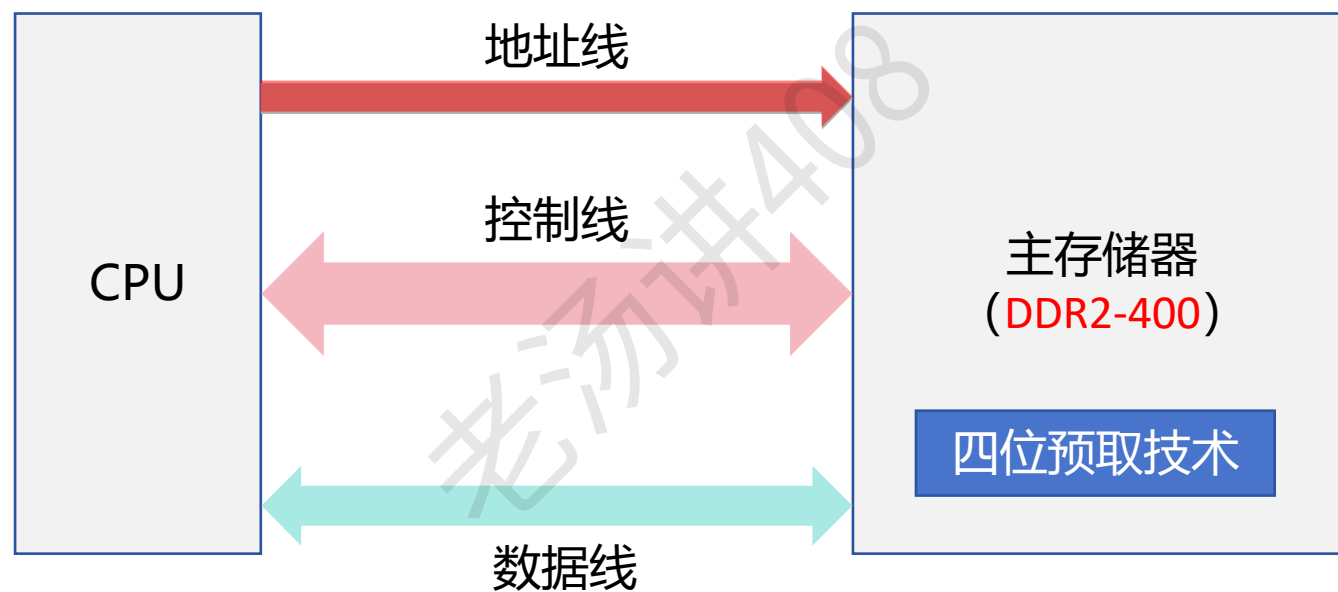
存储器总线 时钟频率：200MHz 工作频率：400MHz



存储器总线的带宽：400MHz × 64 / 8 = 3.2GB/s

DDR (Double Data Rate) SDRAM

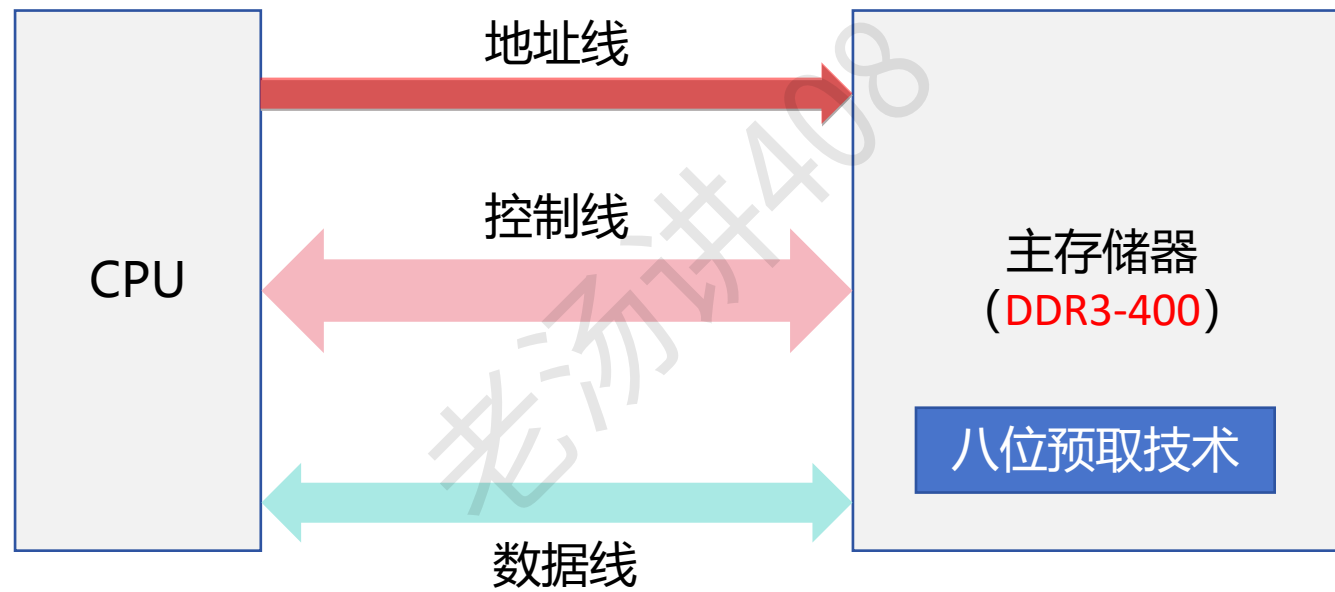
存储器总线 时钟频率：400MHz 工作频率：800MHz



存储器总线的带宽： $800\text{MHz} \times 64 / 8 = 6.4\text{GB/s}$

DDR (Double Data Rate) SDRAM

存储器总线 时钟频率：800MHz 工作频率：1600MHz



存储器总线的带宽：1600MHz × 64 / 8 = 12.8GB/s

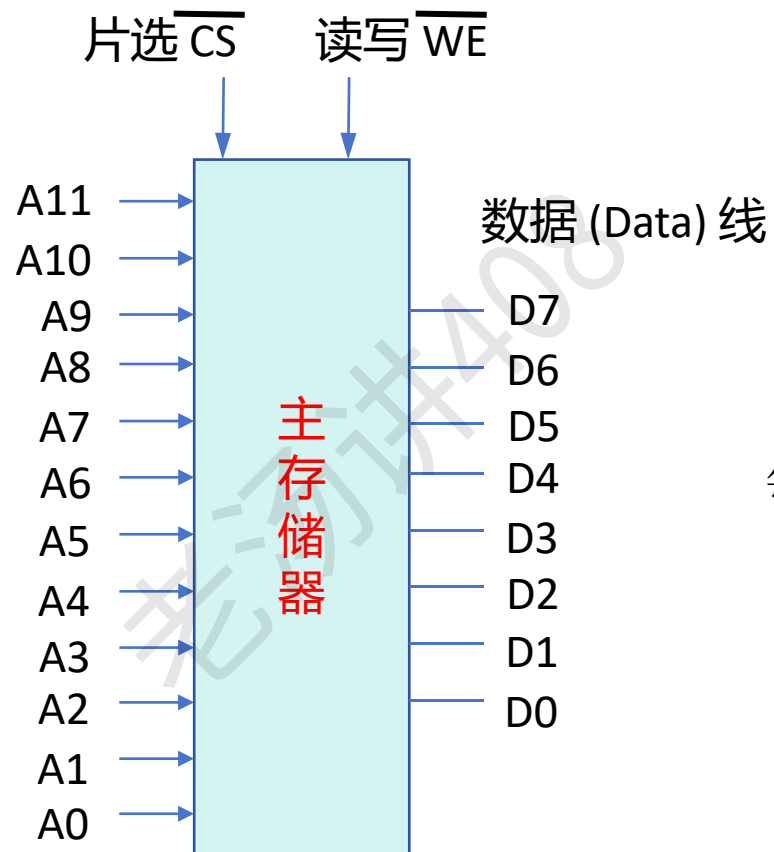
DDR (Double Data Rate) SDRAM

内存类型	工作频率范围	数据传输速率范围	带宽范围	工作电压
DDR	100MHz-200MHz	1.6GB/s-3.2GB/s	1.6GB/s-3.2GB/s	2.5V/2.6V
DDR2	400MHz-800MHz	4.2GB/s-6.4GB/s	4.2GB/s-6.4GB/s	1.8V
DDR3	800MHz-1600MHz	8.5GB/s-14.9GB/s	8.5GB/s-14.9GB/s	1.5V/1.35V
DDR4	1600MHz-3200MHz	17GB/s-25.6GB/s	17GB/s-25.6GB/s	1.2V
DDR5	4800MHz 起步	理论可达 64GB/s 以上	理论可达 64GB/s 以上	1.1V

按字节寻址 vs 按字寻址

$$2^{12} = 4096 \text{ 个地址}$$

地址 (Address) 线

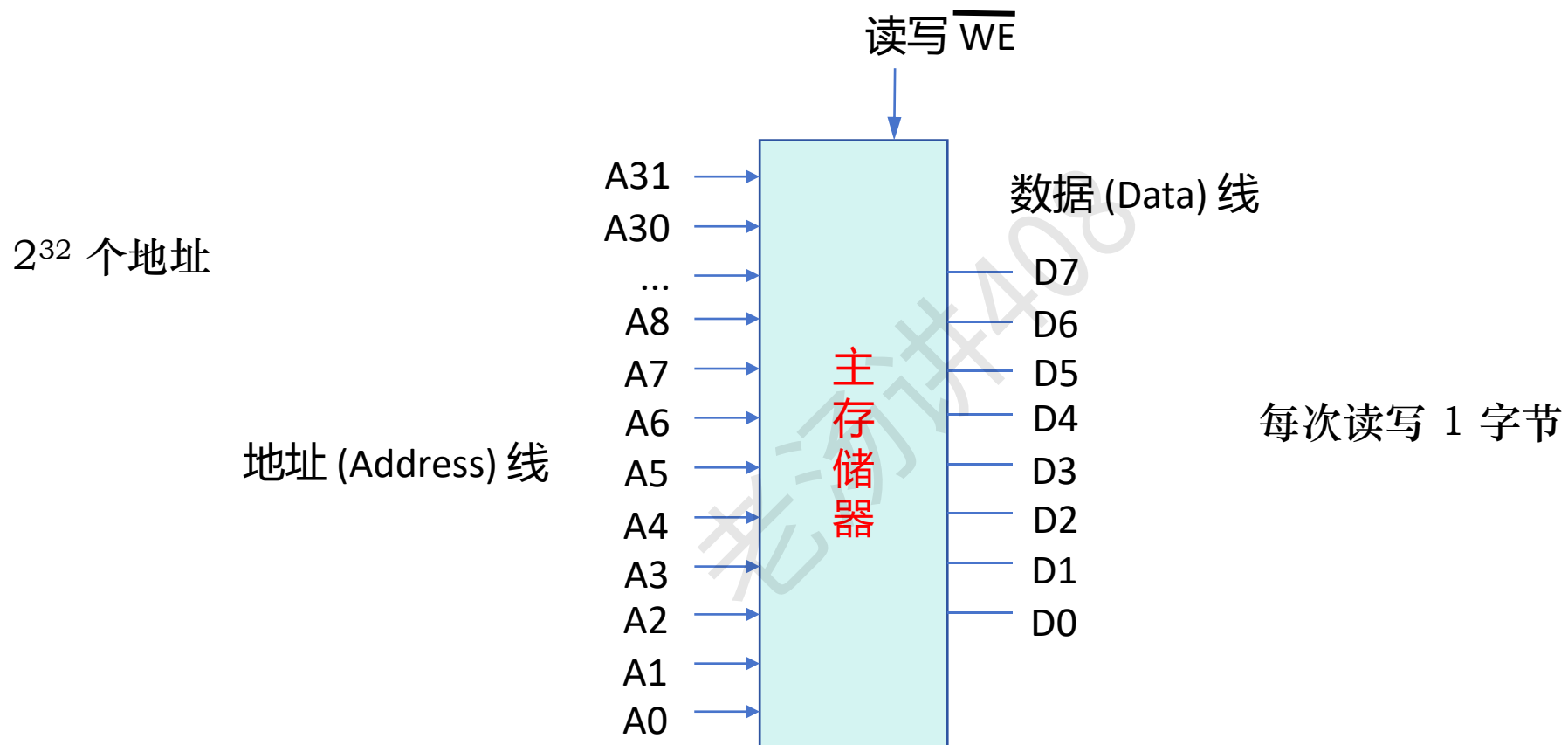


数据 (Data) 线

每次读写 1 字节

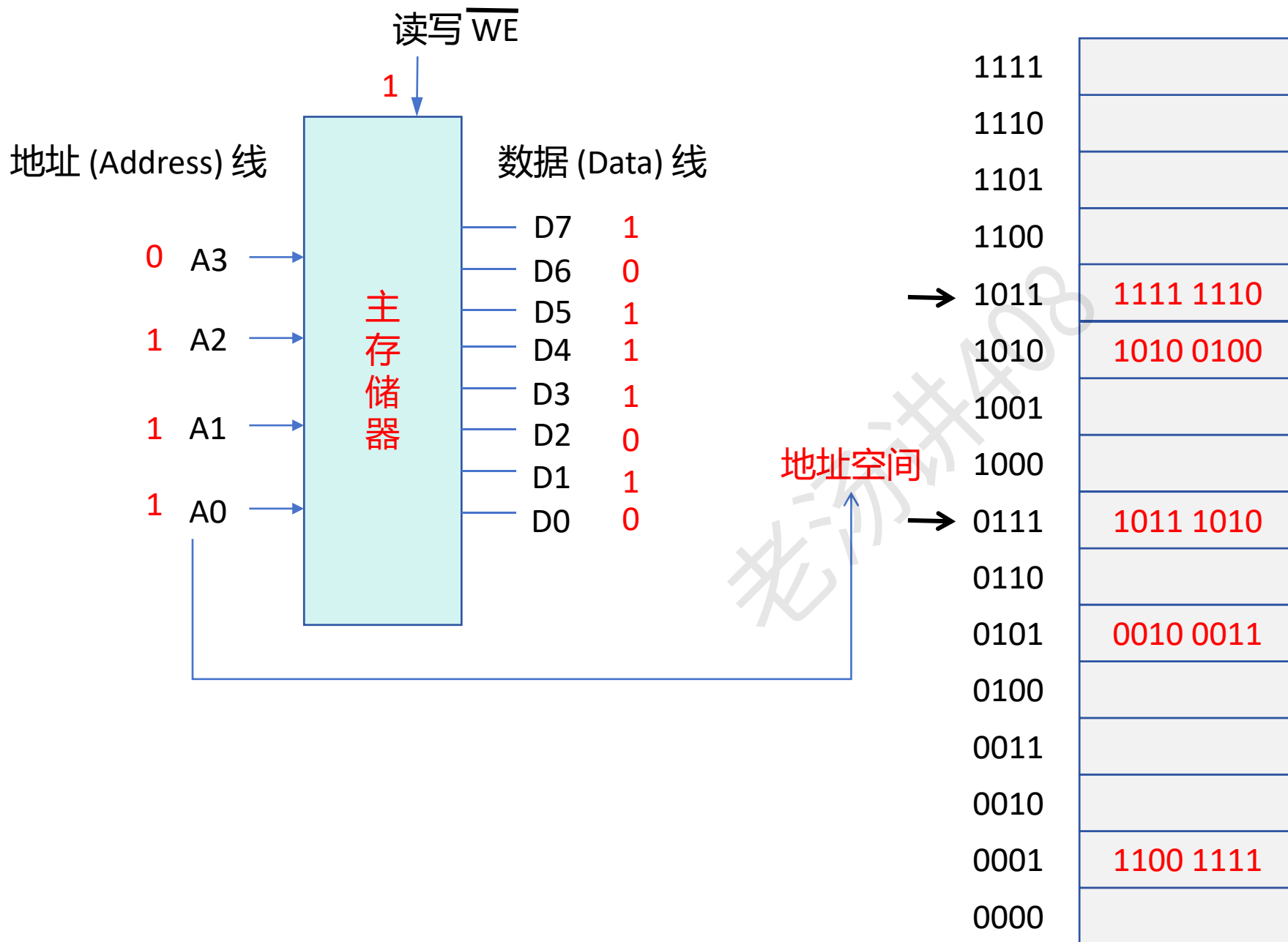
$$\text{容量: } 4096 \times 1 \text{ B} = 4096 \text{ B} = 4\text{KB}$$

按字节寻址 vs 按字寻址

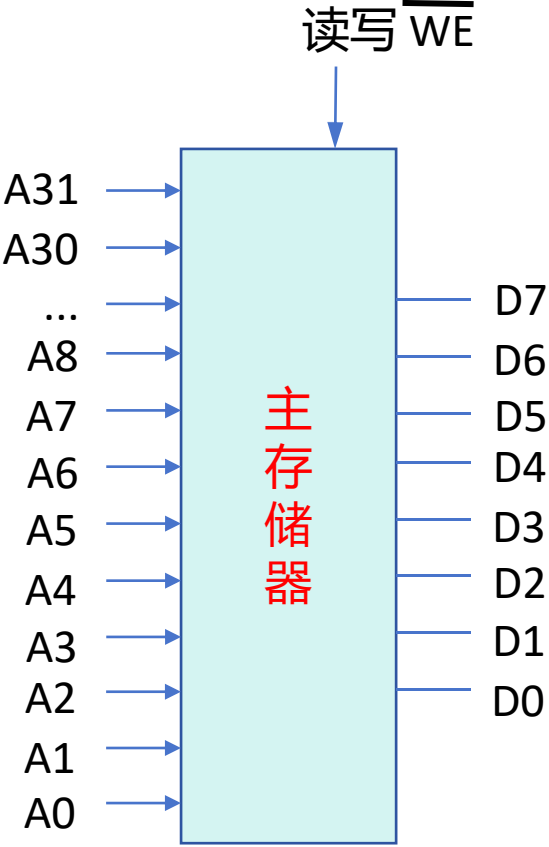


$$\text{容量: } 2^{32} \times 1 \text{ B} = 2^2 * 2^{10} * 2^{10} * 2^{10} \text{ B} = 4 \text{ GB}$$

按字节寻址 vs 按字寻址



按字节寻址 vs 按字寻址



1111 1111 1111 1111 1111 1111 1111 1111	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
1111 1111 1111 1111 1111 1111 1111 1111	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
1111 1111 1111 1111 1111 1111 1111 1111	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
1111 1111 1111 1111 1111 1111 1111 1111	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
1111 1111 1111 1111 1111 1111 1111 1111	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	1111 1110
.....			
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	1011 1010
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0010 0011
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	1100 1111
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	0000 0000 0000 0000 0000 0000 0000 0000	

按字节寻址 vs 按字寻址

高地址

低地址



0xFFFFFFFF	
0xFFFFFFFFE	
0xFFFFFFFFD	
0xFFFFFFFFC	
0xFFFFFFFFB	1111 1110
.....	
0x00000009	
0x00000008	
0x00000007	1011 1010
0x00000006	
0x00000005	0010 0011
0x00000004	
0x00000003	
0x00000002	
0x00000001	1100 1111
0x00000000	

低地址

高地址



0x00000000	
0x00000001	
0x00000002	
0x00000003	
0x00000004	1111 1110
.....	
0xFFFFFFFF6	
0xFFFFFFFF7	
0xFFFFFFFF8	1011 1010
0xFFFFFFFF9	
0xFFFFFFF9A	0010 0011
0xFFFFFFF9B	
0xFFFFFFF9C	
0xFFFFFFF9D	
0xFFFFFFF9E	1100 1111
0xFFFFFFF9F	

按字节寻址 vs 按字寻址

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

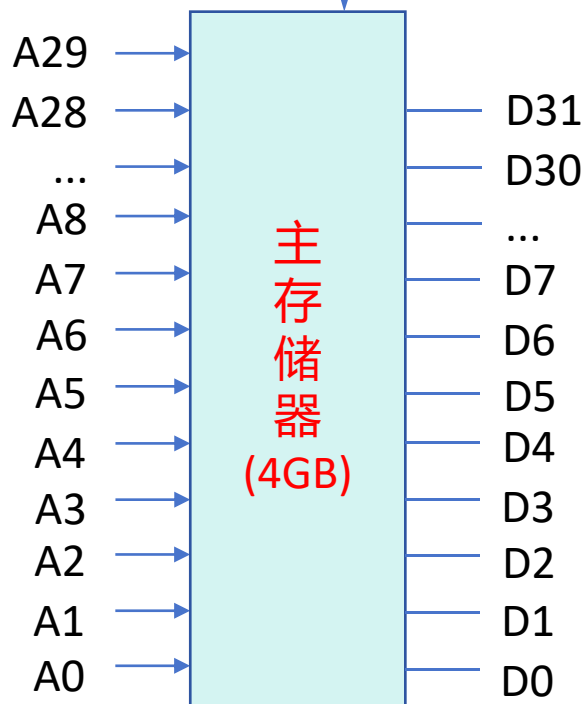


扫一扫上面的二维码图案，加我为朋友。

0xFFFFFFFF	
0xFFFFFFFFE	
0xFFFFFFFFD	
0xFFFFFFFFC	
0xFFFFFFFFB	1111 1110
.....	
0x00000009	
0x00000008	
0x00000007	1011 1010
0x00000006	
0x00000005	0010 0011
0x00000004	
0x00000003	
0x00000002	
0x00000001	1100 1111
0x00000000	

存储字大小：1 字节

读写 $\overline{WE}$



每次读写 4 字节

$4\text{GB}/4\text{B} = 1\text{G}$ 个存储字

$1\text{G} = 2^{10} * 2^{10} * 2^{10} = 2^{30}$

按字节寻址 vs 按字寻址

按字寻址的地址空间

11 1111 1111 1111 1111 1111 10k3EFFFFFFF
11 1111 1111 1111 1111 1111 10k3EFFFFFFE
11 1111 1111 1111 1111 1111 10k3EFFFFFFD
11 1111 1111 1111 1111 1111 10k3EFFFFFFC
11 1111 1111 1111 1111 1111 10k3EFFFFFFB
.....
00 0000 0000 0000 0000 0000 x00000009
00 0000 0000 0000 0000 0000 x00000008
00 0000 0000 0000 0000 0000 x00000007
00 0000 0000 0000 0000 0000 x00000006
00 0000 0000 0000 0000 0000 x00000005
00 0000 0000 0000 0000 0000 x00000004
00 0000 0000 0000 0000 0000 x00000003
00 0000 0000 0000 0000 0000 x00000002
00 0000 0000 0000 0000 0000 x00000001
00 0000 0000 0000 0000 0000 x00000000

1111 1110 1110 0001 1111 0101 0000 1100

.....

0001 1110 1110 0101 1011 0101 1100 1100

1111 1000 1110 0001 1111 0101 0011 1100

1100 1111 1100 1111 1100 1111 1100 1111

按字节寻址 vs 按字寻址

存储字大小是 4 B

按字寻址

地址的位数是 10 位

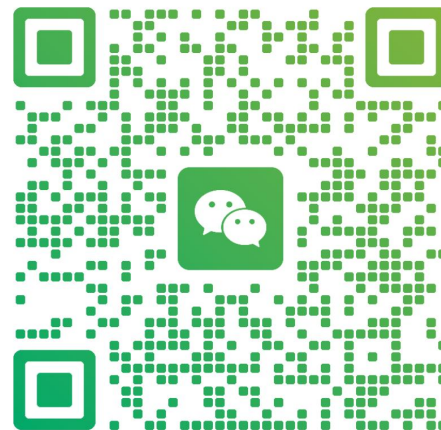
$$2^{10} * 4B = 4KB$$

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

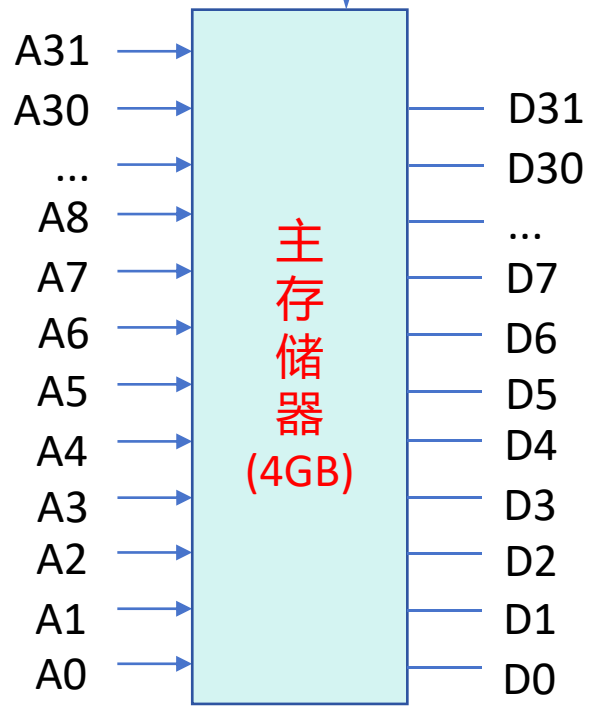
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

按字节寻址 vs 按字寻址

读写 $\overline{WE}$



存储字大小: 4 字节

$4GB / 1B = 4G$ 个地址

30 位

PC

1111 1111 1111 1111 1111 1111 1111 1100	
1111 1111 1111 1111 1111 1111 1111 1000	
1111 1111 1111 1111 1111 1111 1111 0100	
1111 1111 1111 1111 1111 1111 1111 0000	
1111 1111 1111 1111 1111 1111 1110 1100	1111 1110 1110 0001 1111 0101 0000 1100
.....	
0000 0000 0000 0000 0000 0000 0010 0100	
0000 0000 0000 0000 0000 0000 0010 0000	
0000 0000 0000 0000 0000 0000 0001 1100	0001 1110 1110 0101 1011 0101 1100 1100
0000 0000 0000 0000 0000 0000 0001 1000	
0000 0000 0000 0000 0000 0000 0001 0100	1111 1000 1110 0001 1111 0101 0011 1100
0000 0000 0000 0000 0000 0000 0001 0000	
0000 0000 0000 0000 0000 0000 0000 1100	
0000 0000 0000 0000 0000 0000 0000 1000	
0000 0000 0000 0000 0000 0000 0000 0100	1100 1111 1100 1111 1100 1111 1100 1111
0000 0000 0000 0000 0000 0000 0000 0000	1100 1111 1001 1001 0000 0001 0010 0101

按字节寻址 vs 按字寻址

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```



低地址

高地址



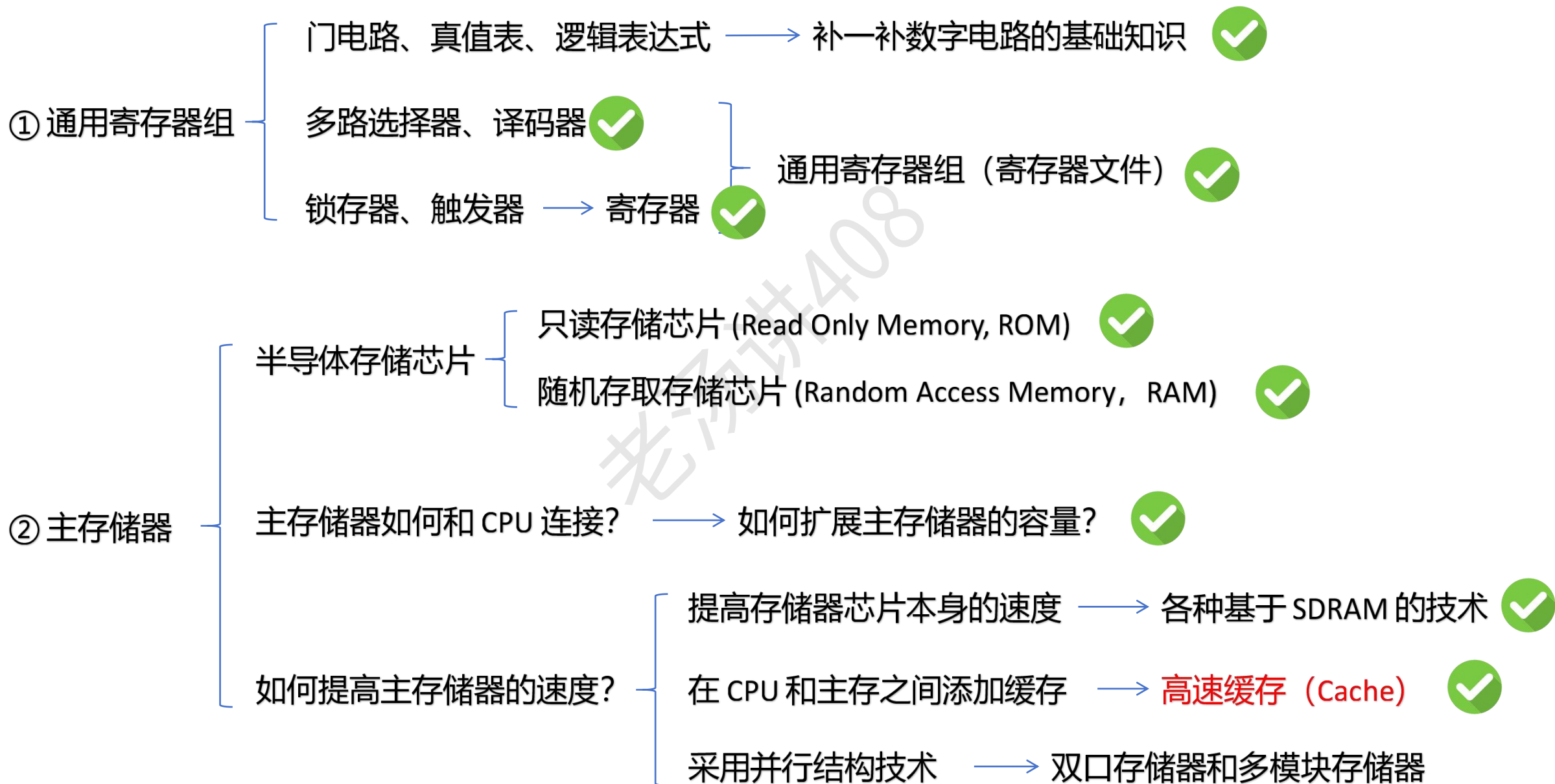
....
0x100
0x104
0x108
0x10C
0x110
0x114
0x118
0x11C
....

主存储器	
...	...
0x100	mov \$0 %R1
0x104	jmp 0x114
0x108	addq (%R2), R1
0x10C	addq \$8, %R2
0x110	subq \$1, %R3
0x114	testq %R3, %R3
0x118	jne 0x108
0x11C	ret
....	

存储字：4 字节

按字节寻址

【第三章：主存储器】内容



高速缓存 (Cache) 主要内容

① 为什么需要 Cache? (局部性原理)

② Cache 的基本工作原理

③ Cache 行和主存块的三种映射方式

直接映射

组相联映射

全相联映射

④ Cache 行的替换算法

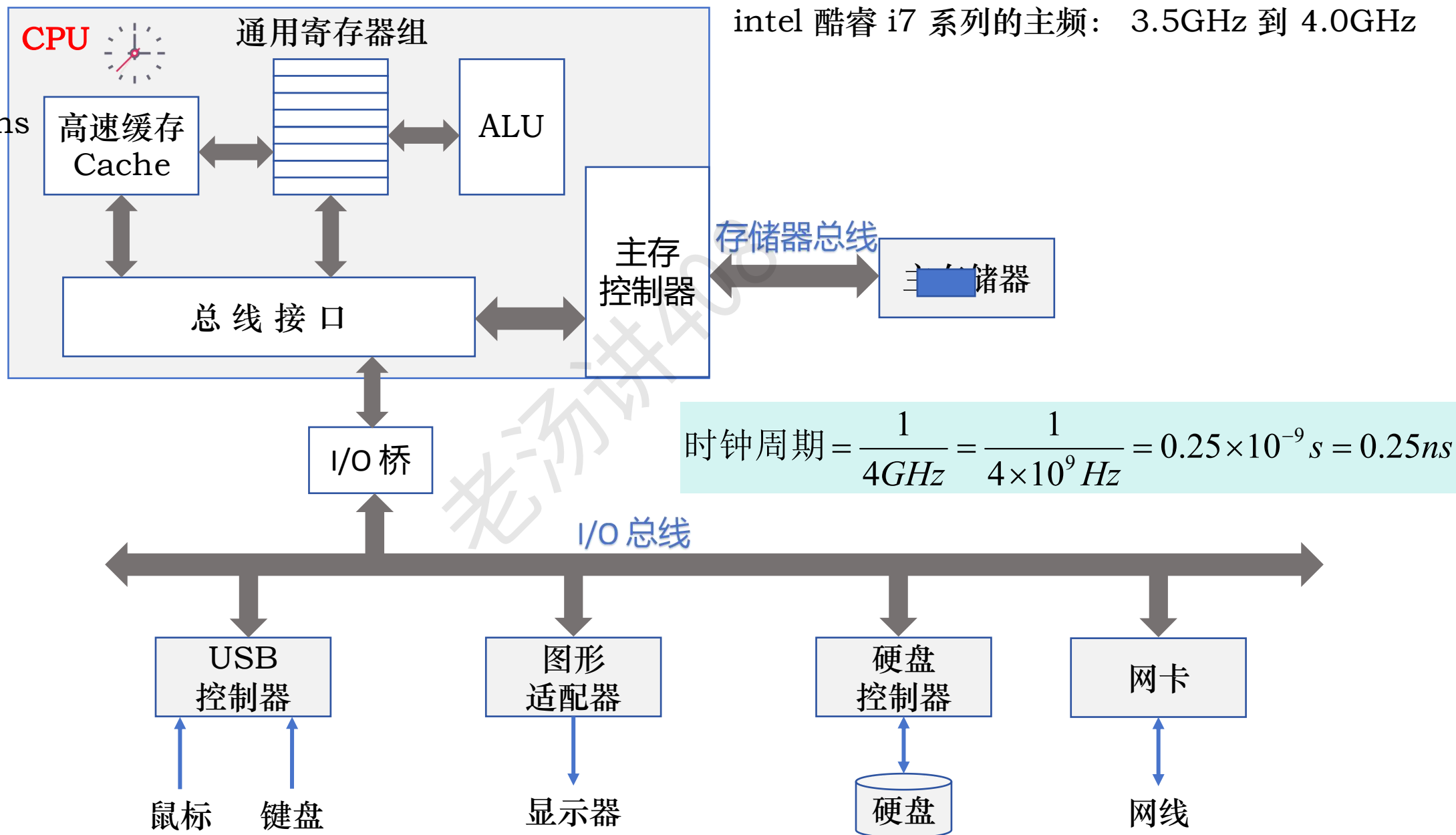
④ Cache 的数据一致性问题

为什么需要 Cache ?

主频: 4.0 GHz

时钟周期: 0.25ns

intel 酷睿 i7 系列的主频: 3.5GHz 到 4.0GHz



为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```

主存储器

	...
0x100	mov \$0 %R1
0x104	jmp 0x114
0x108	addq (%R2), R1
0x10C	addq \$8, %R2
0x110	subq \$1, %R3
0x114	testq %R3, %R3
0x118	jne 0x108
0x11C	ret
	

循环

res 暂存在寄存器 R1 中

数组首地址 start 暂存在寄存器 R2 中

count 暂存在寄存器 R3 中

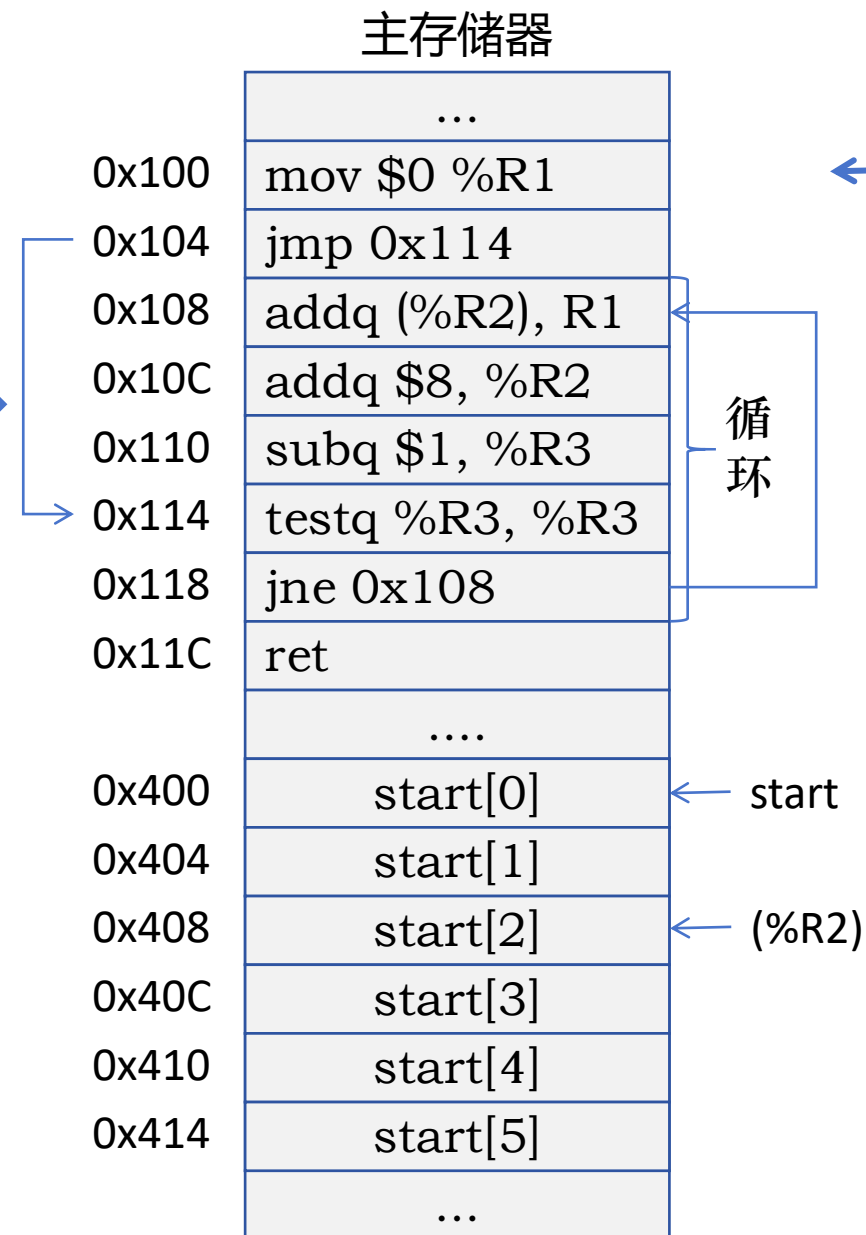
为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```

res 暂存在寄存器 R1 中

R2 = 0x408 数组首地址 start 暂存在寄存器 R2 中

count 暂存在寄存器 R3 中



为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```

CPU

PC: 0x114

主存储器

...	
0x100	mov \$0 %R1
0x104	jmp 0x114
0x108	addq (%R2), R1
0x10C	addq \$8, %R2
0x110	subq \$1, %R3
0x114	testq %R3, %R3
0x118	jne 0x108
0x11C	ret
....	
0x400	start[0]
0x404	start[1]
0x408	start[2]
0x40C	start[3]
0x410	start[4]
0x414	start[5]
...	

循环

start

为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```

CPU

PC: 0x114

主存储器

...	
0x100	mov \$0 %R1
0x104	jmp 0x114
0x108	addq (%R2), R1
0x10C	addq \$8, %R2
0x110	subq \$1, %R3
0x114	testq %R3, %R3
0x118	jne 0x108
0x11C	ret
....	
0x400	start[0]
0x404	start[1]
0x408	start[2]
0x40C	start[3]
0x410	start[4]
0x414	start[5]
...	

循环

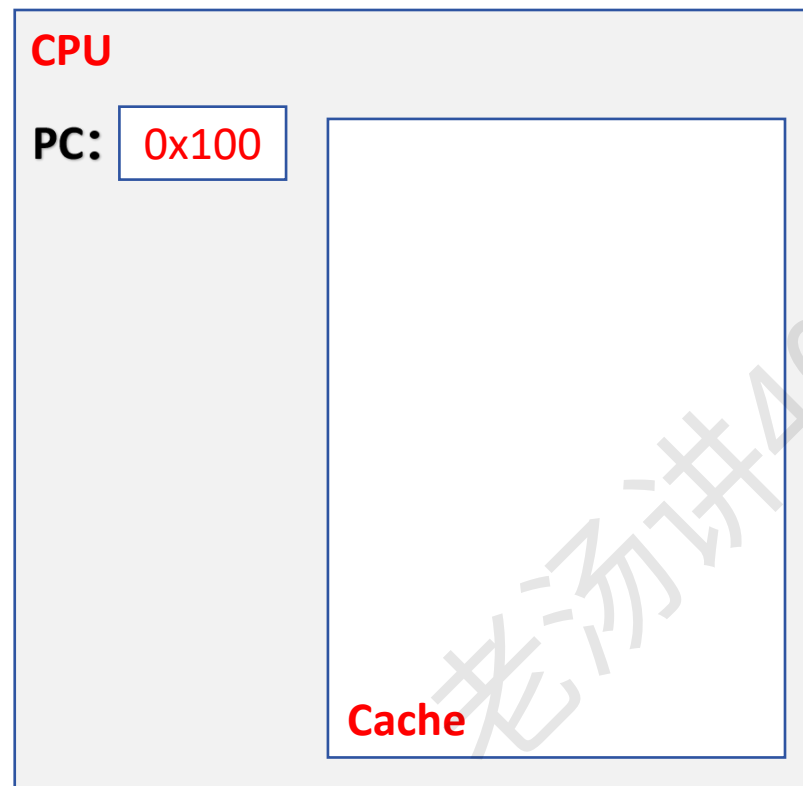
start

在主存中的某一条指令刚被访问了，在很短的时间内再次会被访问

时间局部性 指被访问的某个存储单元在一个较短的时间间隔内很可能又被访问

为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```



突发传输

0x100

0x104

0x108

0x10C

0x110

0x114

0x118

0x11C

0x400

0x404

0x408

0x40C

0x410

0x414

主存储器

...

mov \$0 %R1

jmp 0x114

addq (%R2), R1

addq \$8, %R2

subq \$1, %R3

testq %R3, %R3

jne 0x108

ret

....

start[0]

start[1]

start[2]

start[3]

start[4]

start[5]

...

循环

start

为什么需要 Cache ? - 局部性原理

```
long sum(int *start, int count) {
```

```
    long res = 0;
```

```
    while (count) {
```

```
        res += *start;
```

```
        start++;
```

```
        count--;
```

```
    }
```

```
    return res;
```

```
}
```

CPU

PC: 0x100

mov \$0 %R1

jmp 0x114

addq (%R2), R1

addq \$8, %R2

subq \$1, %R3

testq %R3, %R3

jne 0x108

ret

Cache

突发传输

主存储器

0x100

mov \$0 %R1

0x104

jmp 0x114

0x108

addq (%R2), R1

0x10C

addq \$8, %R2

0x110

subq \$1, %R3

0x114

testq %R3, %R3

0x118

jne 0x108

0x11C

ret

....

0x400

start[0]

0x404

start[1]

0x408

start[2]

0x40C

start[3]

0x410

start[4]

0x414

start[5]

...

循环

start

被访问的某个存储单元的
邻近单元在一个较短时间
间隔内很可能也被访问

空间局部性

对数组 start 的访问程序的时间局部性很差

为什么需要 Cache ? - 局部性原理

程序一：按行访问二维数组 A

```
long sum_array_rows(int A[M][N]) {  
    int i, j, res = 0;  
    for (i = 0; i < M; i++)  
        for (j = 0; j < N; j++)  
            res += A[i][j];  
    return res;  
}
```

代码指令：

时间局部性好

空间局部性好

程序二：按列访问二维数组 A

```
long sum_array_cols(int A[M][N]) {  
    int i, j, res = 0;  
    for (j = 0; j < N; j++)  
        for (i = 0; i < M; i++)  
            res += A[i][j];  
    return res;  
}
```

主存储器

	...
0x100	mov \$0 %R1
0x104	
0x108	for 内/外循环 代码指令
0x10C	
0x110	
0x114	
0x118	jne 0x108
0x11C	ret
	

循环

为什么需要 Cache ? - 局部性原理

程序一：按行访问二维数组 A

```
long sum_array_rows(int A[M][N]) {
    int i, j, res = 0;
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            res += A[i][j];
    return res;
}
```

数组 A 的访问:

程序一空间局部性好

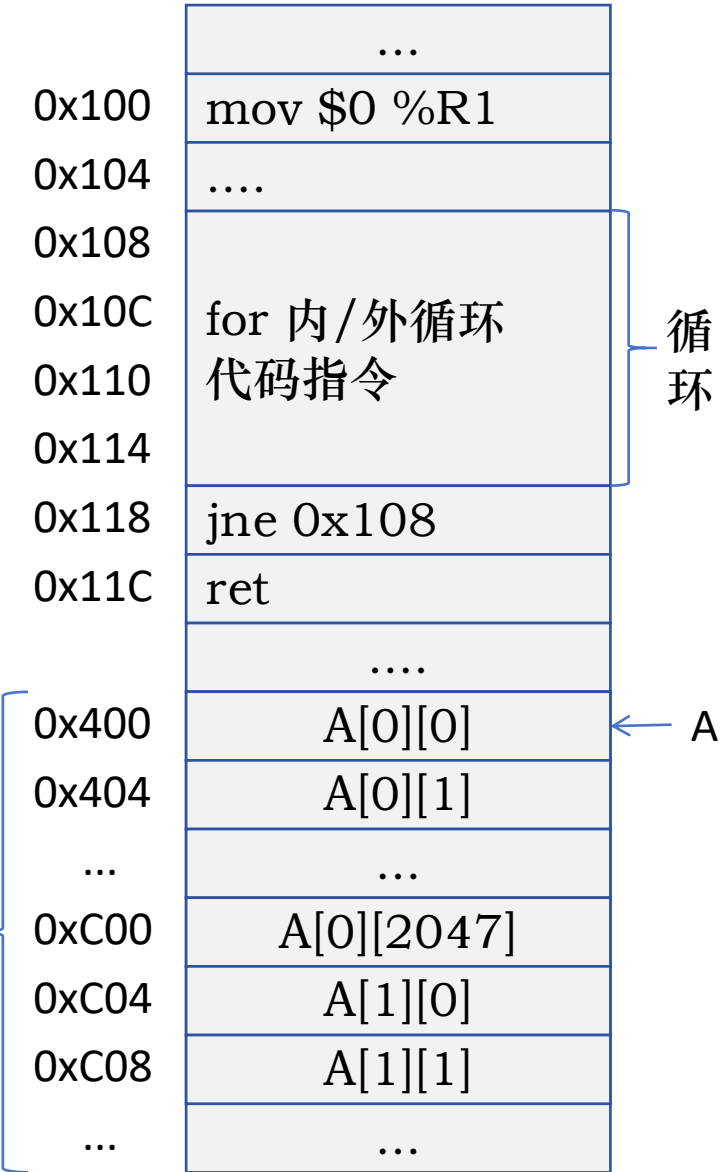
程序二：按列访问二维数组 A

```
long sum_array_cols(int A[M][N]) {
    int i, j, res = 0;
    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            res += A[i][j];
    return res;
}
```

程序二空间局部性不好

按行存储

主存储器



为什么需要 Cache ? - 局部性原理

程序一：按行访问二维数组 A

```
long sum_array_rows(int A[M][N]) {
    int i, j, res = 0;
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            res += A[i][j];
    return res;
}
```

变量 res 的访问：

时间局部性好

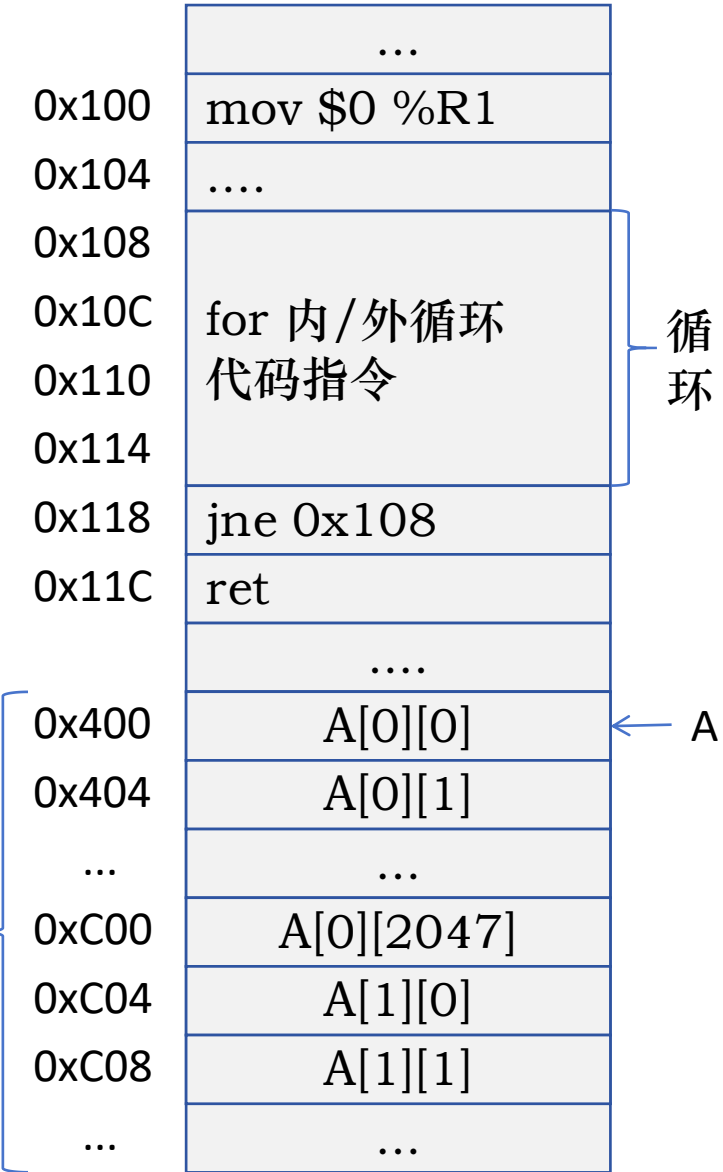
空间局部性没意义

程序二：按列访问二维数组 A

```
long sum_array_cols(int A[M][N]) {
    int i, j, res = 0;
    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            res += A[i][j];
    return res;
}
```

按行存储

主存储器



高速缓存 (Cache) 主要内容

① 为什么需要 Cache? (局部性原理)



② Cache 的基本工作原理



**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

③ Cache 行和主存块的三种映射方式

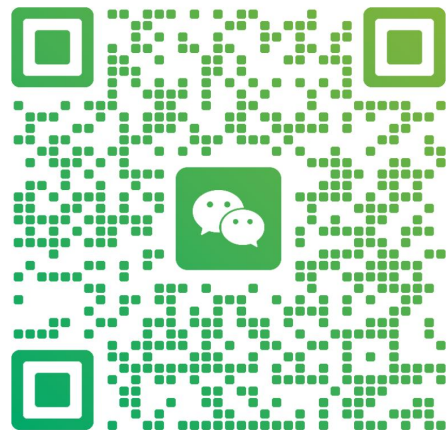
直接映射

组相联映射

全相联映射

④ Cache 行的替换算法

④ Cache 的数据一致性问题



扫一扫上面的二维码图案，加我为朋友。

Cache 的基本工作原理

```
long sum(int *start, int count) {  
    long res = 0;  
    while (count) {  
        res += *start;  
        start++;  
        count--;  
    }  
    return res;  
}
```

res 暂存在寄存器 R1 中

数组首地址 start 暂存在寄存器 R2 中

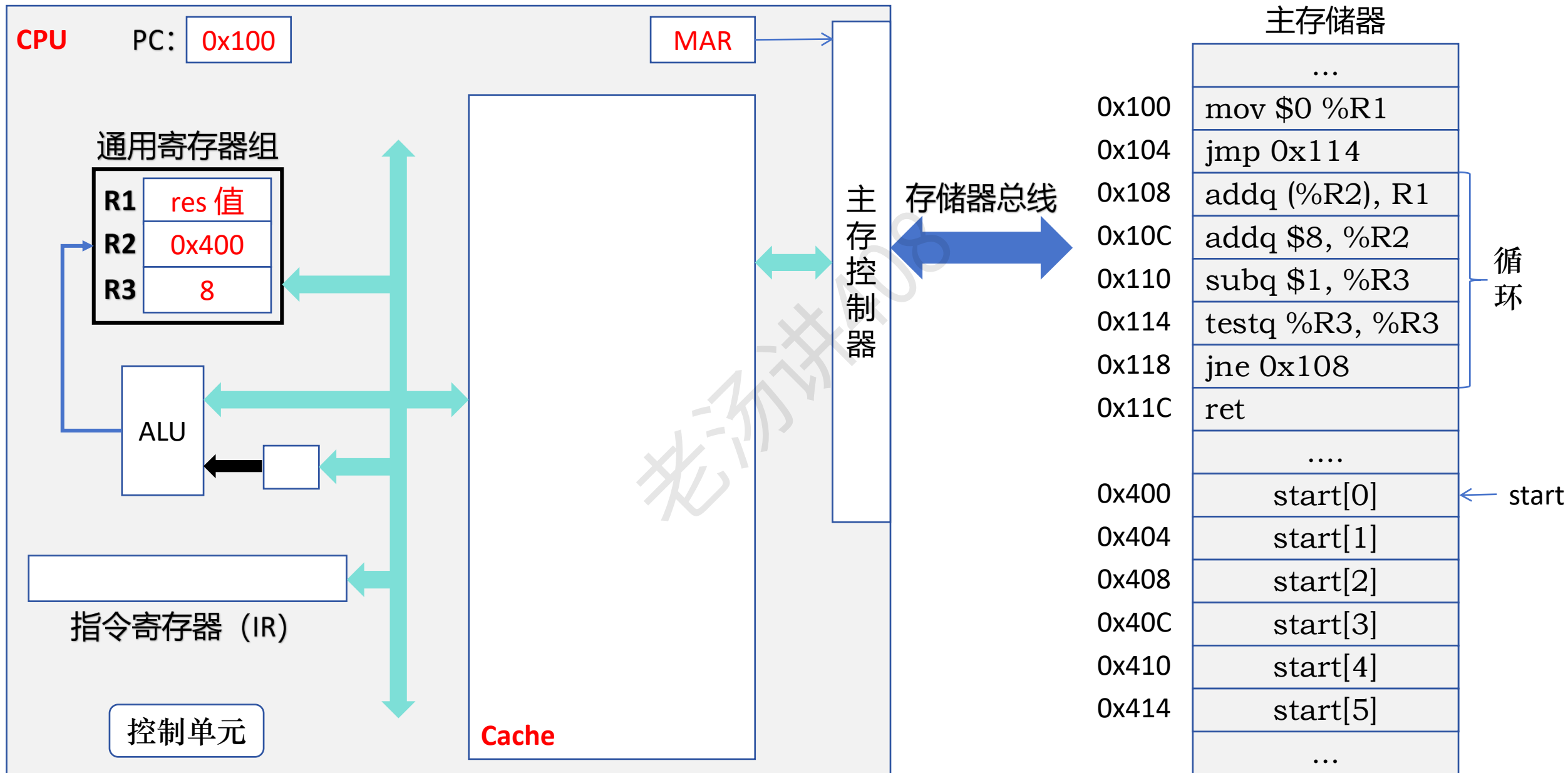
count 暂存在寄存器 R3 中

主存储器	
	...
0x100	mov \$0 %R1
0x104	jmp 0x114
0x108	addq (%R2), R1
0x10C	addq \$8, %R2
0x110	subq \$1, %R3
0x114	testq %R3, %R3
0x118	jne 0x108
0x11C	ret
	
0x400	start[0]
0x404	start[1]
0x408	start[2]
0x40C	start[3]
0x410	start[4]
0x414	start[5]
	...

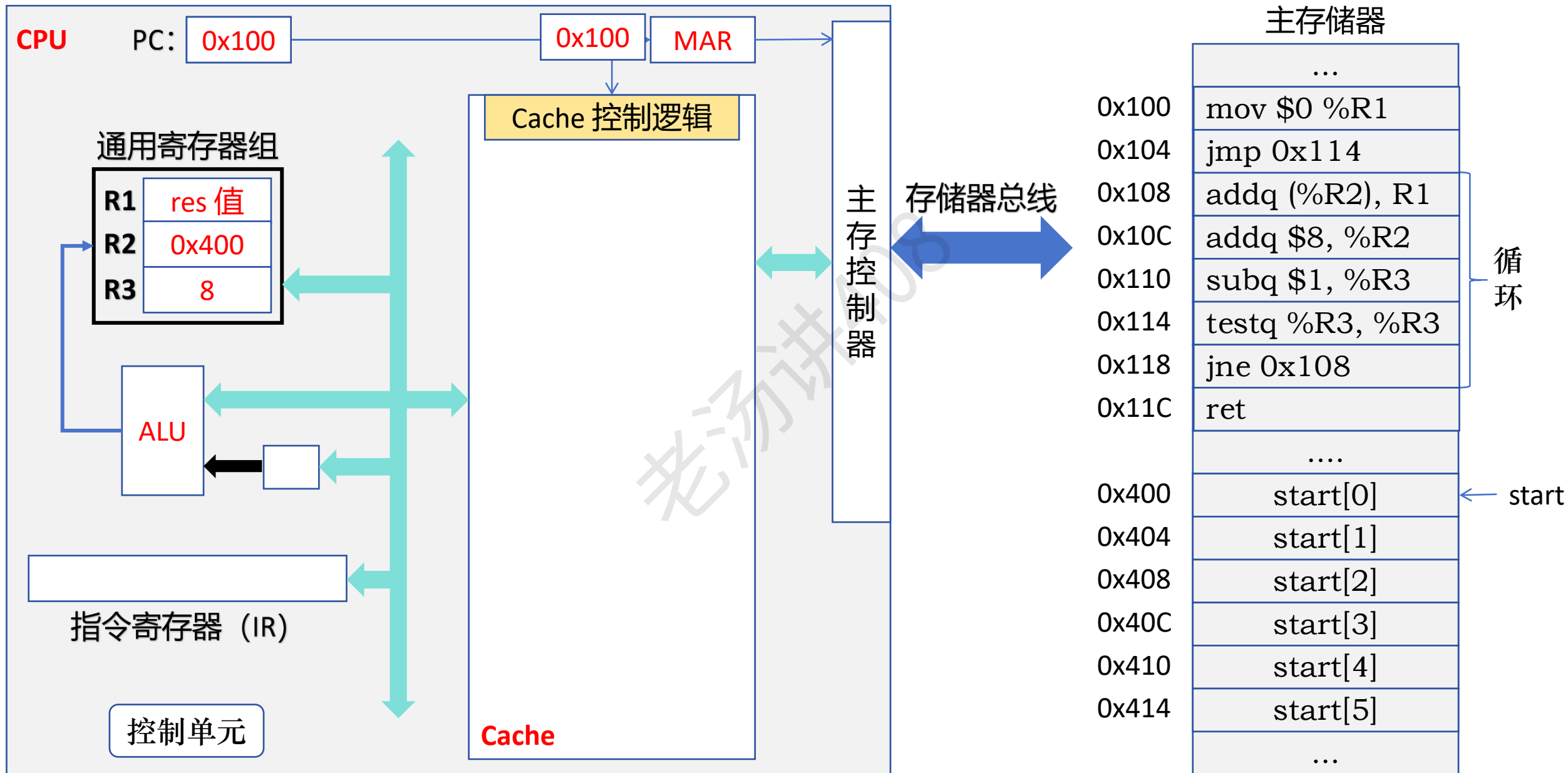
循环

← start

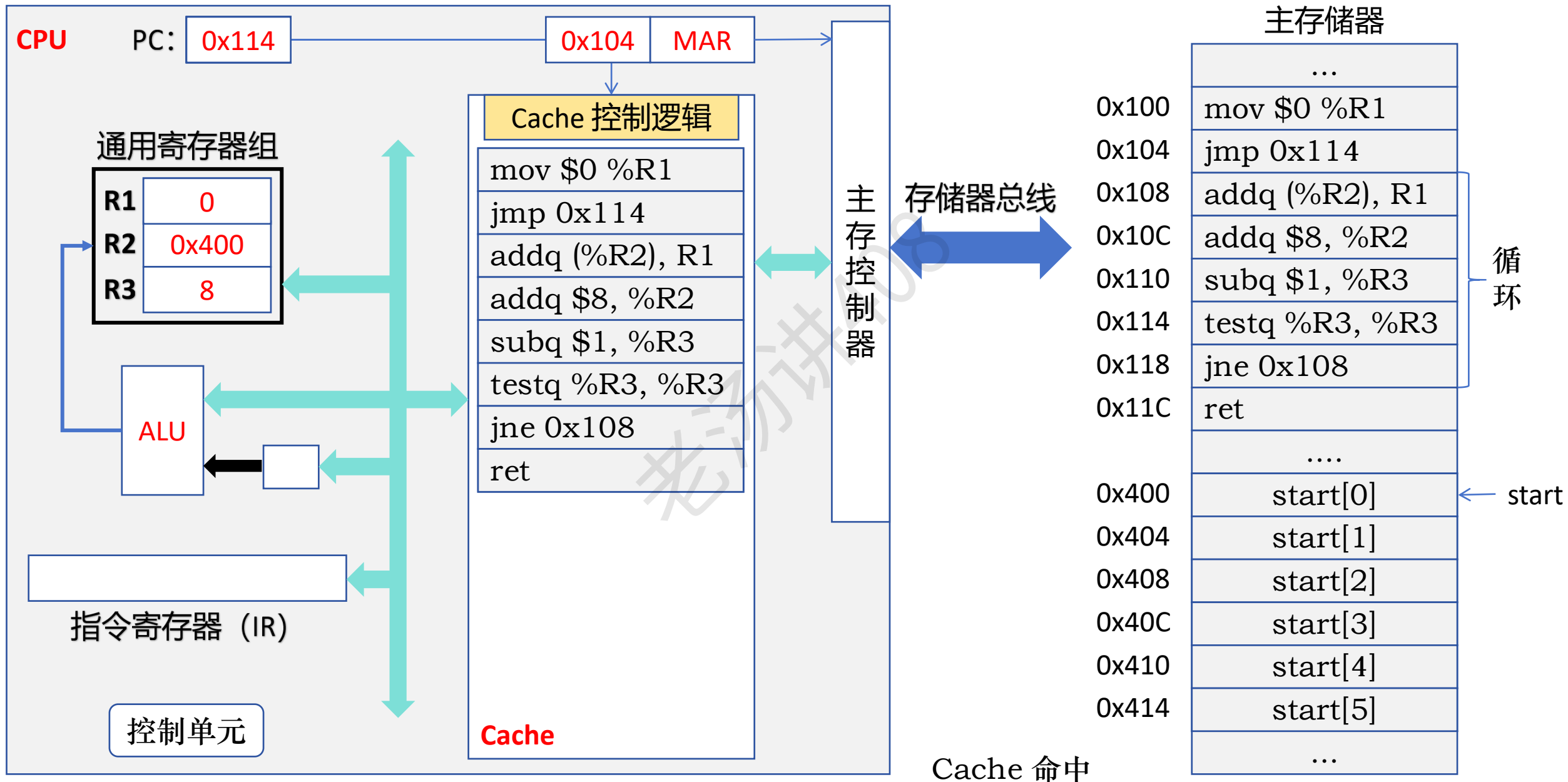
Cache 的基本工作原理



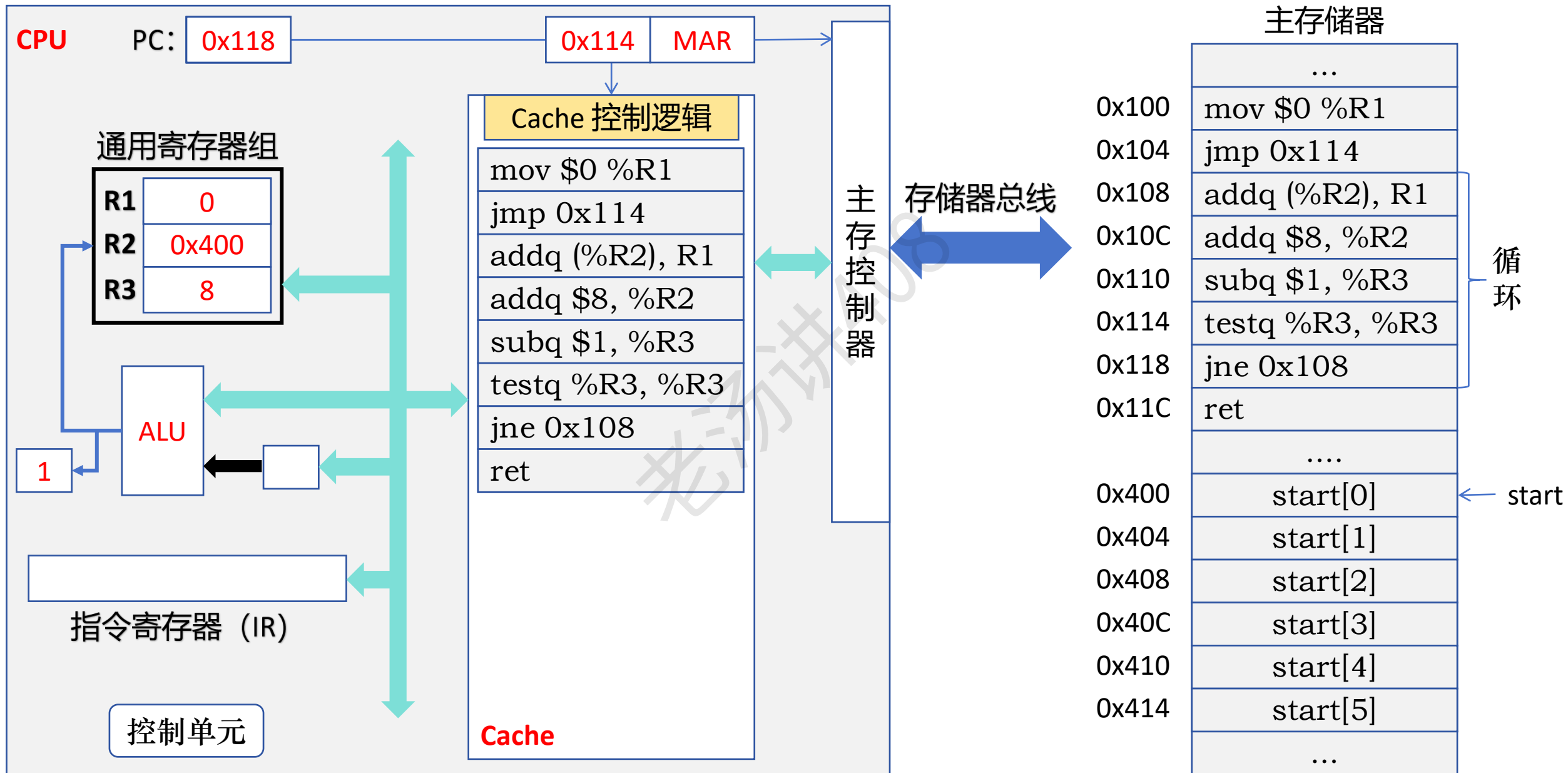
Cache 的基本工作原理



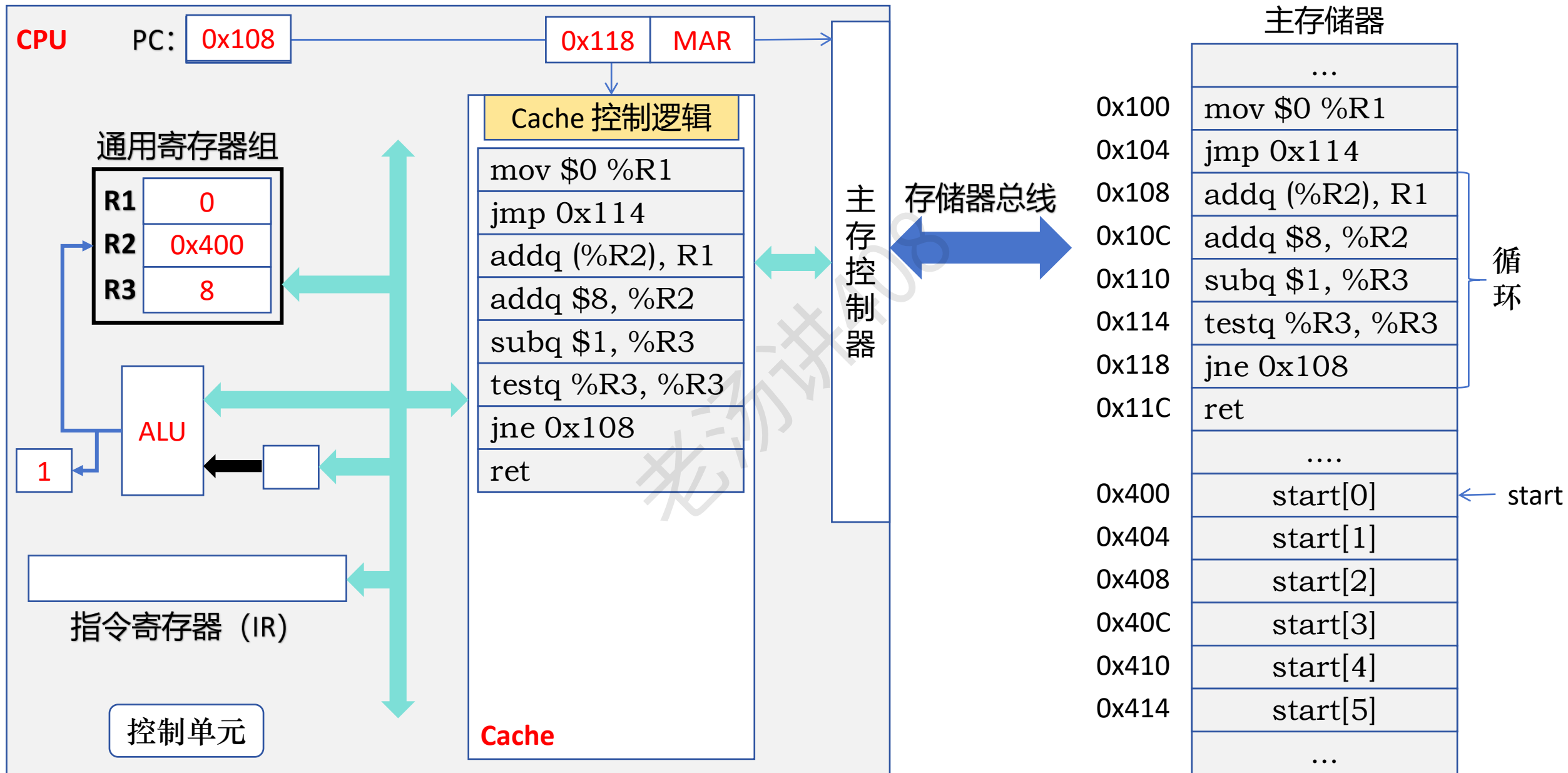
Cache 的基本工作原理



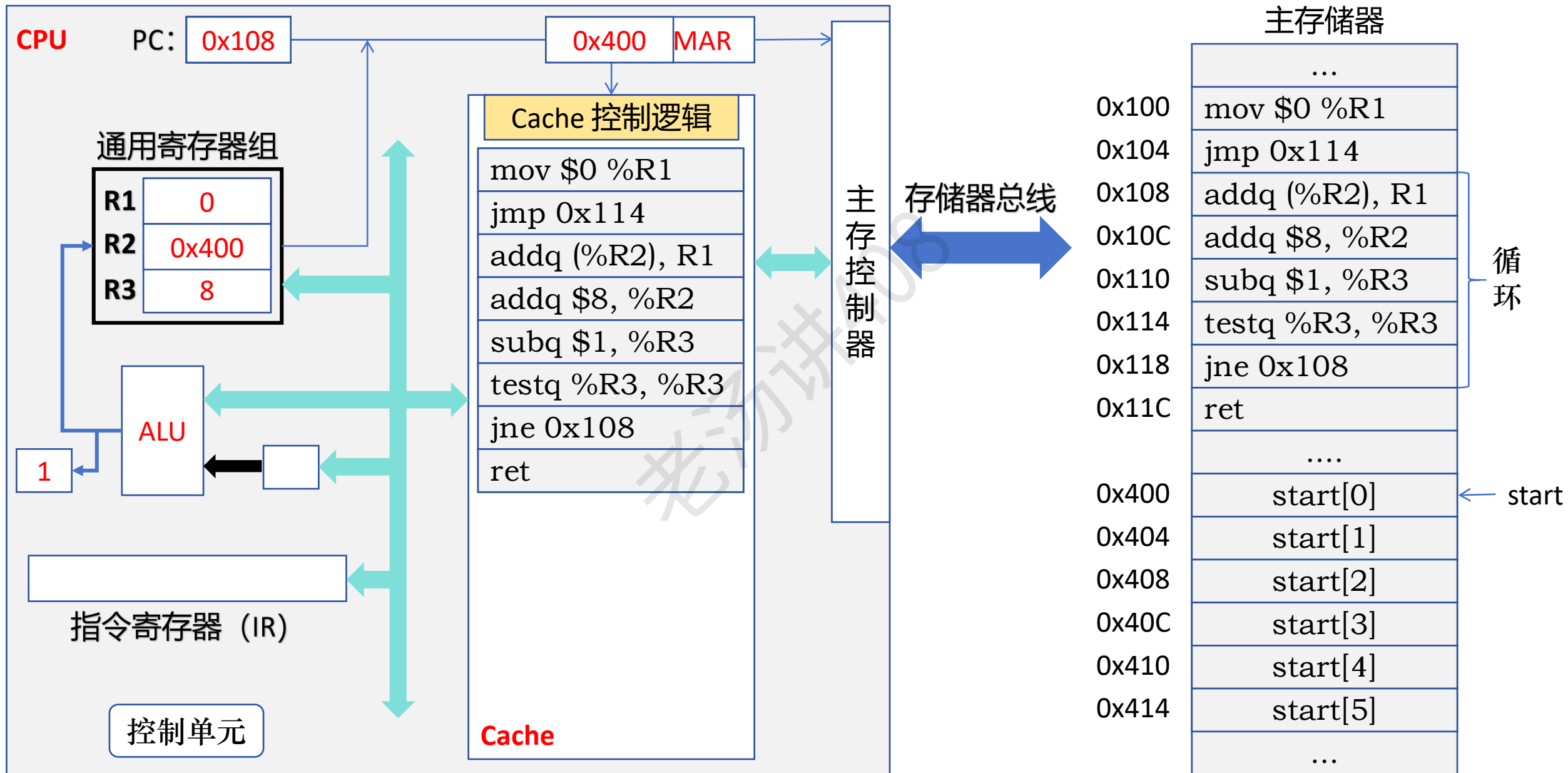
Cache 的基本工作原理



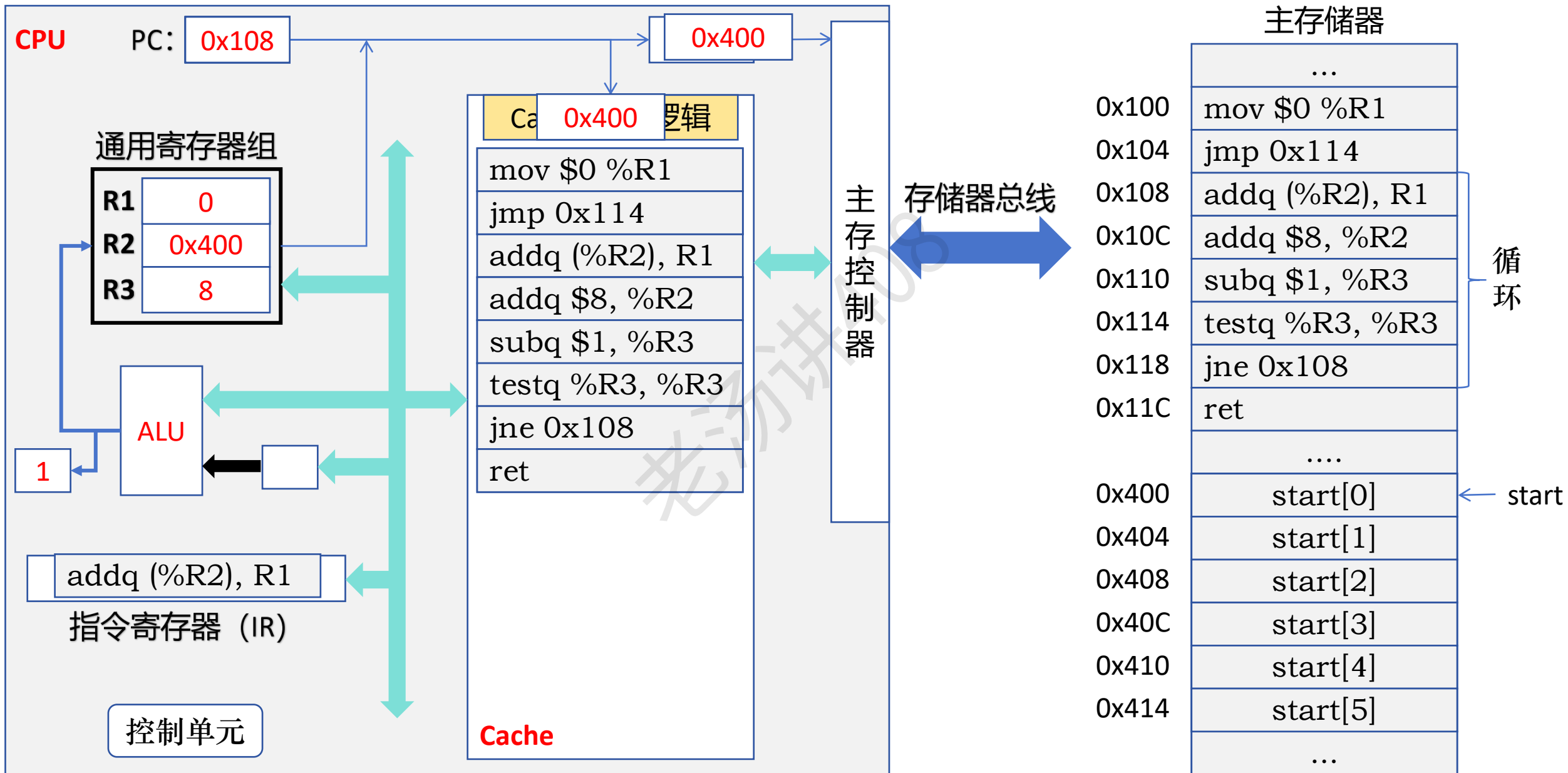
Cache 的基本工作原理



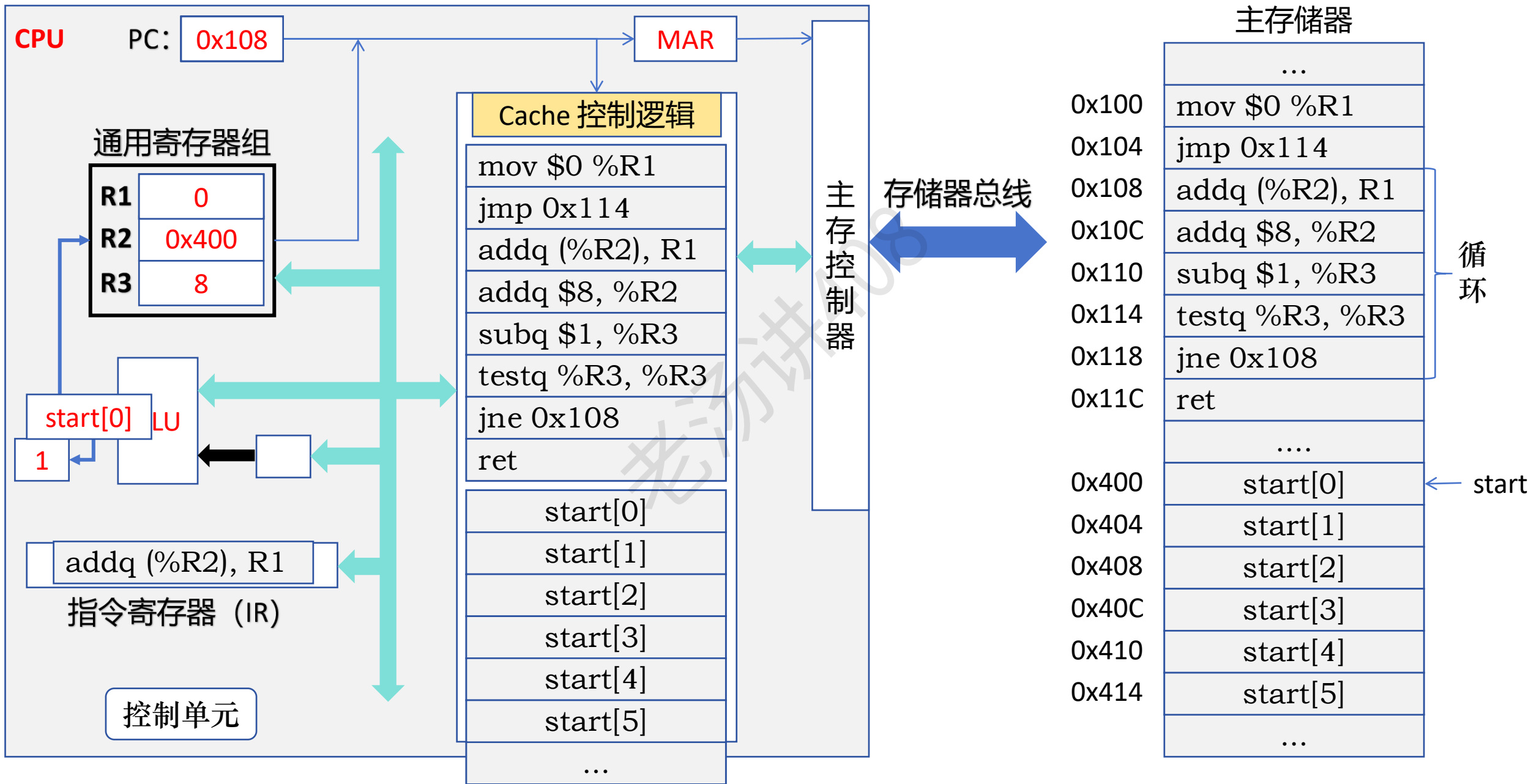
Cache 的基本工作原理



Cache 的基本工作原理



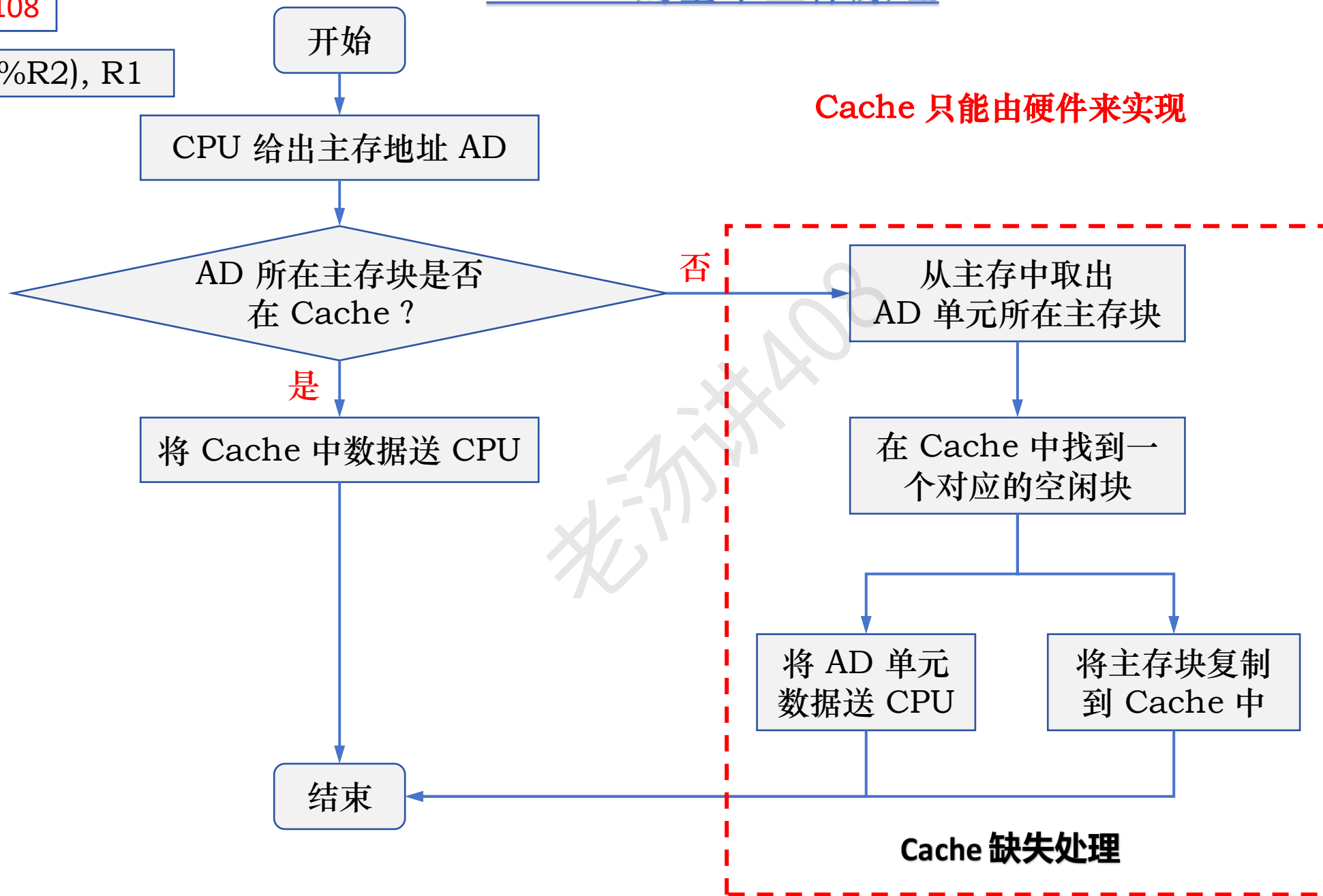
Cache 的基本工作原理



Cache 的基本工作原理

PC: 0x108

addq (%R2), R1



Cache 的基本工作原理：命中率

如果 CPU 访问主存单元所在的主存块在 Cache 中，则称为 Cache 命中 (hit)

命中的概率称为命中率 (hit rate)，它等于命中次数与访问总次数之比

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

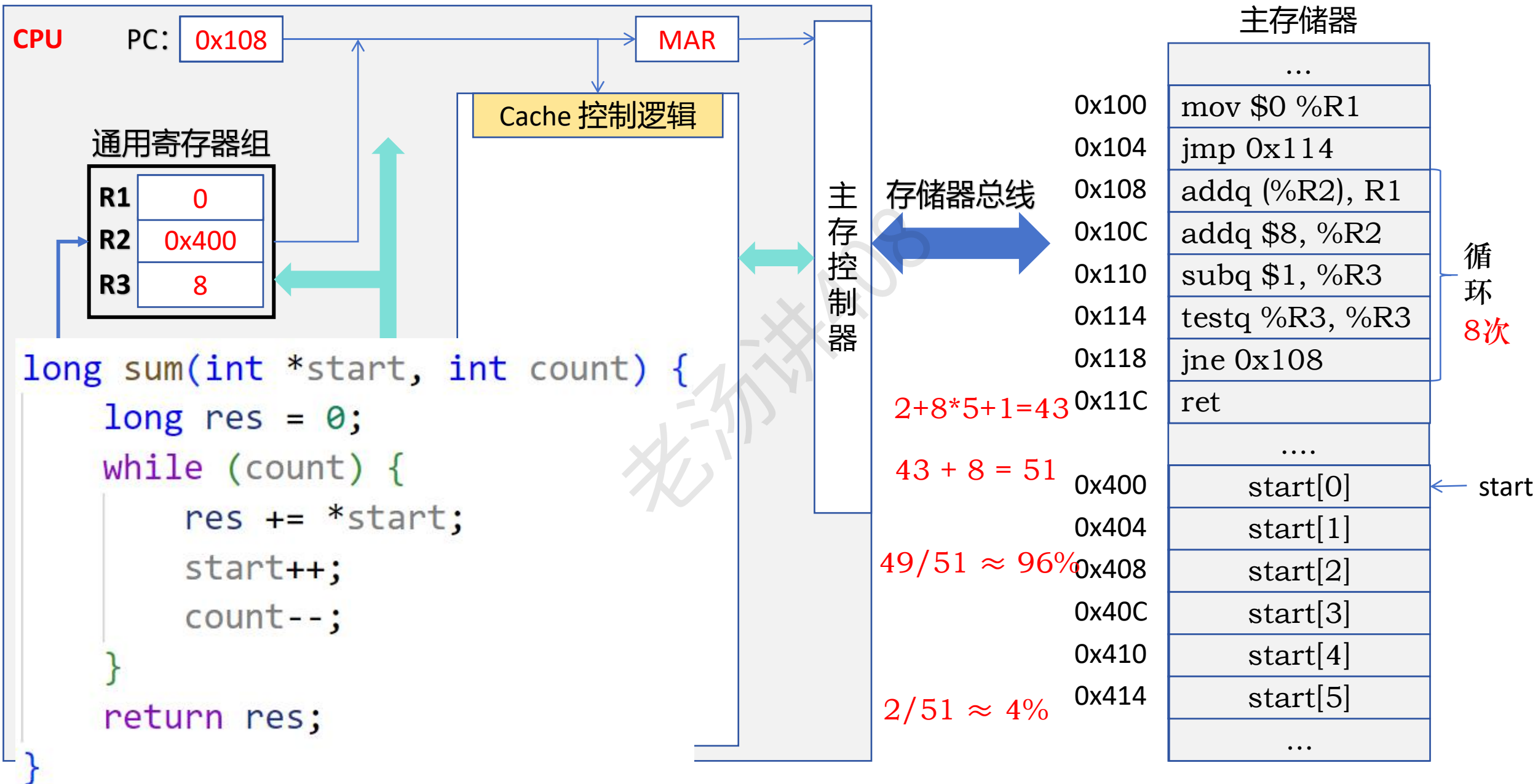
如果 CPU 访问主存单元所在的主存块不在 Cache 中，则称为 Cache 不命中 (miss)

不命中的概率称为缺失率 (miss rate)，它等于不命中次数与访问总次数之比

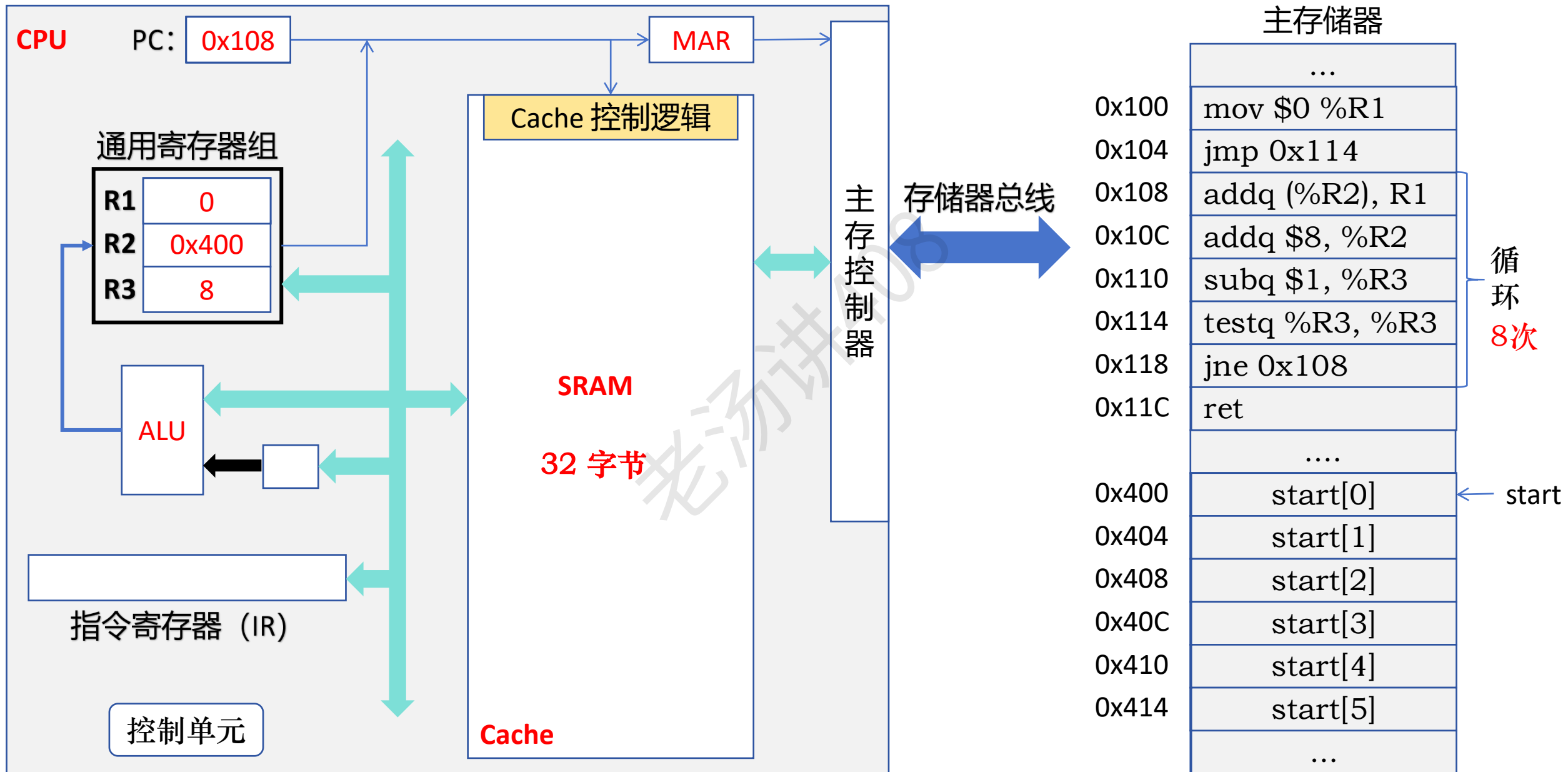


扫一扫上面的二维码图案，加我为朋友。

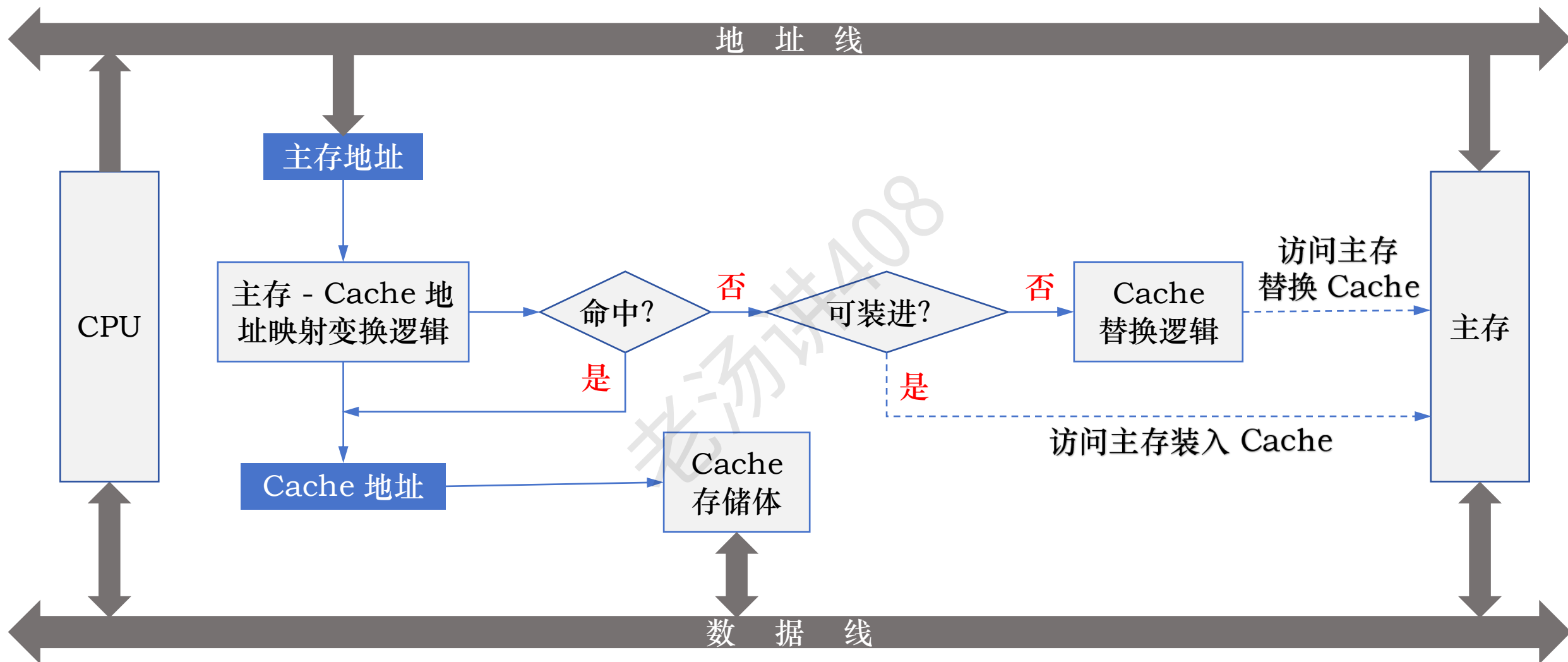
Cache 的基本工作原理：命中率



Cache 的基本工作原理：命中率



Cache 的基本工作原理



高速缓存 (Cache) 主要内容

① 为什么需要 Cache? (局部性原理)



② Cache 的基本工作原理



③ Cache 行和主存块的三种映射方式



直接映射

组相联映射

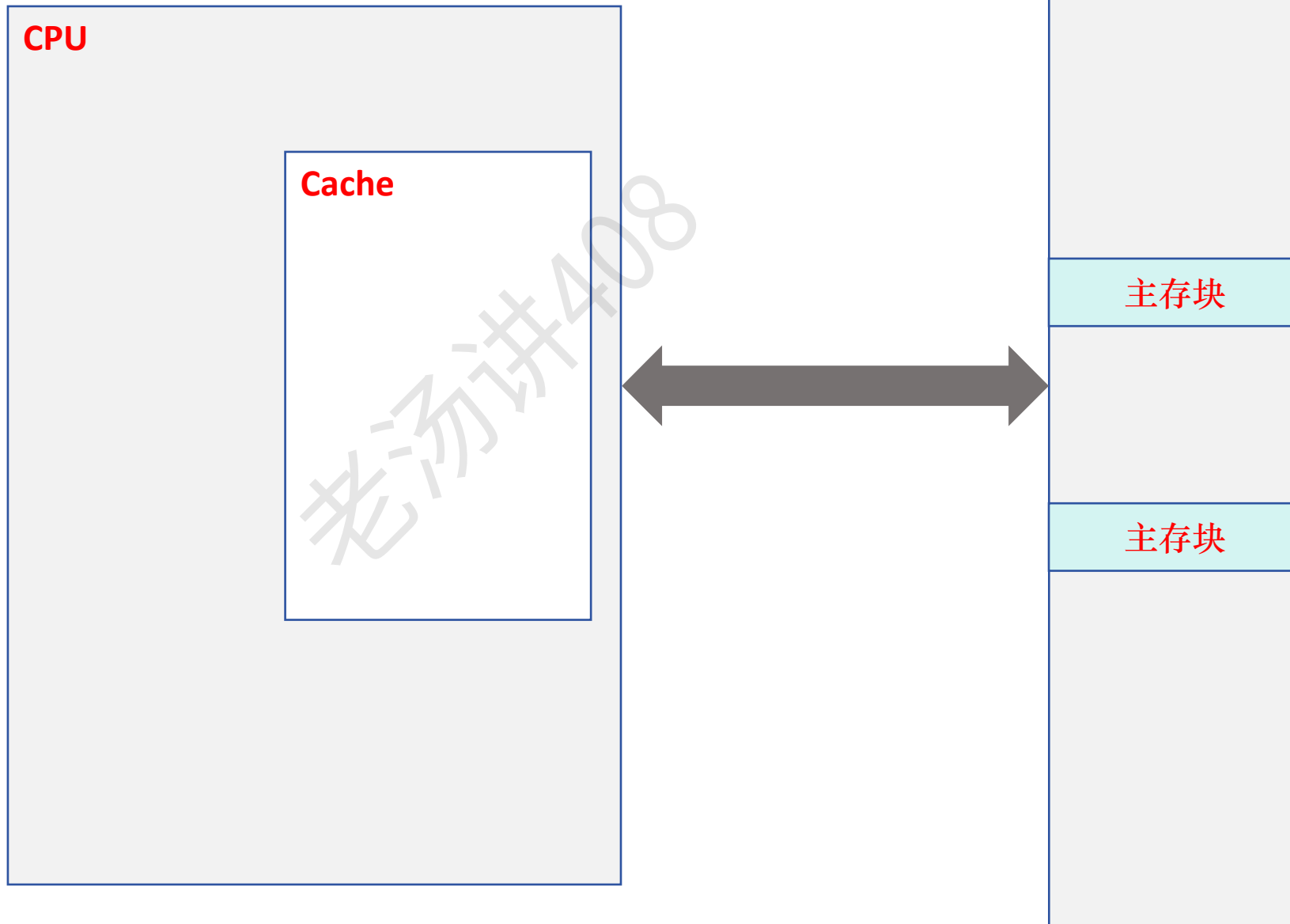
全相联映射

④ Cache 行的替换算法

④ Cache 的数据一致性问题

Cache 行和主存块之间的映射方式

主存储器



一个是可以利用好程序的
空间局部性

一个是可以利用突发传输，
提高传输效率

Cache 行和主存块之间的映射方式

CPU

主存地址： 01 0000 1010

Cache

32B

0 号 Cache 行

1 号 Cache 行

2 号 Cache 行

3 号 Cache 行

$128\text{ B} = 4 \times 32\text{ B}$

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

主存块大小： 32B

$1024\text{ B} / 32\text{ B} = 32$

Cache 行和主存块之间的映射方式

CPU

主存地址: 01 0000 1010 ÷ 32

$266 \div 32 = 8 \dots 10$

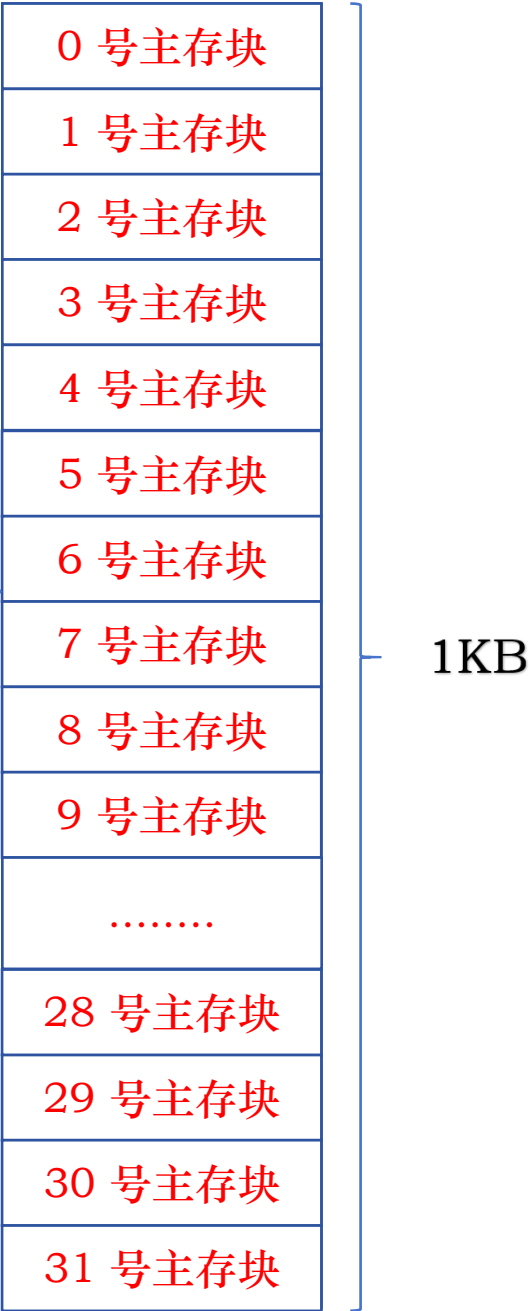


$128\text{ B} = 4 \times 32\text{ B}$

主存块大小: 32B

$1024\text{ B} / 32\text{ B} = 32$

主存储器



Cache 行和主存块之间的映射方式

主存块号

主存地址: 01 0000 1010 ÷ 32

$$138 \div 32 = 8 \dots 10$$

32B = 2⁵B

11 号主存块

主存地址: 01 0110 1001

块内地址

$$361 \div 32 = 11 \dots 9$$

8 号主存块

字节 0	32B
字节 1	
字节 2	
字节 3	
字节 4	
字节 5	
字节 6	
字节 7	
字节 8	
字节 9	
字节 10	
字节 11	
字节 12	
字节 13	
字节 14	
字节 15	
字节 16	
字节 17	
.....	
字节 22	
字节 23	
字节 24	
字节 25	
字节 26	
字节 27	
字节 28	
字节 29	
字节 30	
字节 31	

Cache 行和主存块之间的映射方式

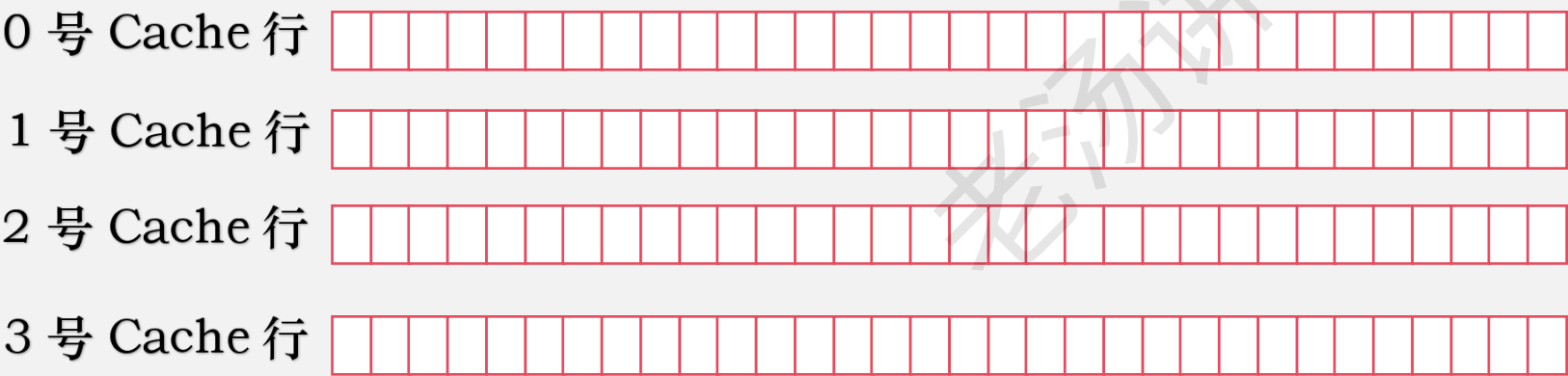
CPU

主存地址: 01 0000 1010

直接映射
全相联映射
组相联映射

Cache

32B



$128\text{ B} = 4 \times 32\text{ B}$

主存块大小: 32B
 $1024\text{ B} / 32\text{ B} = 32$

主存储器

0 号主存块
1 号主存块
2 号主存块
3 号主存块
4 号主存块
5 号主存块
6 号主存块
7 号主存块
8 号主存块
9 号主存块
.....
28 号主存块
29 号主存块
30 号主存块
31 号主存块

1KB

Cache 行和主存块之间的映射方式：直接映射

CPU

主存地址： 01 0000 1010

Cache 行号 = 主存块号 % Cache 行数

Cache

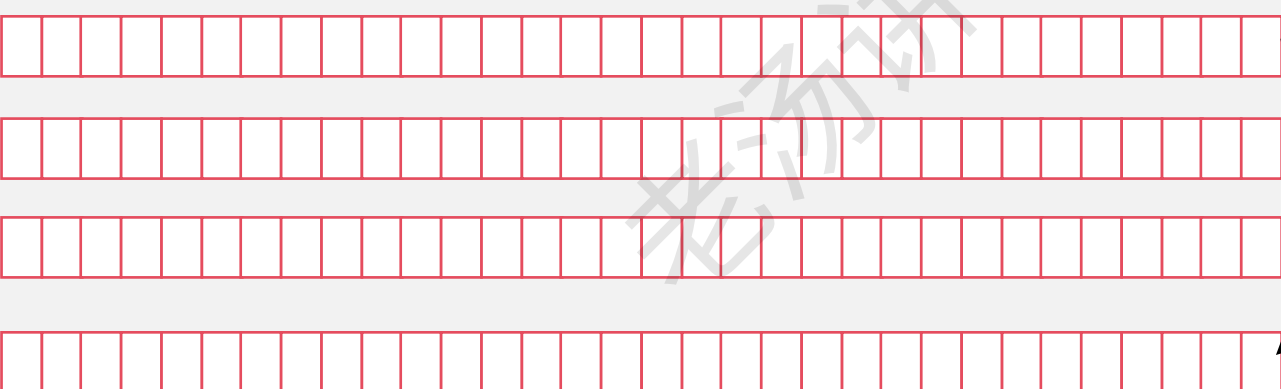
32B

0 号 Cache 行

1 号 Cache 行

2 号 Cache 行

3 号 Cache 行



128 B = 4 × 32B

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

主存块大小： 32B

1024 B / 32B = 32



Cache 行和主存块之间的映射方式: 直接映射

CPU

主存块号 $\text{Cache 行号} = \text{主存块号} \% \text{Cache 行数}$

主存地址: 01 0000 1010

主存地址: 00 0011 1010

主存地址: 11 1111 1010

Cache

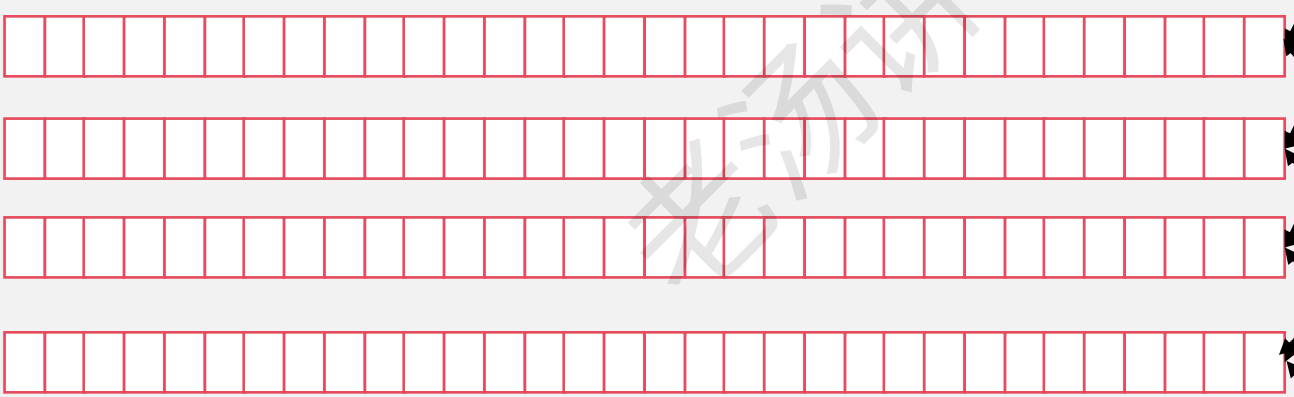
32B

0 号 Cache 行

1 号 Cache 行

2 号 Cache 行

3 号 Cache 行



128 B = 4 × 32B

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

主存块大小: 32B

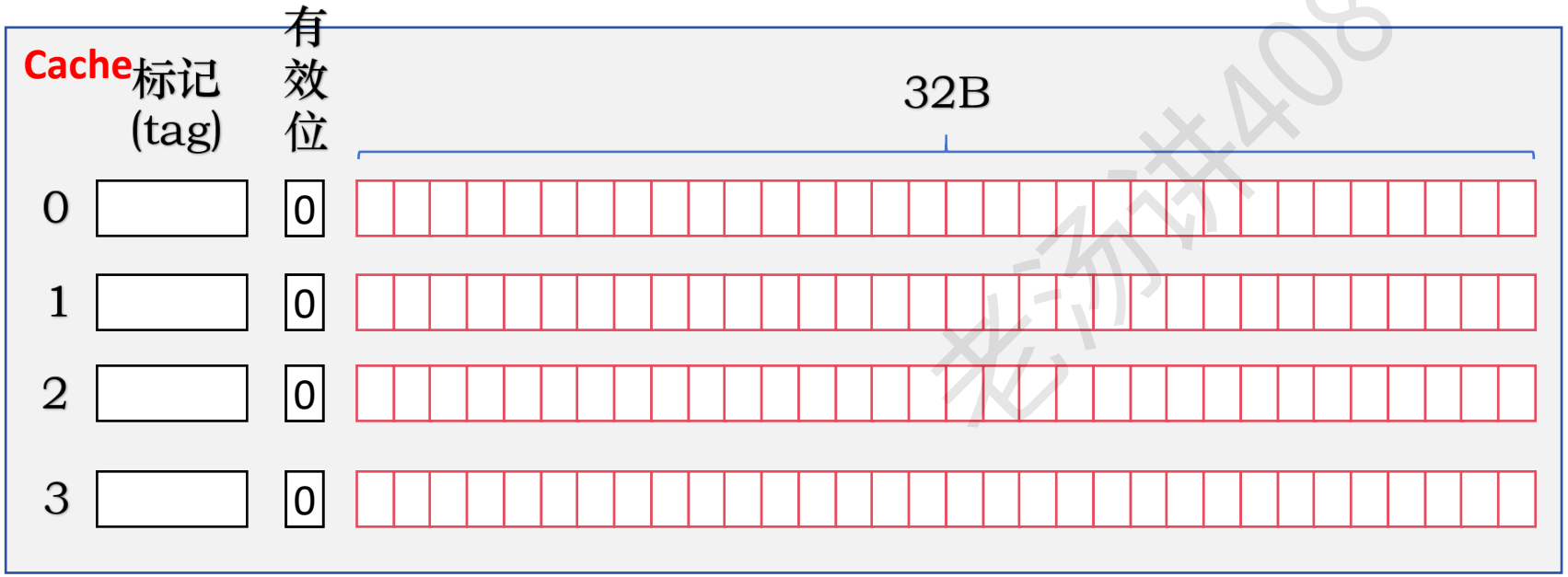
1024 B / 32B = 32

Cache 行和主存块之间的映射方式：直接映射

CPU

主存块号 $\text{Cache 行号} = \text{主存块号} \% \text{Cache 行数}$

主存地址： 01 0000 1010



$128\text{ B} = 4 \times 32\text{ B}$

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

主存块大小： 32B

$1024\text{ B} / 32\text{ B} = 32$

Cache 行和主存块之间的映射方式: 直接映射

CPU

主存块号 $\text{Cache 行号} = \text{主存块号} \% \text{Cache 行数}$

主存地址: 01 0000 1010

Cache		有效位	32B
标记 (tag)			
0	010	1	
1		0	
2		0	
3		0	

$128\text{ B} = 4 \times 32\text{ B}$

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

主存块大小: 32B

$1024\text{ B} / 32\text{ B} = 32$

Cache 行和主存块之间的映射方式: **直接映射**

CPU

Cache 行号 = 主存块号 % Cache 行数

主存地址: 01 0000 1010
主存地址: 00 0000 1010

主存块号

块内地址

Cache 标记 (tag)	有效位	32B
0 010	1	
1	0	
2	0	
3	0	

128 B = 4 × 32B

主存储器

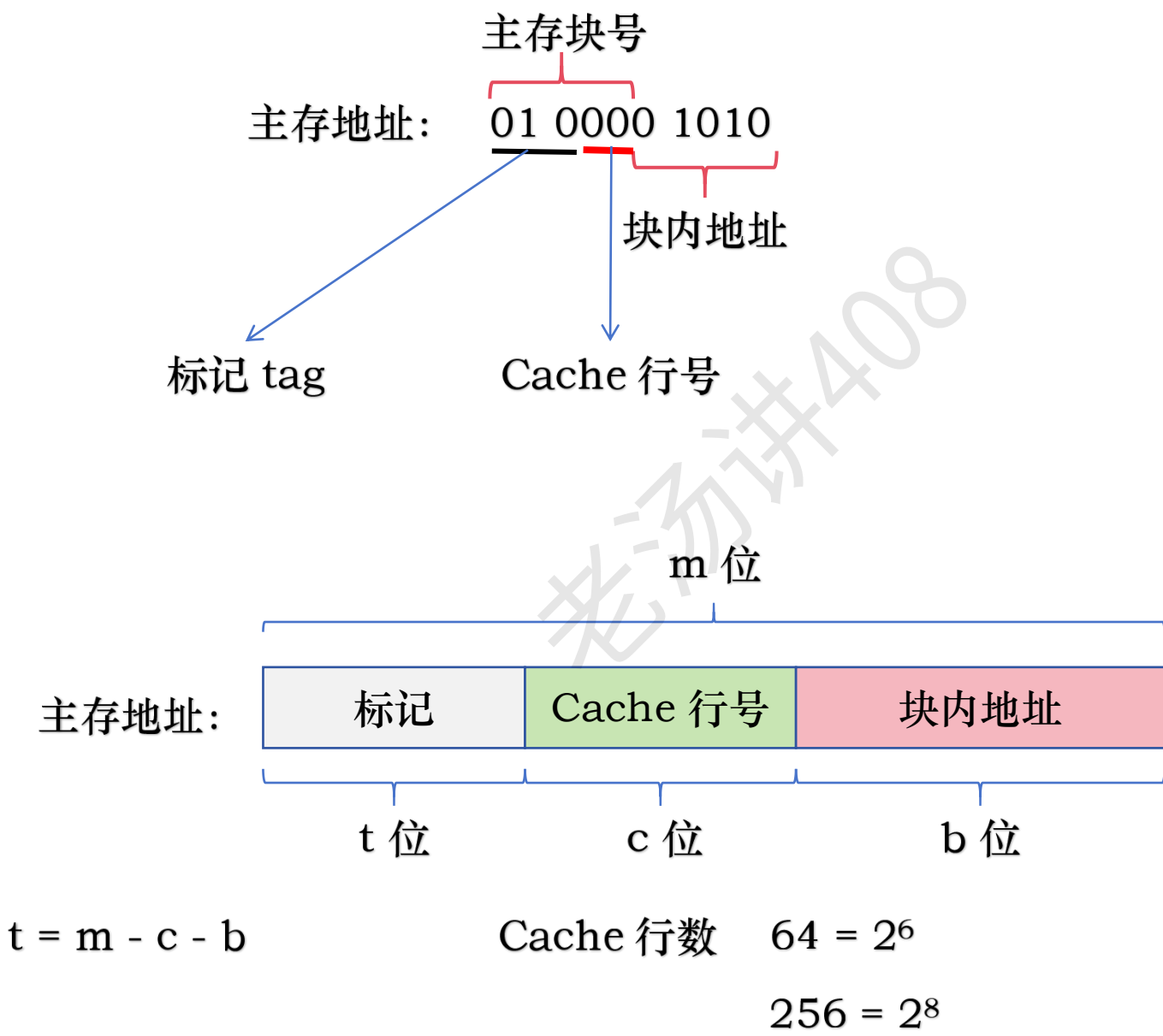
- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

主存块大小: 32B

1024 B / 32B = 32

Cache 行和主存块之间的映射方式: 直接映射



主存块的大小

$16B = 2^4B$

$1KB = 1024B = 2^{10}B$

Cache 行和主存块之间的映射方式：直接映射 - 例子

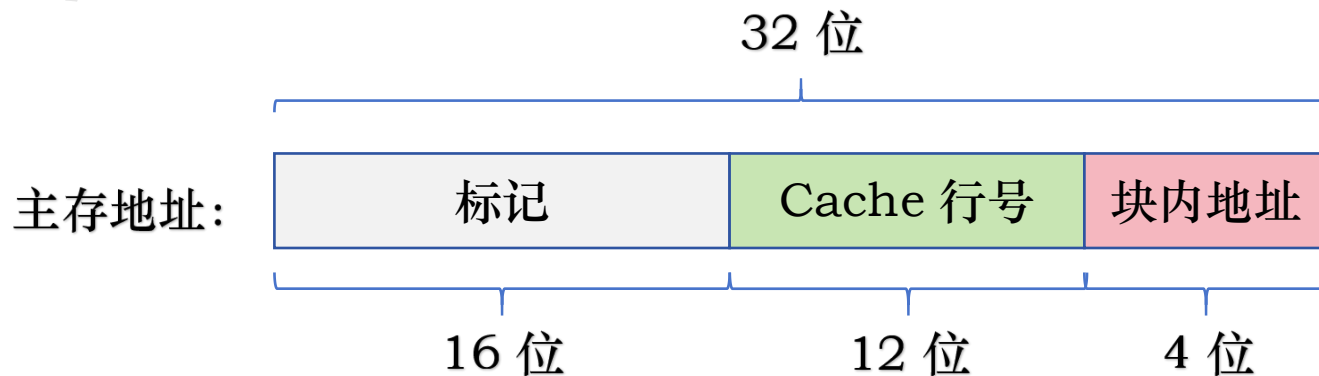
假定主存和 Cache 之间采用直接映射方式，块大小为 16B。Cache 的数据区容量为 64KB，主存地址为 32 位，按字节编址。问：主存地址如何划分？说明访存过程，并计算 Cache 总容量为多少？

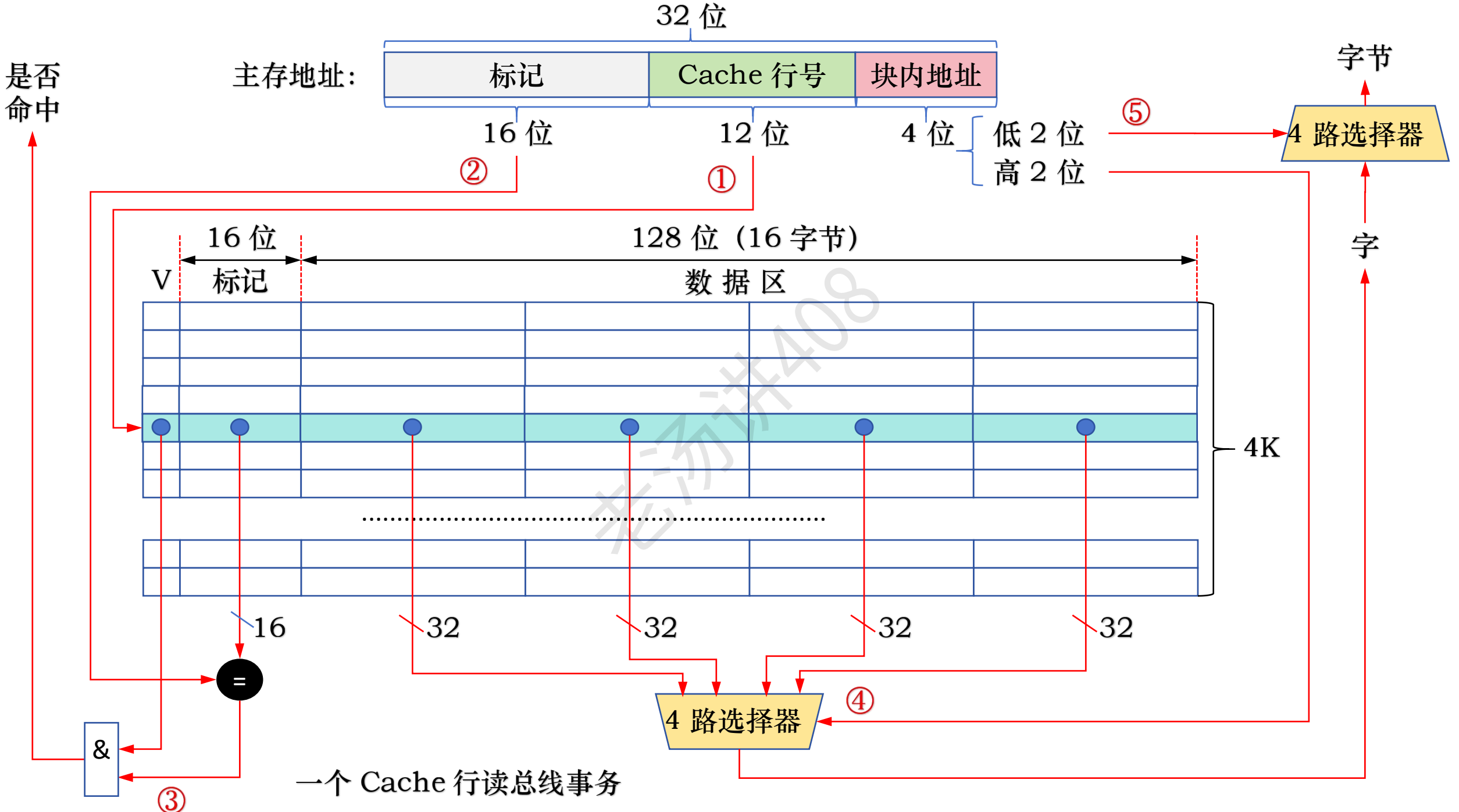
块大小为 $16\text{B} = 2^4\text{B}$ ，所以块内偏移占 4 位

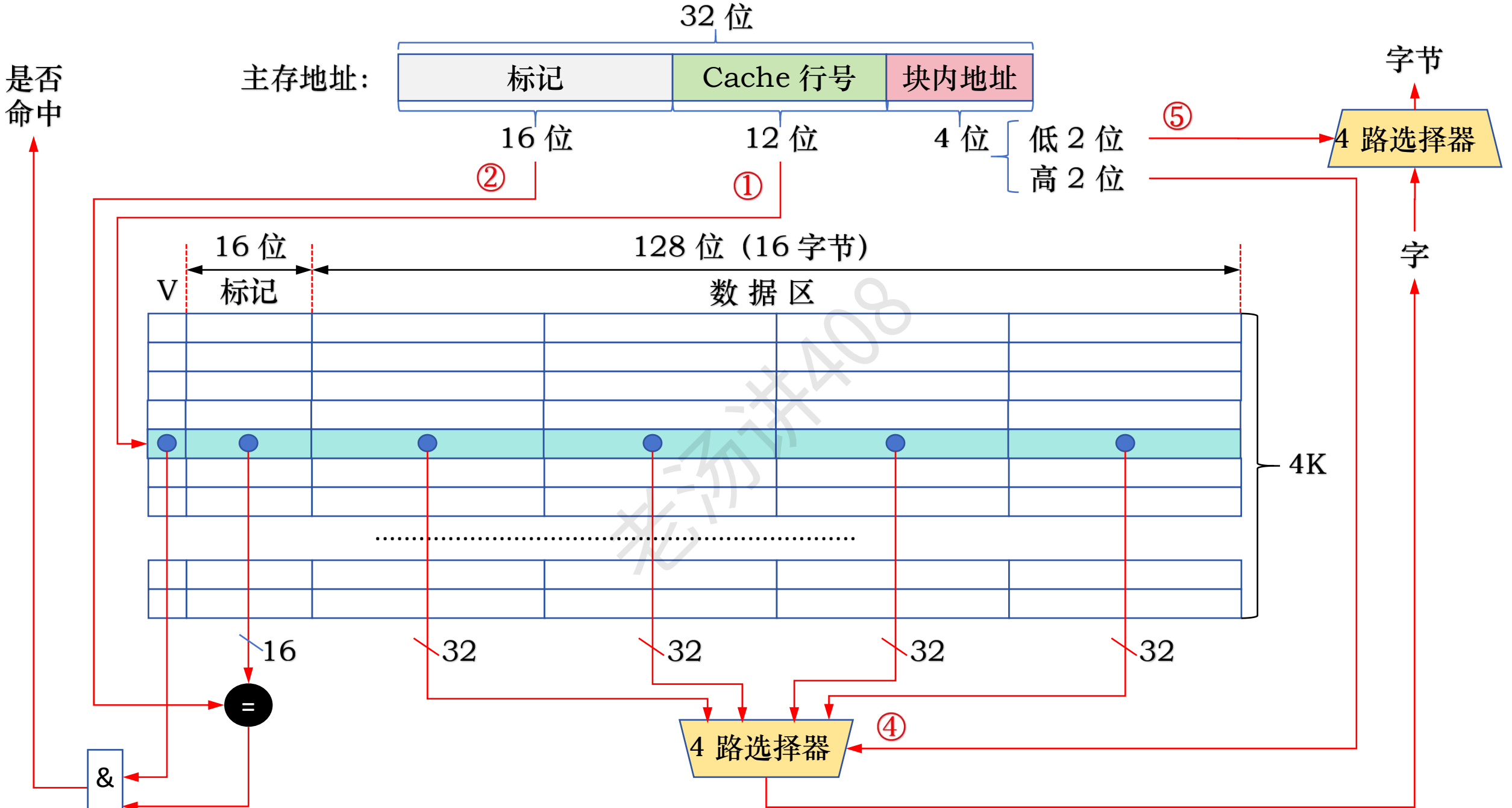
Cache 行数 = Cache 数据区容量 / 块大小 = $64\text{KB} / 16\text{B} = 2^{16}\text{B} / 2^4\text{B} = 4\text{K} = 2^{12}$

所以 Cache 行占 12 位

标记占的位数 = $32 - 4 - 12 = 16$ 位







③ 每个 Cache 行: 1位 + 16 位 + 128 位 Cache 总容量: $4K * (1位 + 16 位 + 128 位) = 72.5KB$

Cache 行和主存块之间的映射方式：直接映射的优缺点

CPU

主存地址： 00 1000 1010
主存块号

Cache	标记 (tag)	有效位	32B
0	001	1	
1		0	
2		0	
3		0	

128 B = 4 × 32B

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

主存块大小： 32B

1024 B / 32B = 32

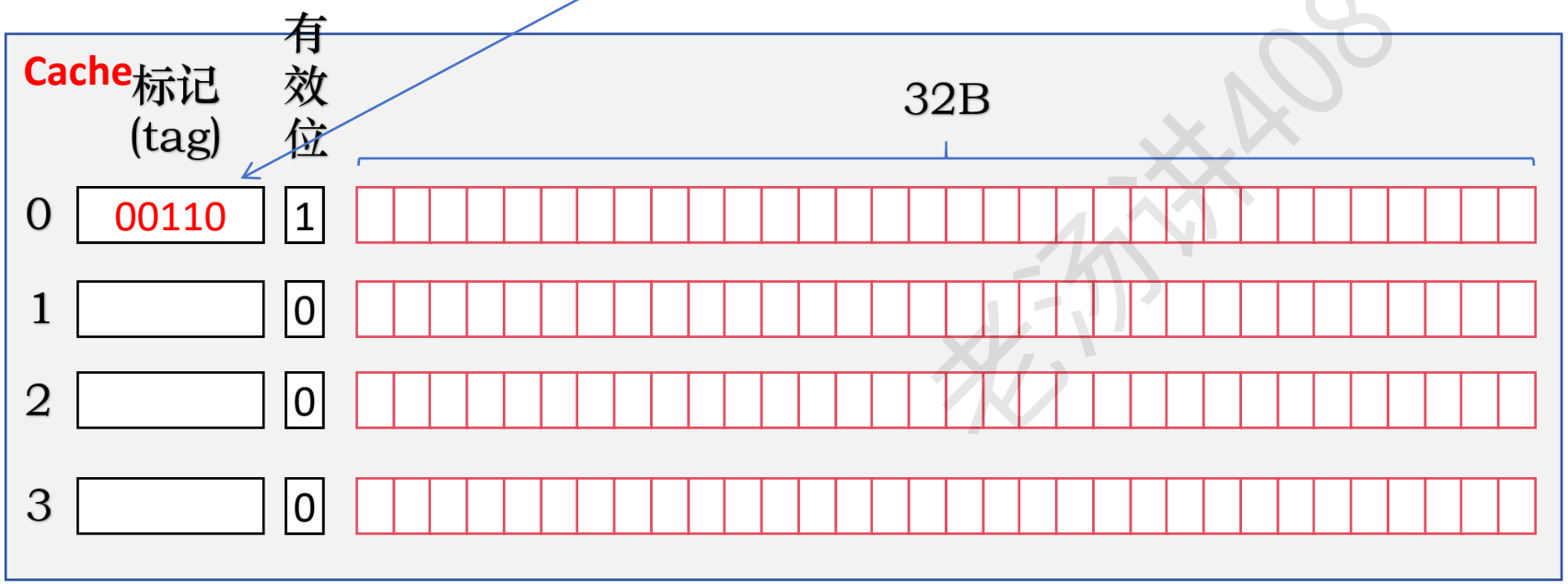
优点：实现简单，命中时间短

Cache 行和主存块之间的映射方式：全相联映射

CPU

主存块号

主存地址： 00 1100 1010



128 B = 4 × 32B

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块



1KB

主存块大小： 32B

1024 B / 32B = 32

基本思想：一个主存块可装入 cache 任意一行中

Cache 行和主存块之间的映射方式：全相联映射

CPU

主存块号

主存地址： 00 1100 1010
主存地址： 00 1010 1110

Cache	标记 (tag)	有效位	32B
0	00110	1	
1	00101	1	
2		0	
3		0	

128 B = 4 × 32B

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB



主存块大小： 32B

1024 B / 32B = 32

基本思想：一个主存块可装入 cache 任意一行中

Cache 行和主存块之间的映射方式: 全相联映射

CPU

主存块号

主存地址: 00 1100 1010
主存地址: 00 1010 1110
主存地址: 11 1110 0110

Cache		有效位	32B
标记 (tag)			
0	00110	1	
1	00101	1	
2	11111	1	
3		0	

128 B = 4 × 32B

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

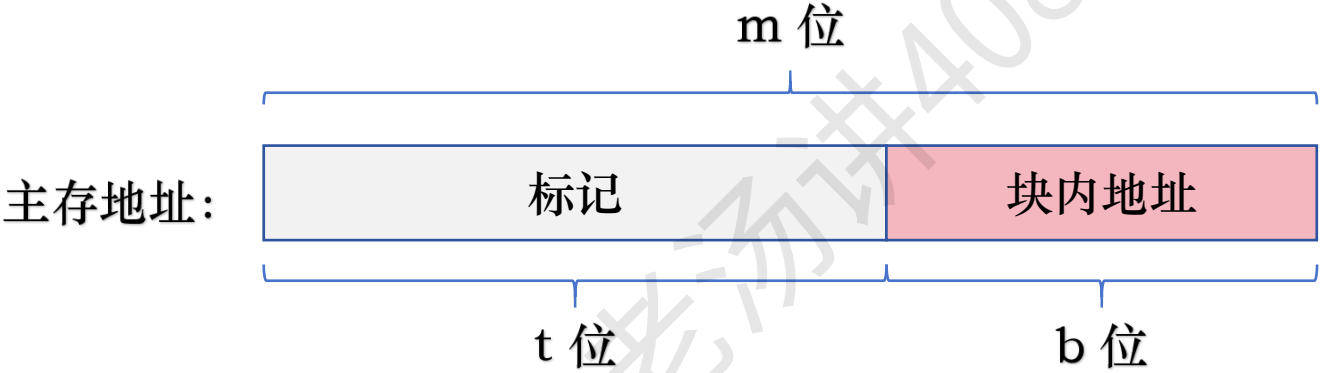
主存块大小: 32B

1024 B / 32B = 32

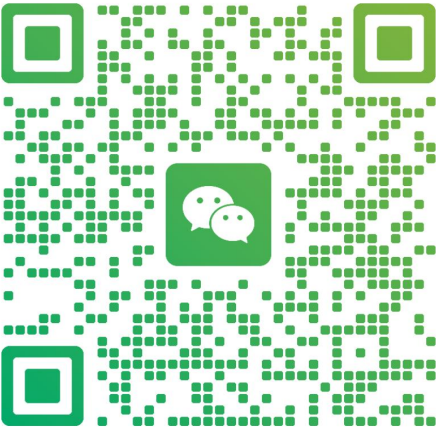
基本思想: 一个主存块可装入 cache 任意一行中

Cache 行和主存块之间的映射方式：全相联映射

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

Cache 行和主存块之间的映射方式: 全相联映射优缺点

CPU

主存块号

主存地址: 00 1100 1010
主存地址: 00 1010 1110
主存地址: 11 1110 0110

Cache	标记 (tag)	有效位	32B
0	00110	1	
1	00101	1	
2	11111	1	
3		0	

128 B = 4 × 32B

主存储器

- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

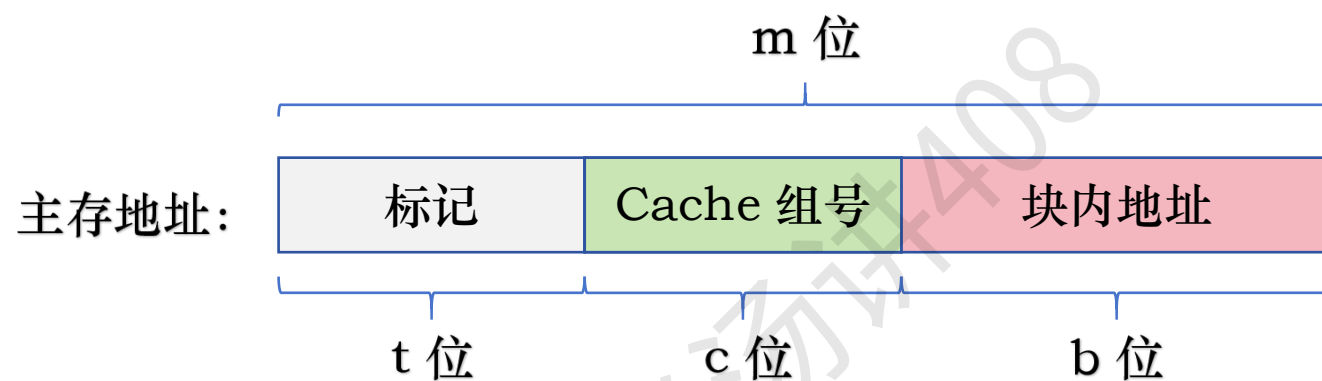


主存块大小: 32B

1024 B / 32B = 32

基本思想: 一个主存块可装入 cache 任意一行中

Cache 行和主存块之间的映射方式：组相联映射



基本思想是：将 Cache 所有的行分成 2^c 个大小相等的组，每组有多个 Cache 行，每个主存块被映射到 Cache 固定的组中的任意一行

也就是说主存块与 Cache 组采用直接映射，而在 Cache 组内，主存块和 Cache 行采用全相联映射的方式

Cache 行和主存块之间的映射方式：组相联映射

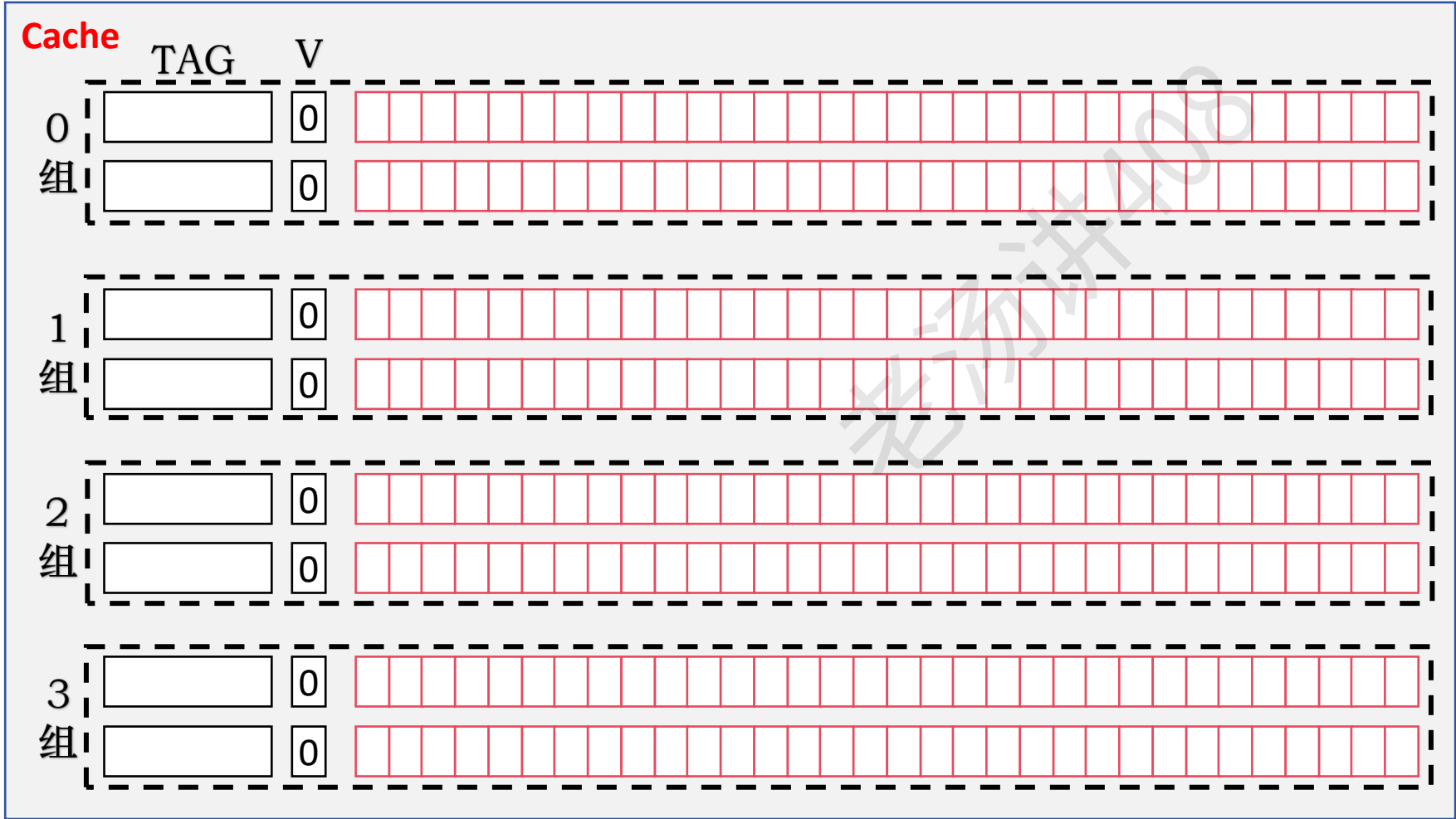
主存块大小： 32B

主存储器

CPU

主存地址： 00 1100 1010

主存块号



- 0 号主存块
- 1 号主存块
- 2 号主存块
- 3 号主存块
- 4 号主存块
- 5 号主存块
- 6 号主存块
- 7 号主存块
- 8 号主存块
- 9 号主存块
-
- 28 号主存块
- 29 号主存块
- 30 号主存块
- 31 号主存块

1KB

Cache 行和主存块之间的映射方式：组相联映射

主存块大小： 32B

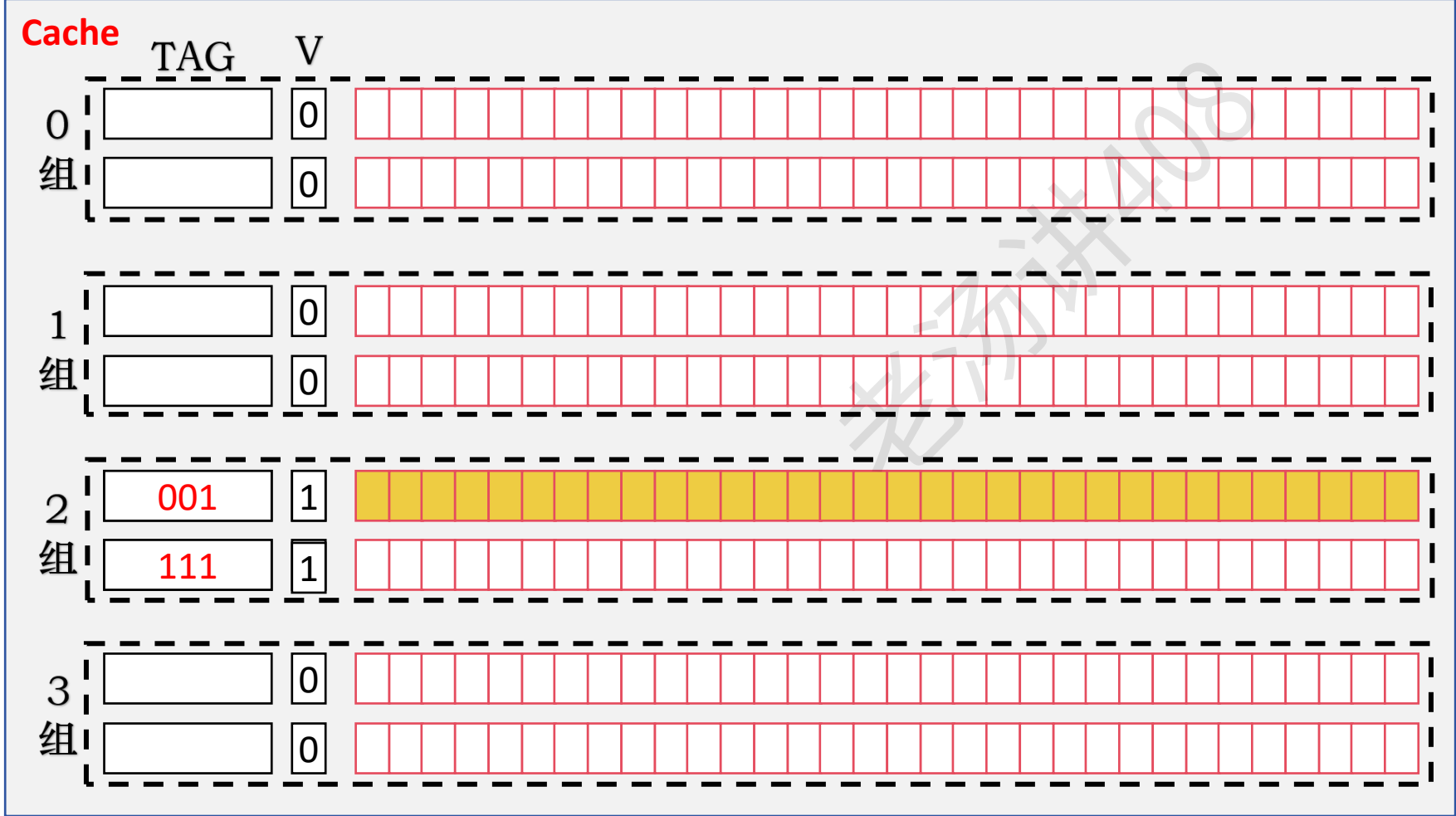
主存储器

CPU

主存块号

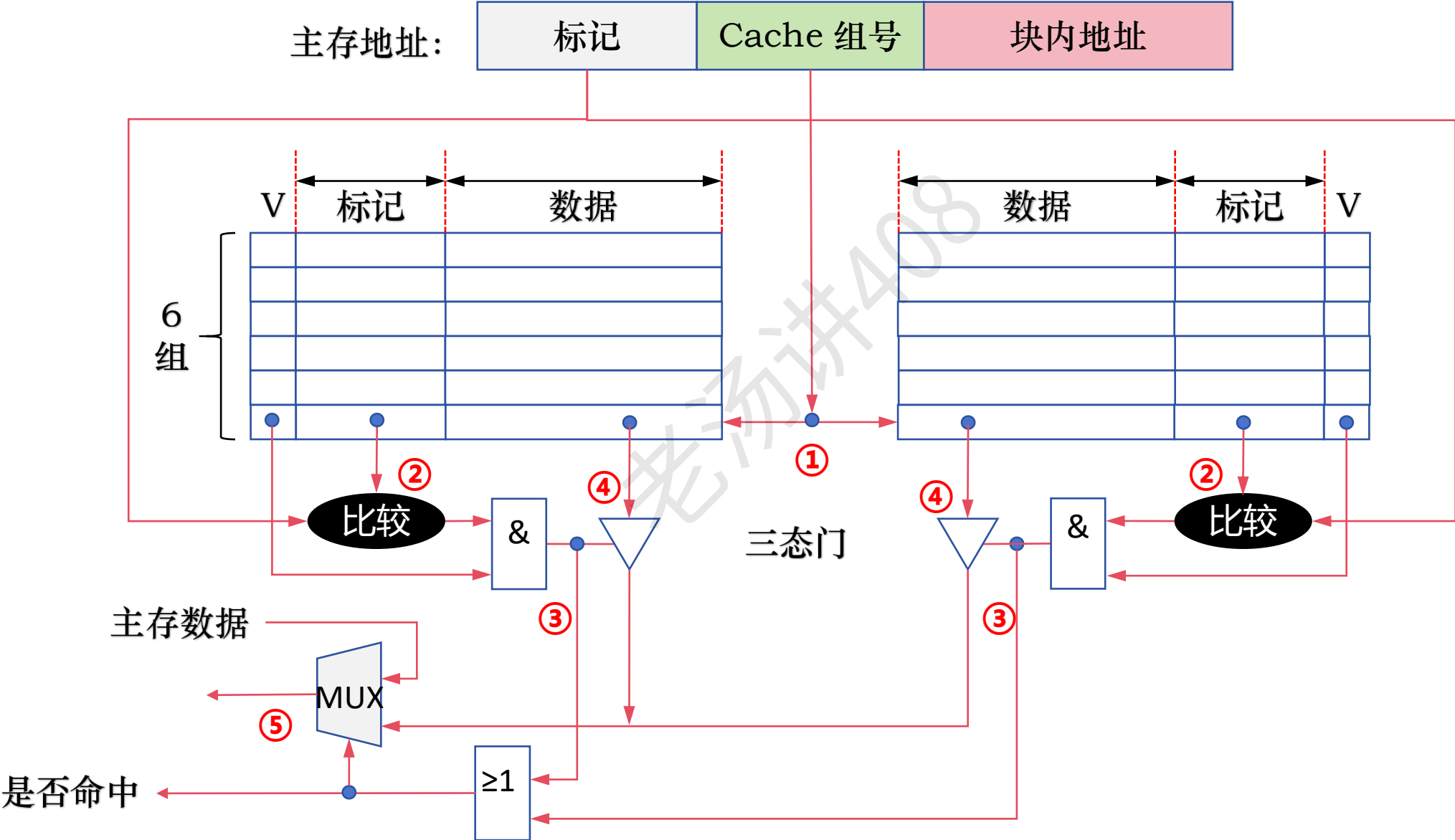
二路组相联

主存地址： 00 1100 1010
主存地址： 11 1100 1010



1KB

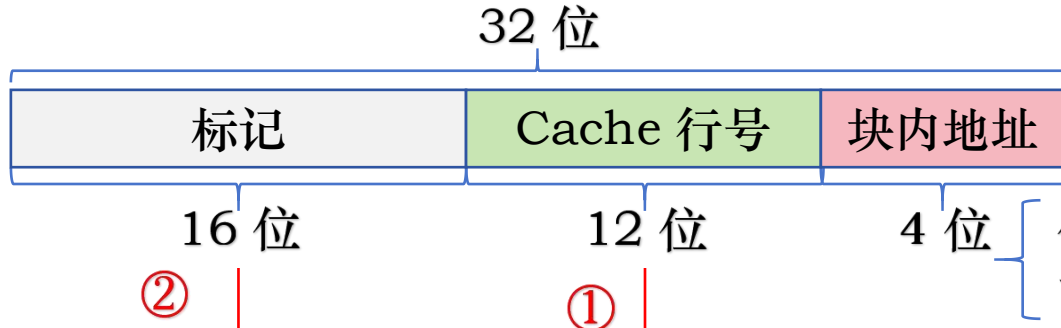
Cache 行和主存块之间的映射方式：二路组相联映射的电路实现



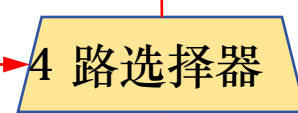
直接相联电路实现

是否命中

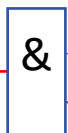
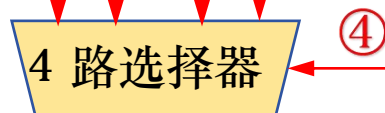
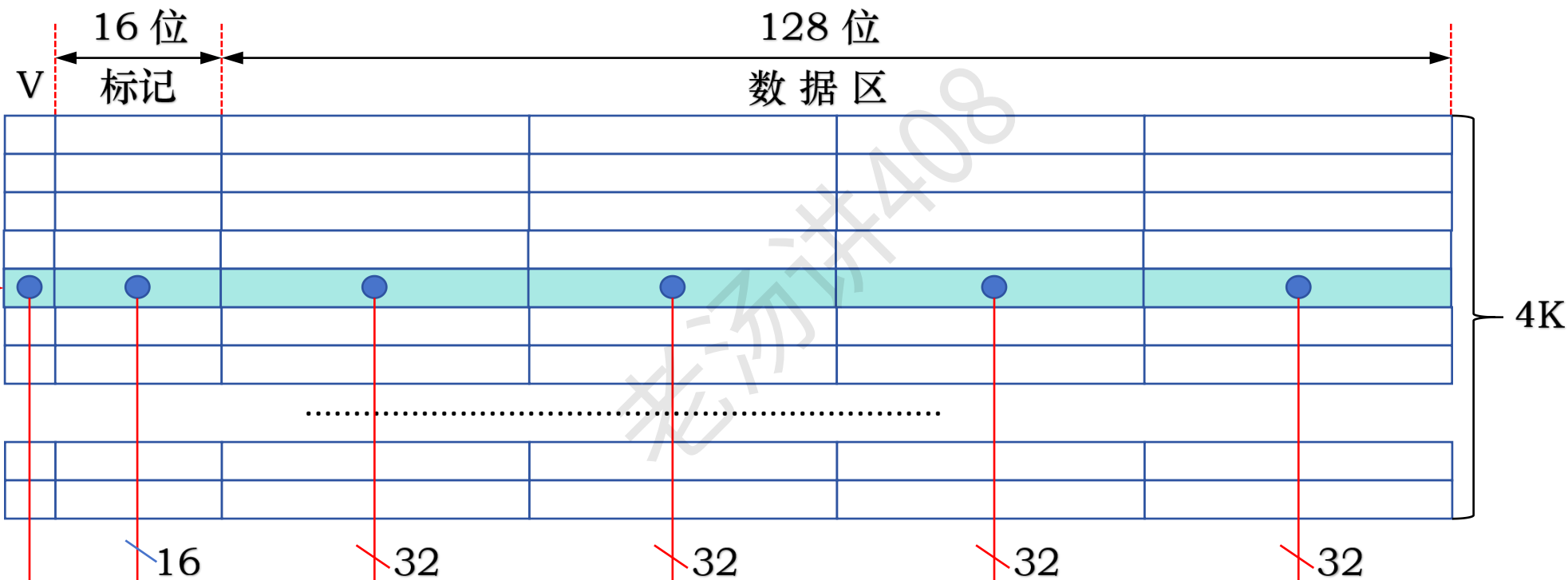
主存地址:



字节



字



③

一个 Cache 行读总线事务

直接映射、全相联映射、组相联映射

直接映射：1 个比较器

组相联映射：每组多少个 Cache 行，就有多少个比较器

全相联映射：有多少 Cache 行，就有多少比较器

直接映射、全相联映射、组相联映射

在组相联映射中，当 Cache 中只有 1 组，则变为全相联映射

在组相联映射中，当 Cache 中每组只有一个 Cache 行，则变为直接映射

组相联映射的冲突概率比直接映射低，由于只有组内各行采用全相联映射，所以比较器的位数和个数都比全相联映射少，易于实现，查找速度也快得多

关联度

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

Cache 行号 = 主存块号 % Cache 行数

主存地址: 00 1000 1010

主存地址: 00 0000 1010

块内地址

CPU

Cache 标记
(tag)

有效位

32B

0 001 1

1 0 0

2 0 0

3 0 0

128 B = 4 × 32B

主存块大小: 32B

1024 B / 32B = 32

关联度

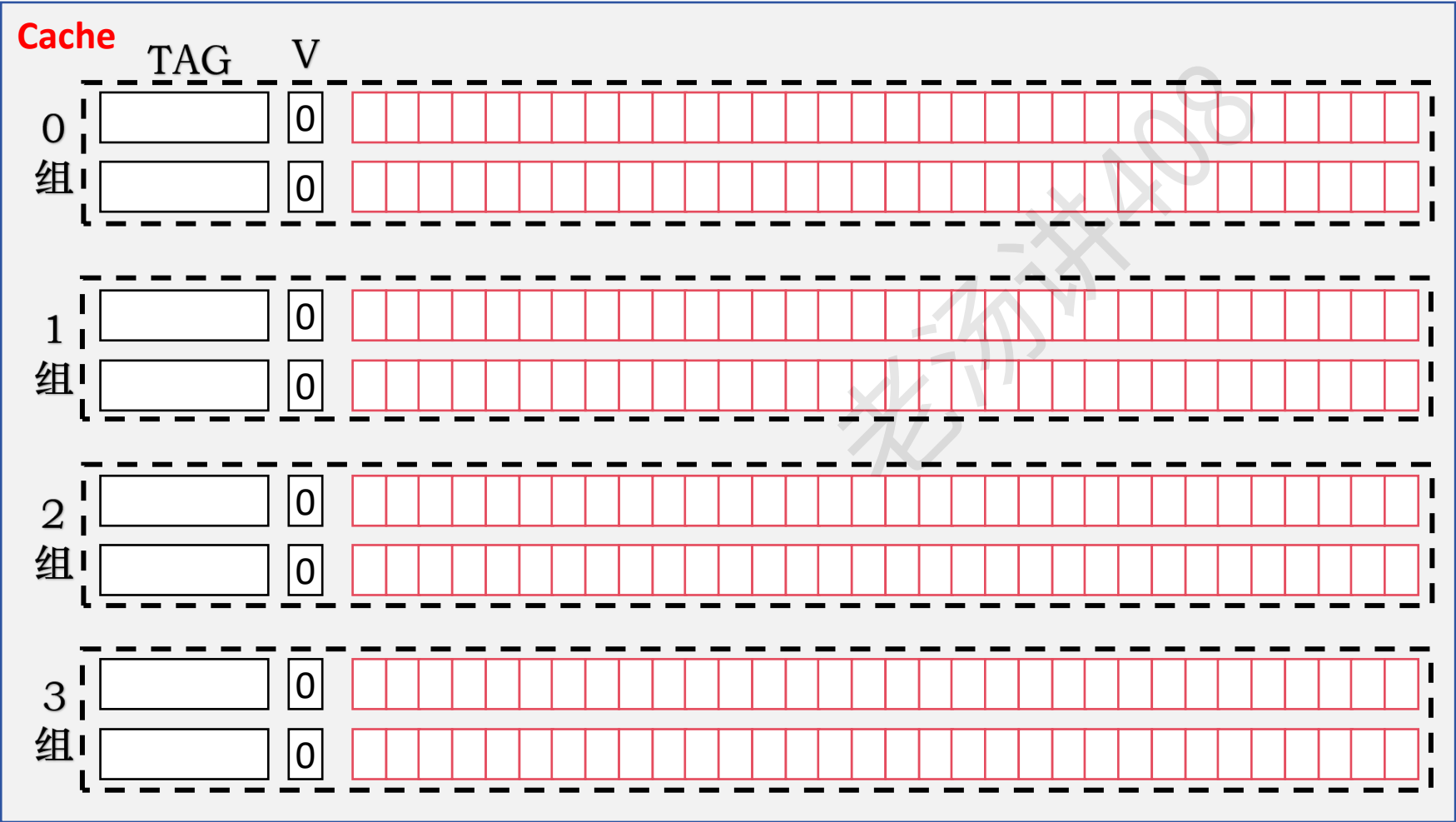
主存块大小： 32B

主存储器

CPU

主存块号

主存地址： 00 1100 1010



0 号主存块
1 号主存块
2 号主存块
3 号主存块
4 号主存块
5 号主存块
6 号主存块
7 号主存块
8 号主存块
9 号主存块
.....
28 号主存块
29 号主存块
30 号主存块
31 号主存块

1KB

关联度

关联度指一个主存块映射到 Cache 中时可能存放的位置个数

直接映射: 1

全相联映射: Cache 总行数

N 路相联映射: N

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



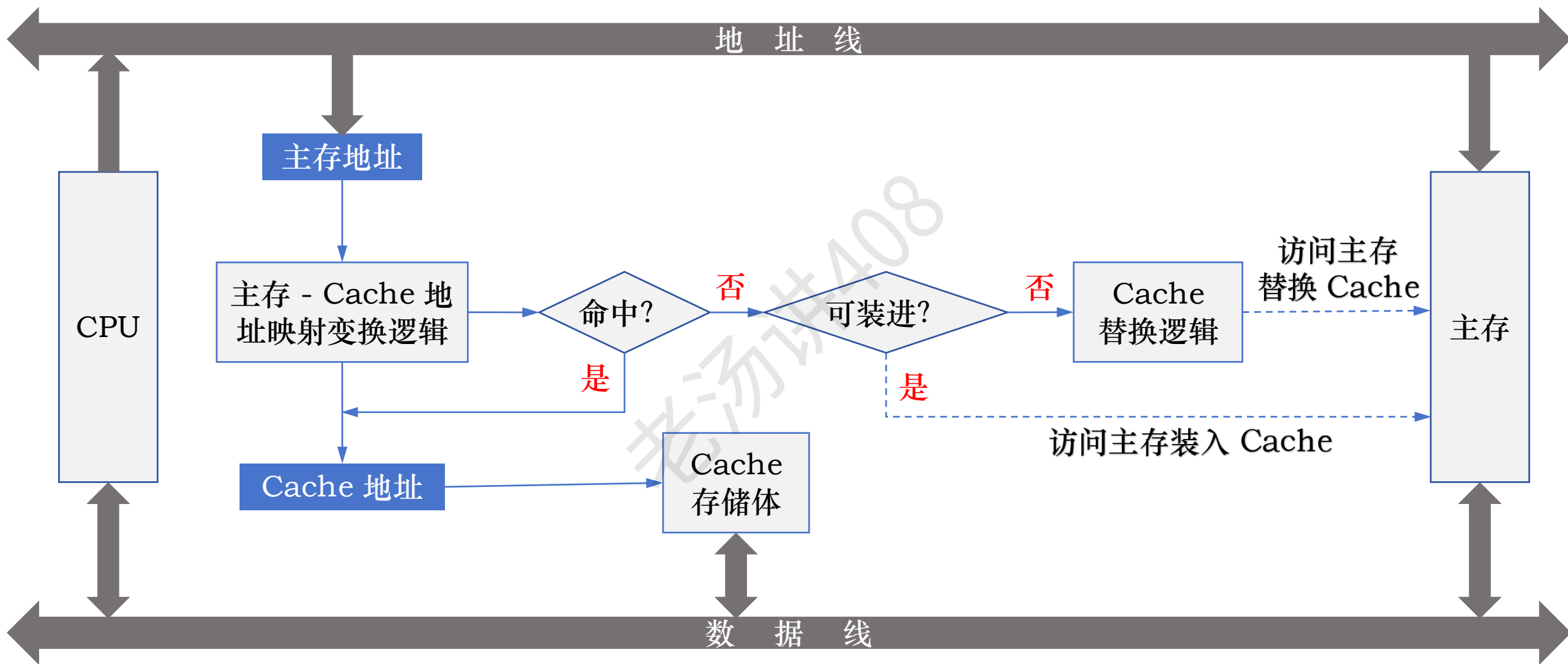
扫一扫上面的二维码图案，加我为朋友。

关联度

当 Cache 大小、主存块大小一定时，关联度和命中率、命中时间、标记所占额外开销等有如下关系：

- (1) 关联度越低，命中率越低。因此直接映射命中率最低，全相联映射命中率最高
- (2) 关联度越低，判断是否命中的开销越小，命中时间越短。因此，直接映射的命中时间最短，全相联映射的命中时间最长
- (3) 关联度越低，标记所占额外空间开销越少。因此，直接映射额外空间开销最少，全相联映射额外空间开销最大

Cache 主存块的替换算法



Cache 主存块的替换算法 - 直接映射

主存储器

0 号主存块

1 号主存块

2 号主存块

3 号主存块

4 号主存块

5 号主存块

6 号主存块

7 号主存块

8 号主存块

9 号主存块

.....

28 号主存块

29 号主存块

30 号主存块

31 号主存块

1KB

主存块号
主存地址: 01 0000 1010
主存地址: 00 0000 1010

CPU

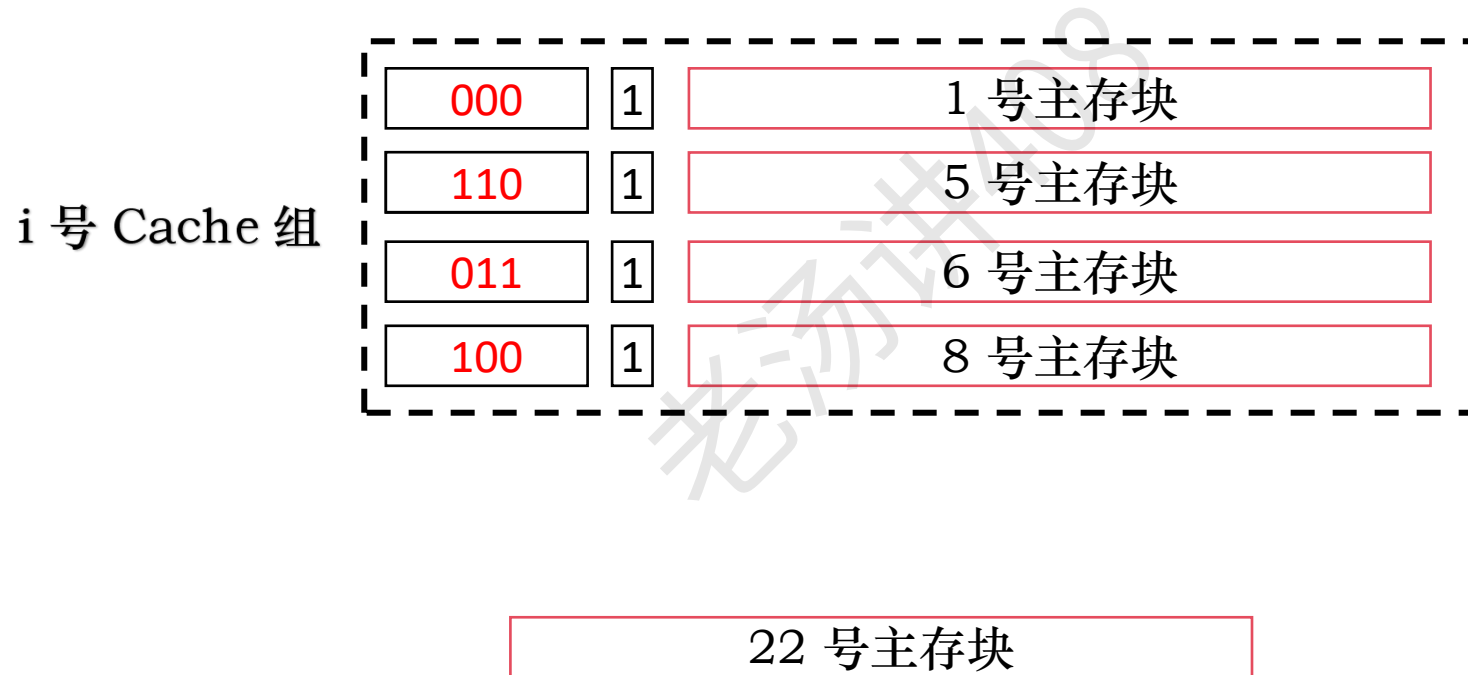
Cache		有效位	32B
标记 (tag)			
0	000	1	
1		0	
2		0	
3		0	

128 B = 4 × 32B

主存块大小: 32B

1024 B / 32B = 32

Cache 主存块的替换算法 - 全相联映射、组相联映射



Cache 主存块的替换算法 - 常用替换算法

① 先进先出 (First-In-First-Out, FIFO)

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

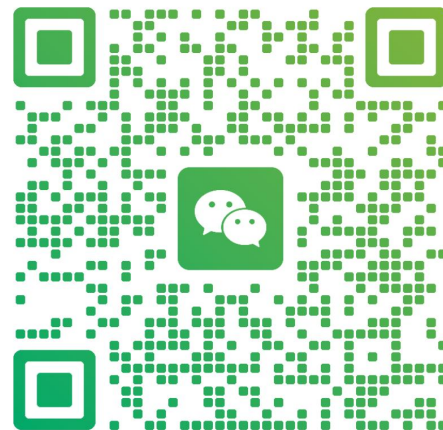
② 最近最少用 (Least-Recently Use, LRU)

③ 最不经常用 (Least-Frequently Used, LFU)

④ 随机替换算法



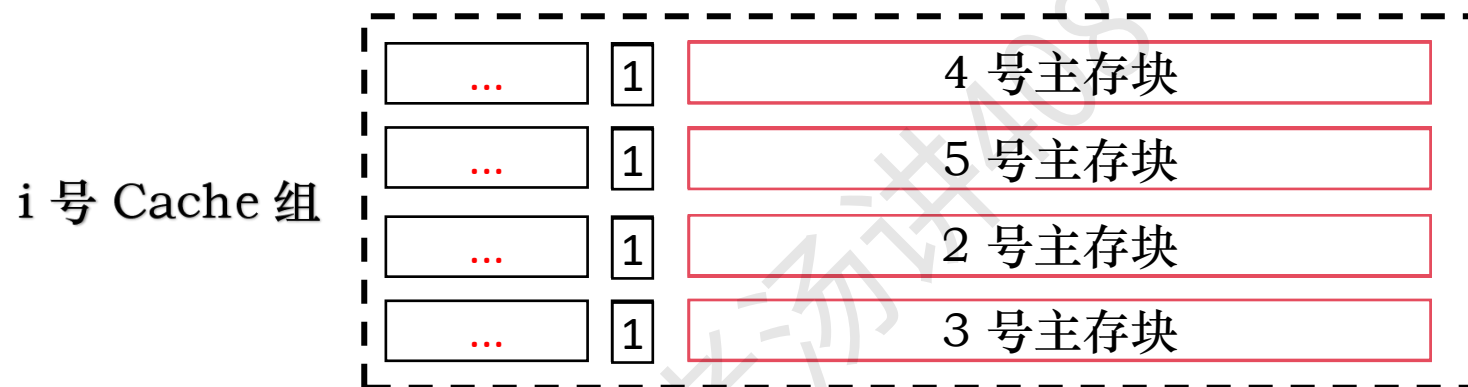
老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

Cache 主存块的替换算法 - 先进先出 (FIFO)

假定主存中 5 个主存块 {1,2,3,4,5} 都映射到这个 Cache 组



FIFO 算法的基本思想是：总是选择最早装入 Cache 的主存块被替换

√ √

CPU 依次访问主存块：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



最先进入的主存块也可能是目前经常要用的

Cache 主存块的替换算法 - 最近最少用 (Least-Recently Use, LRU)

LRU 算法的基本思想是：总是选择近期最少使用的主存块被替换

当前最少使用的块一般来说也是将来最少被访问的

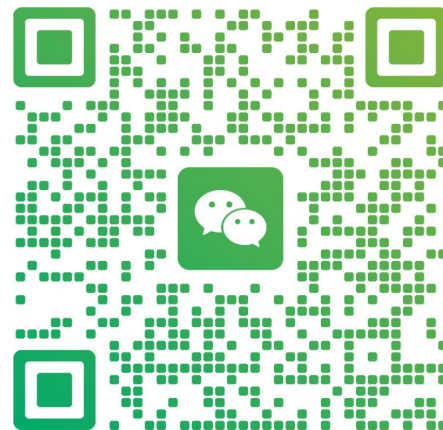
软件实现：哈希表 + 双向链表

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



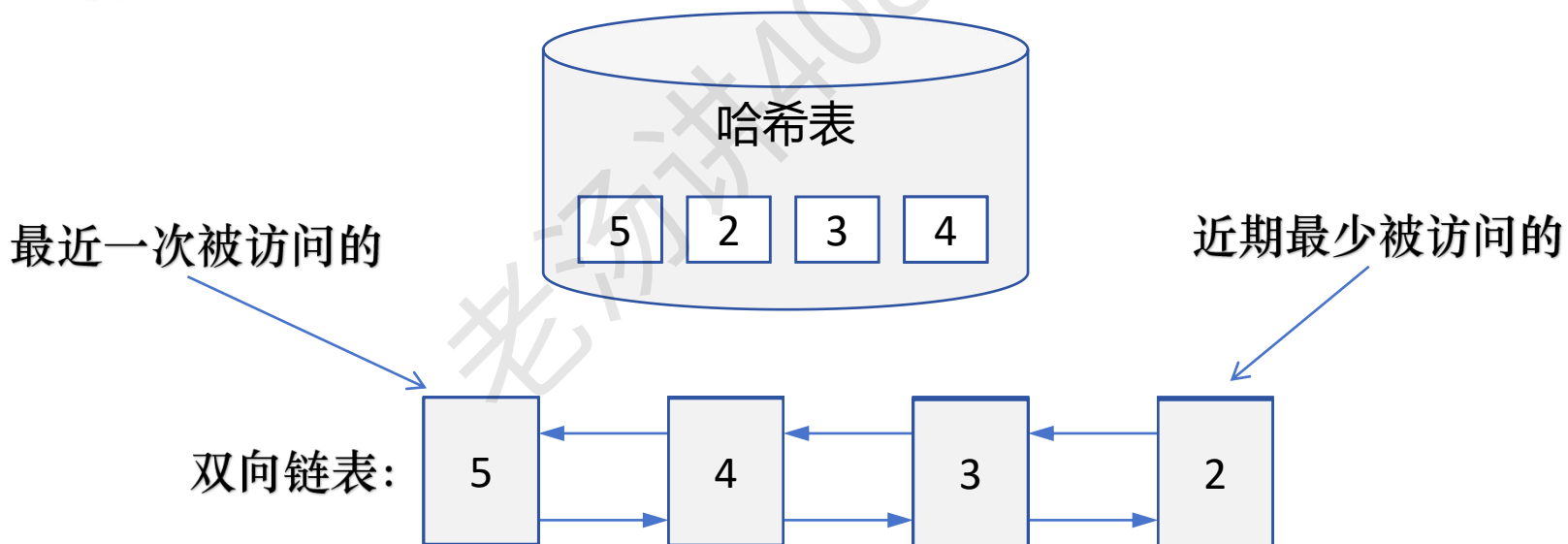
扫一扫上面的二维码图案，加我为朋友。

Cache 主存块的替换算法 - 最近最少用 (Least-Recently Use, LRU)

LRU 算法的基本思想是：总是选择近期最少使用的主存块被替换

当前最少使用的块一般来说也是将来最少被访问的

软件实现：哈希表 + 双向链表



CPU 依次访问主存块：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

↑

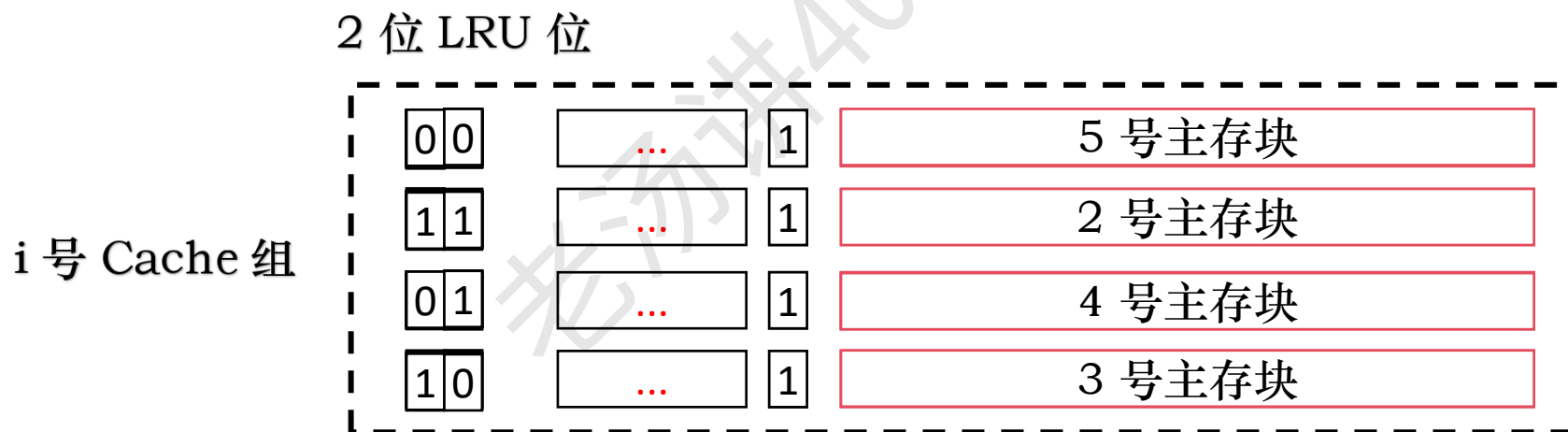
Cache 主存块的替换算法 - 最近最少用 (Least-Recently Use, LRU)

未命中且该组还有空闲行时，则新装入的行的计数器设为 0，其余全加 1

命中时，被访问的行的计数器清零，比其低的计数器加 1，其余不变

未命中且该组无空闲行时，计数值为 3 的那一行中的主存块被淘汰，新装入的行的计数器设为 0，其余加 1

假定主存中 5 个主存块 {1,2,3,4,5} 都映射到这个 Cache 组



√ √ √ √
CPU 依次访问主存块: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
↑

Cache 主存块的替换算法 - 最近最少用 (Least-Recently Use, LRU)

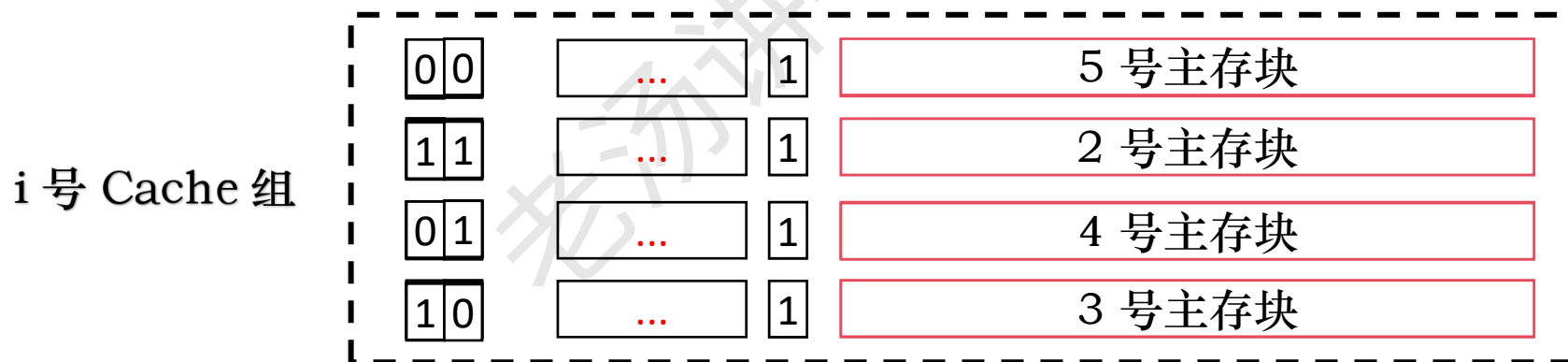
未命中且该组还有空闲行时，则新装入的行的计数器设为 0，其余全加 1

命中时，被访问的行的计数器清零，比其低的计数器加 1，其余不变

未命中且该组无空闲行时，计数值为 3 的那一行中的主存块被淘汰，新装入的行的计数器设为 0，其余加 1

假定主存中 5 个主存块 {1,2,3,4,5} 都映射到这个 Cache 组

2 位 LRU 位 = $\log_2 n$, n 是 Cache 组内的行数



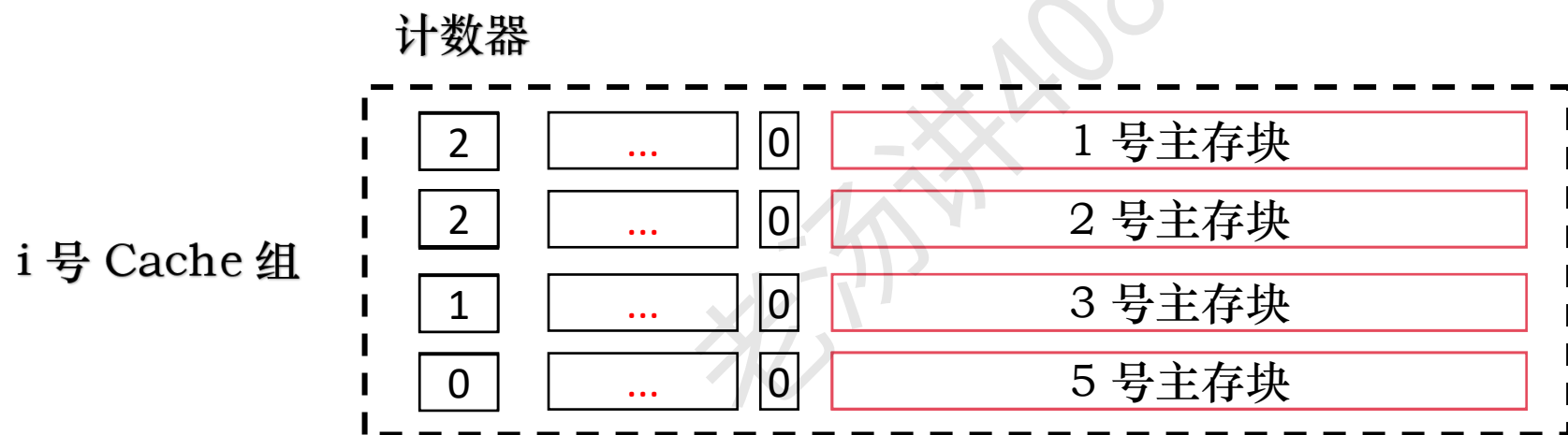
√ √ √ √

CPU 依次访问主存块: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Cache 主存块的替换算法 - 最不经常用 (Least-Frequently Used, LFU)

基本思想是：替换 Cache 中被访用次数最少的块

假定主存中 5 个主存块 {1,2,3,4,5} 都映射到这个 Cache 组



CPU 依次访问主存块: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



Cache 主存块的替换算法 - 随机替换算法

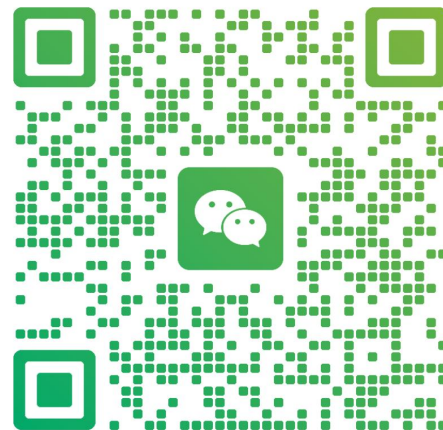
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

从候选行的主存块中随机选取一个淘汰



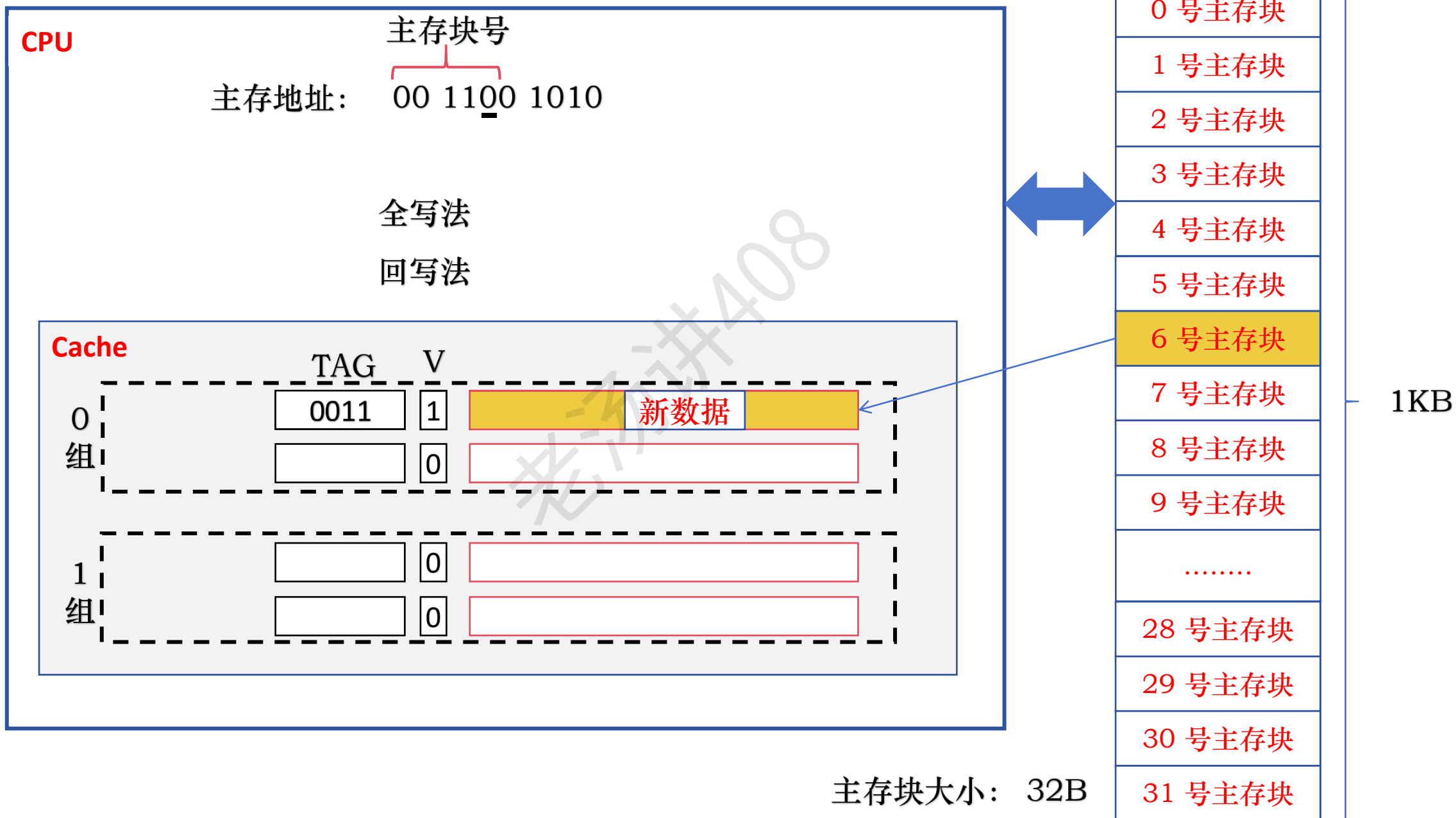
老汤

浙江 杭州

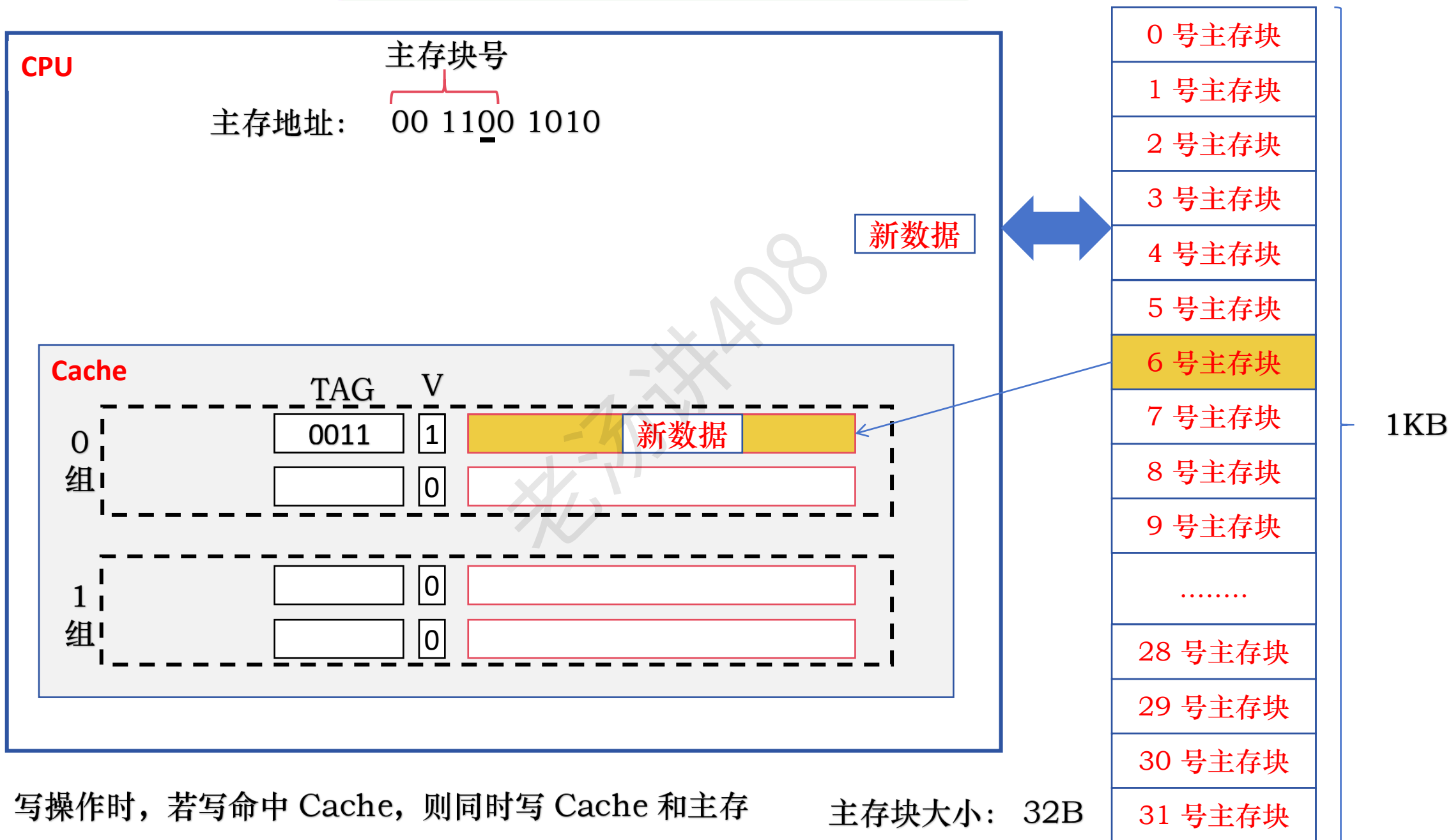


扫一扫上面的二维码图案，加我为朋友。

Cache 数据的一致性问题



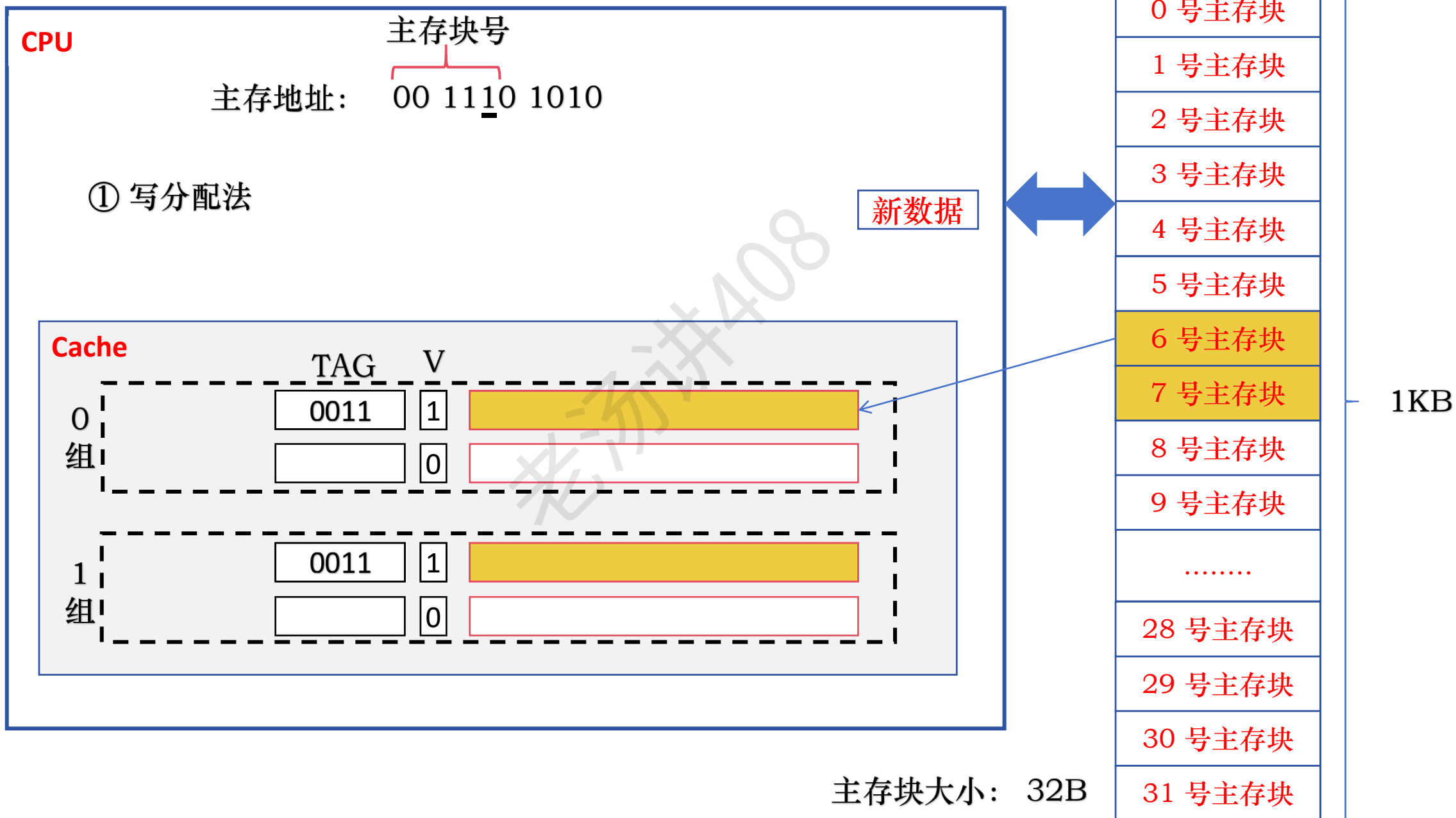
Cache 数据的一致性问题 - 全写法



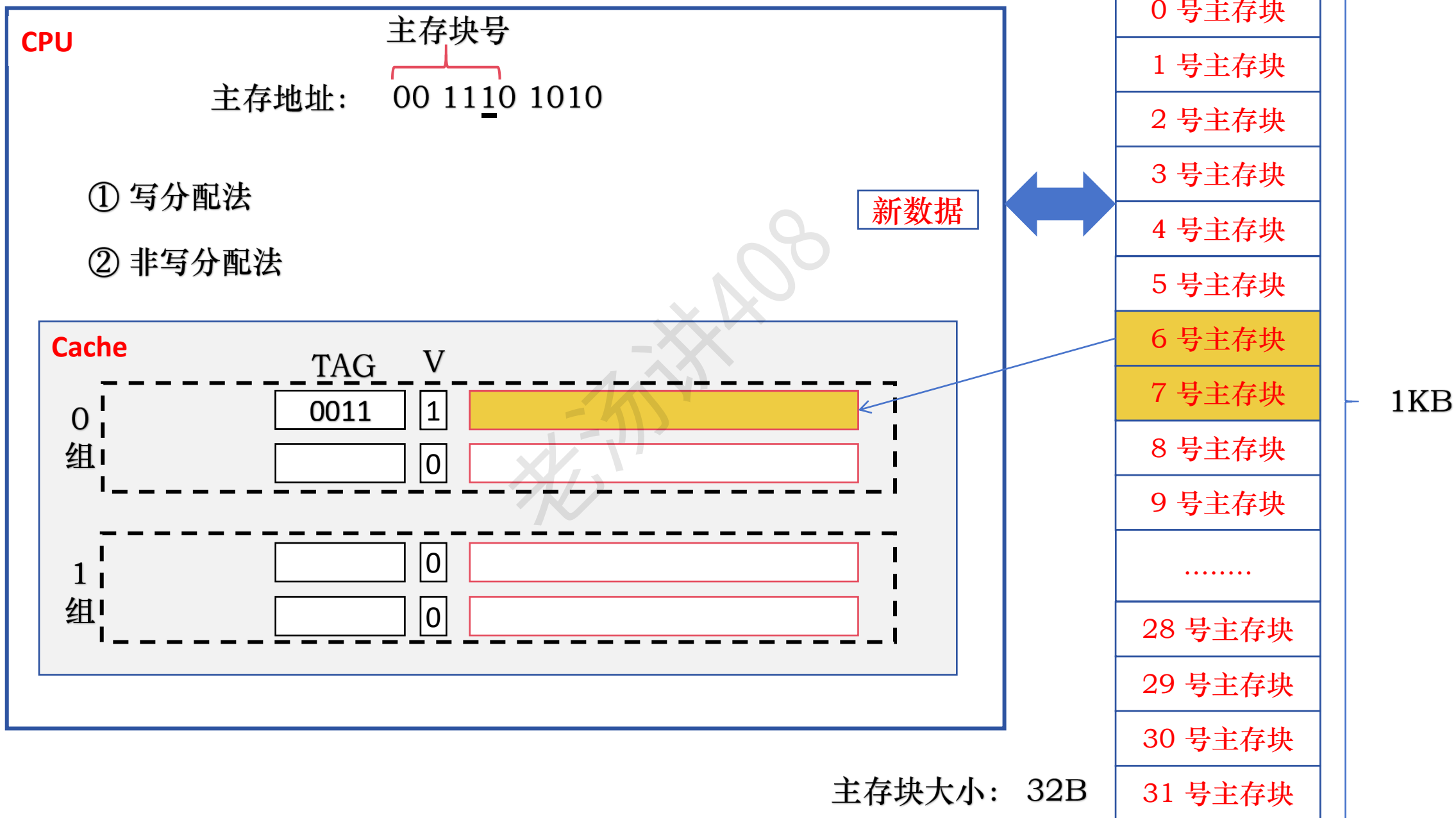
全写法: 写操作时, 若写命中 Cache, 则同时写 Cache 和主存

主存块大小: 32B

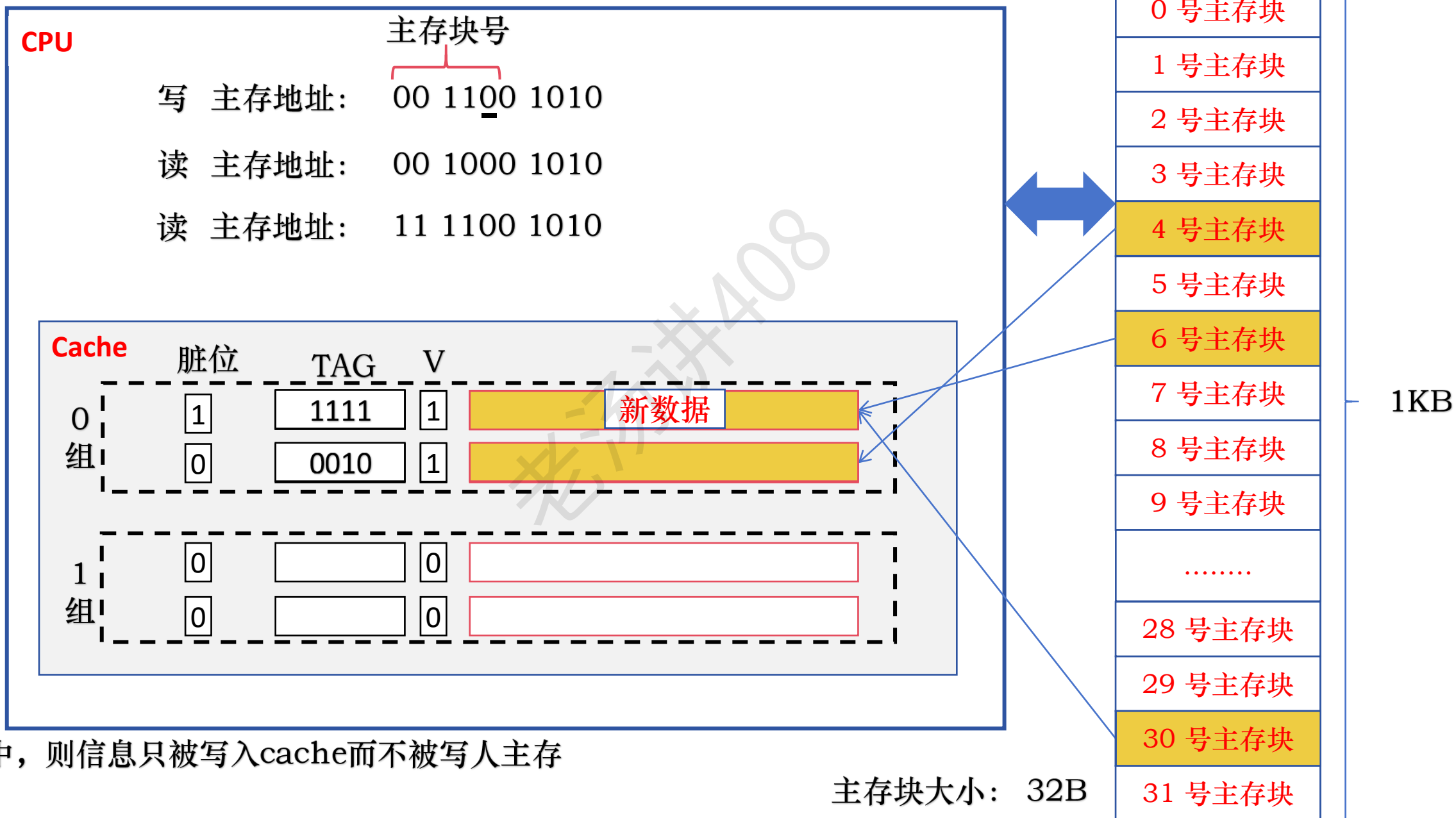
Cache 数据的一致性问题 - 全写法



Cache 数据的一致性问题 - 全写法



Cache 数据的一致性问题 - 回写法



若写命中，则信息只被写入cache而不被写入主存

并行存储器结构技术

提高主存速度的方法：

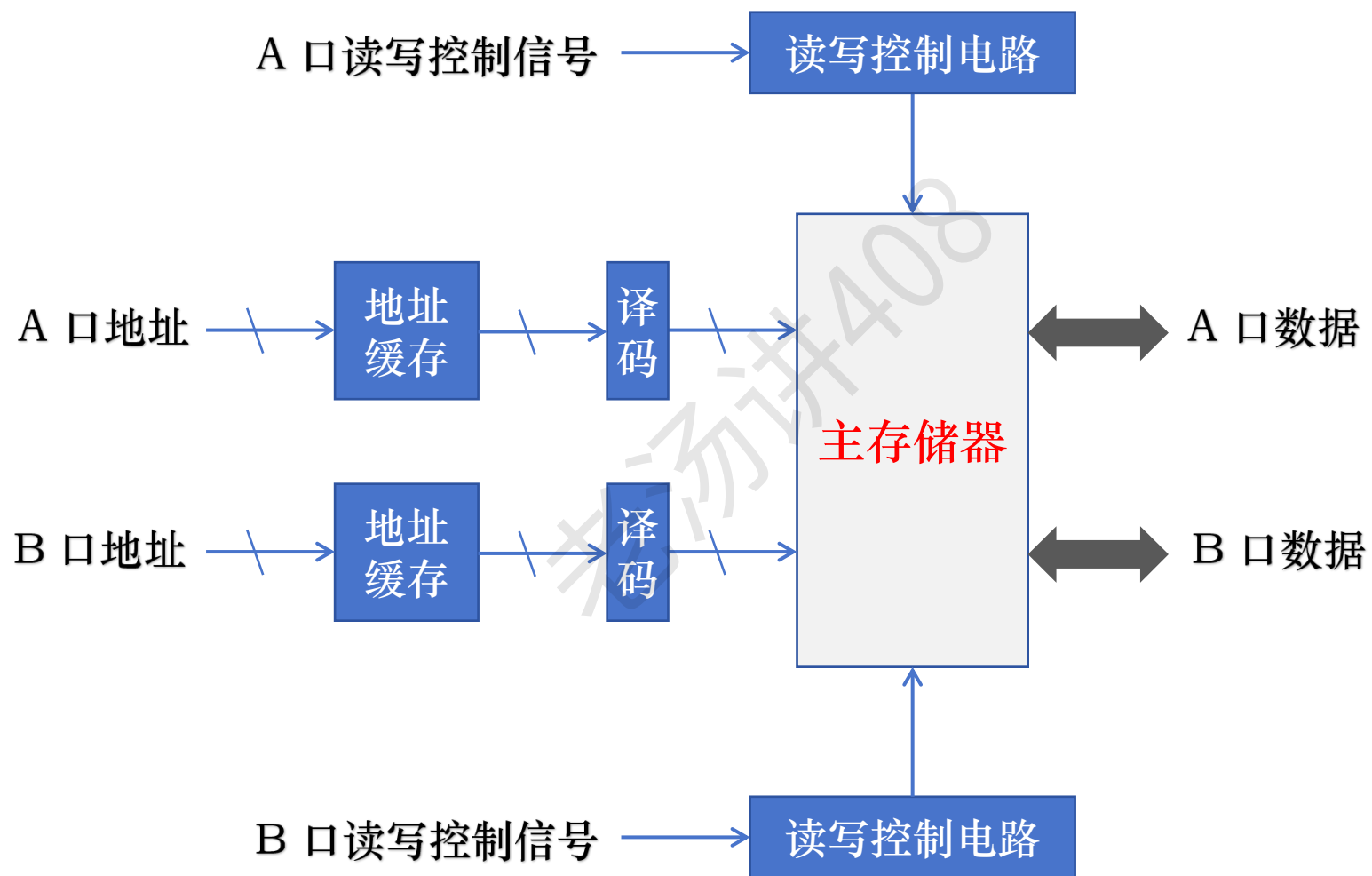
① 提高 DRAM 芯片本身的速度

② 在 CPU 和主存之间增加高速缓存存储器

③ 并行结构技术

- 双口存储器
- 多模块存储器

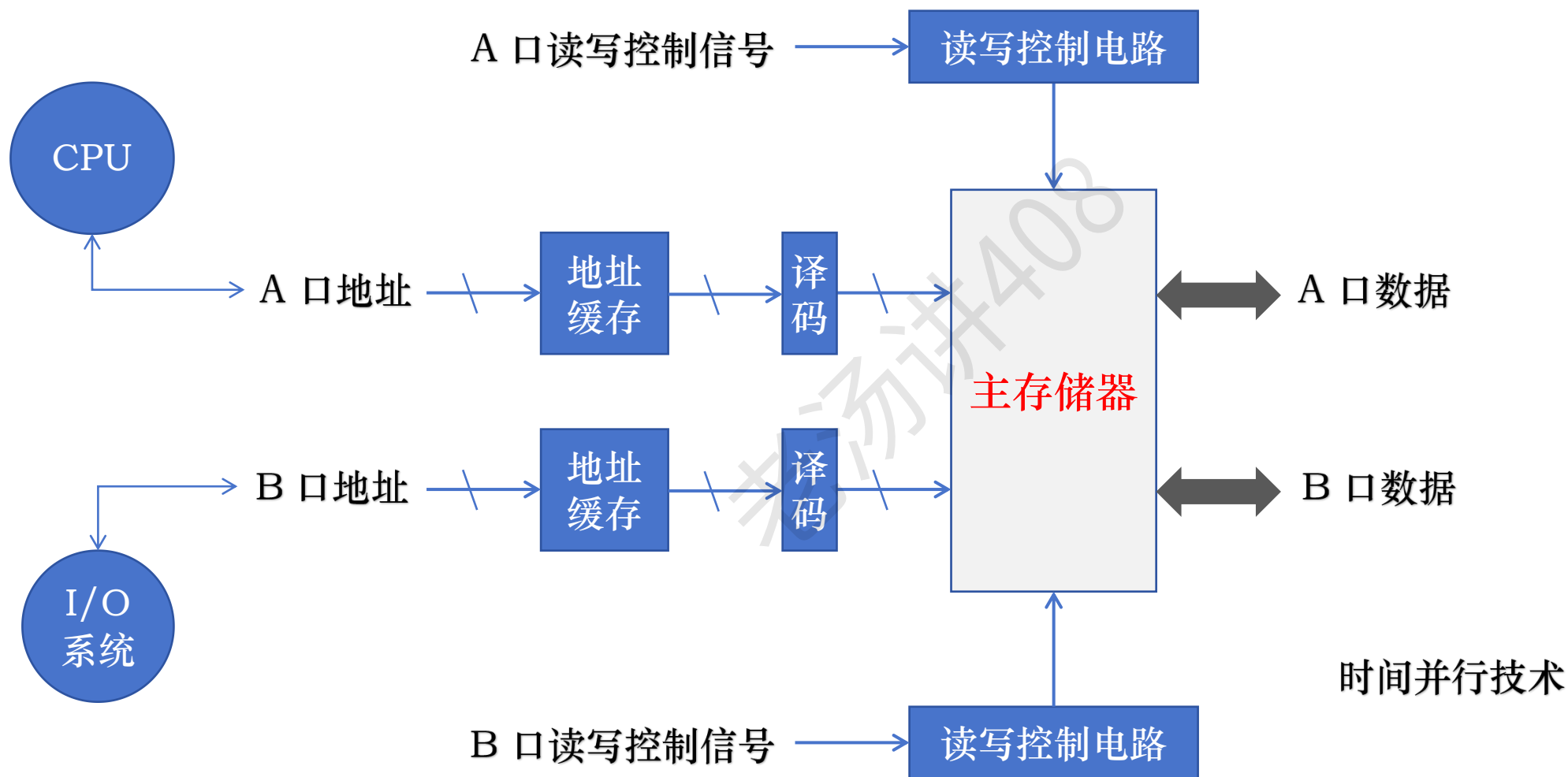
并行存储器结构技术 - 双口存储器



通用寄存器组

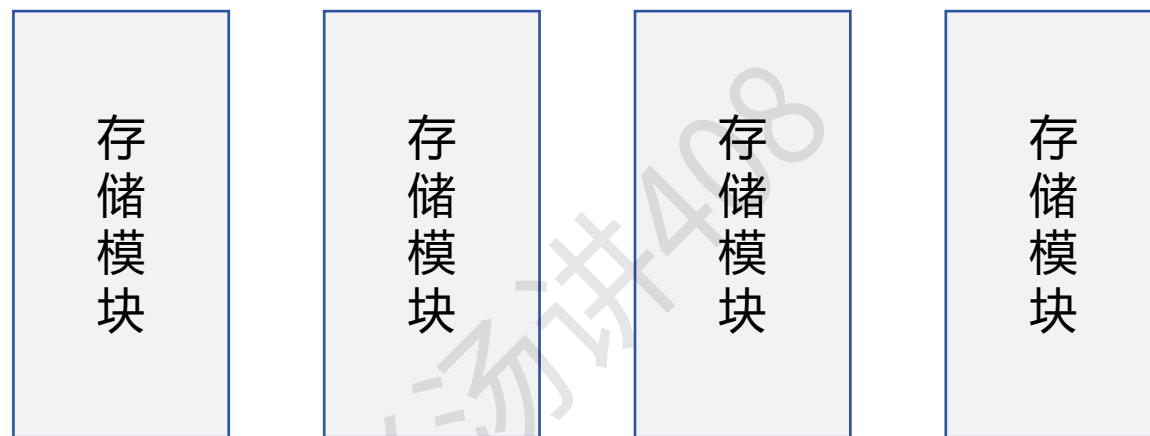


并行存储器结构技术 - 双口存储器



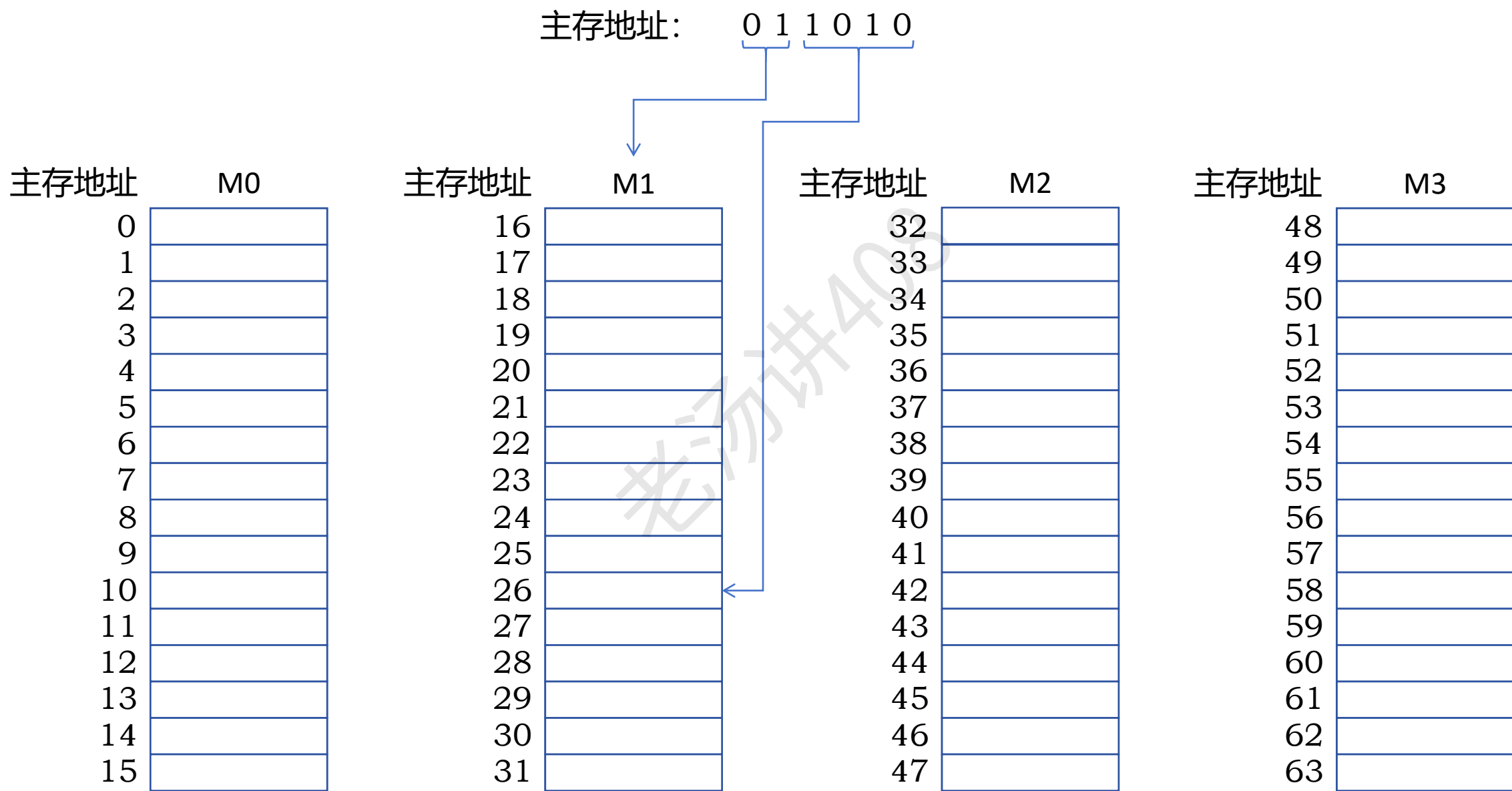
并行存储器结构技术 - 多模块存储器

各自具有独立的地址寄存器 MAR、数据寄存器 MDR、地址译码器、驱动电路和读写电路



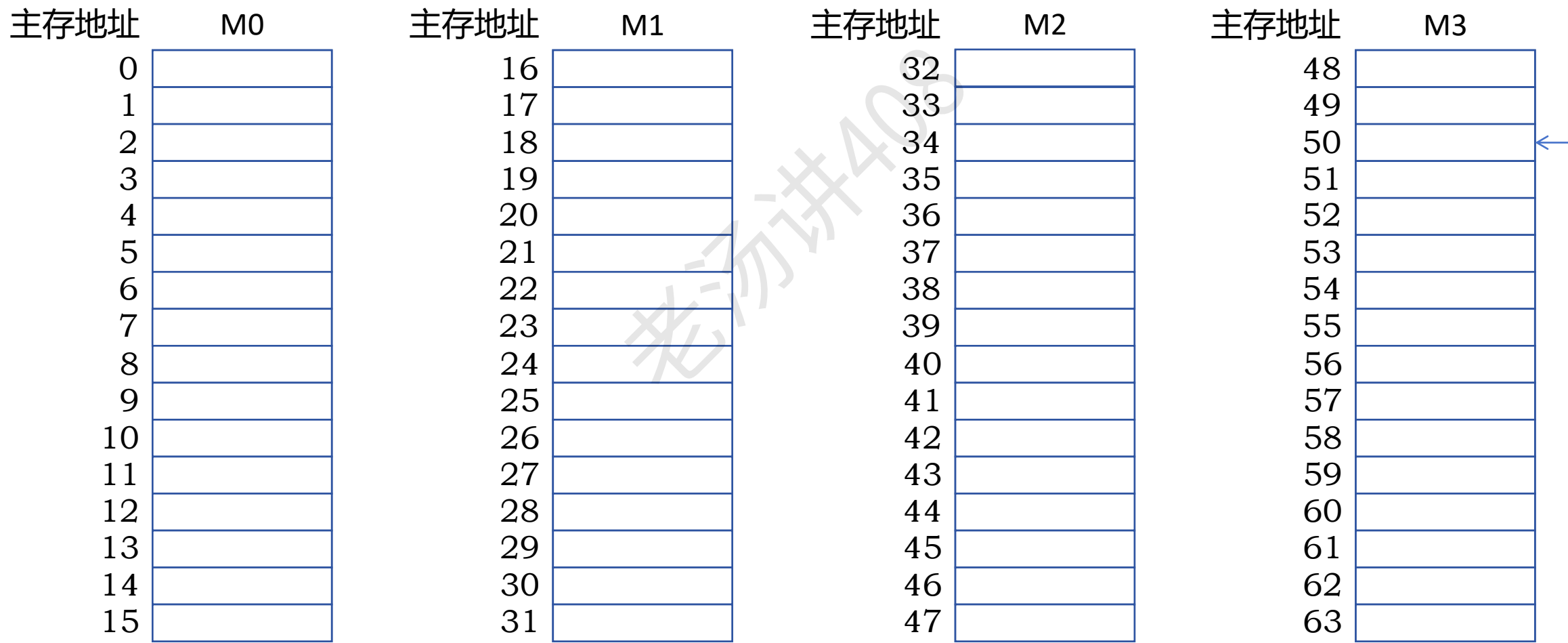
不同的编址方式 {
连续编址方式
交叉编址方式

并行存储器结构技术 - 多模块存储器：连续编址方式

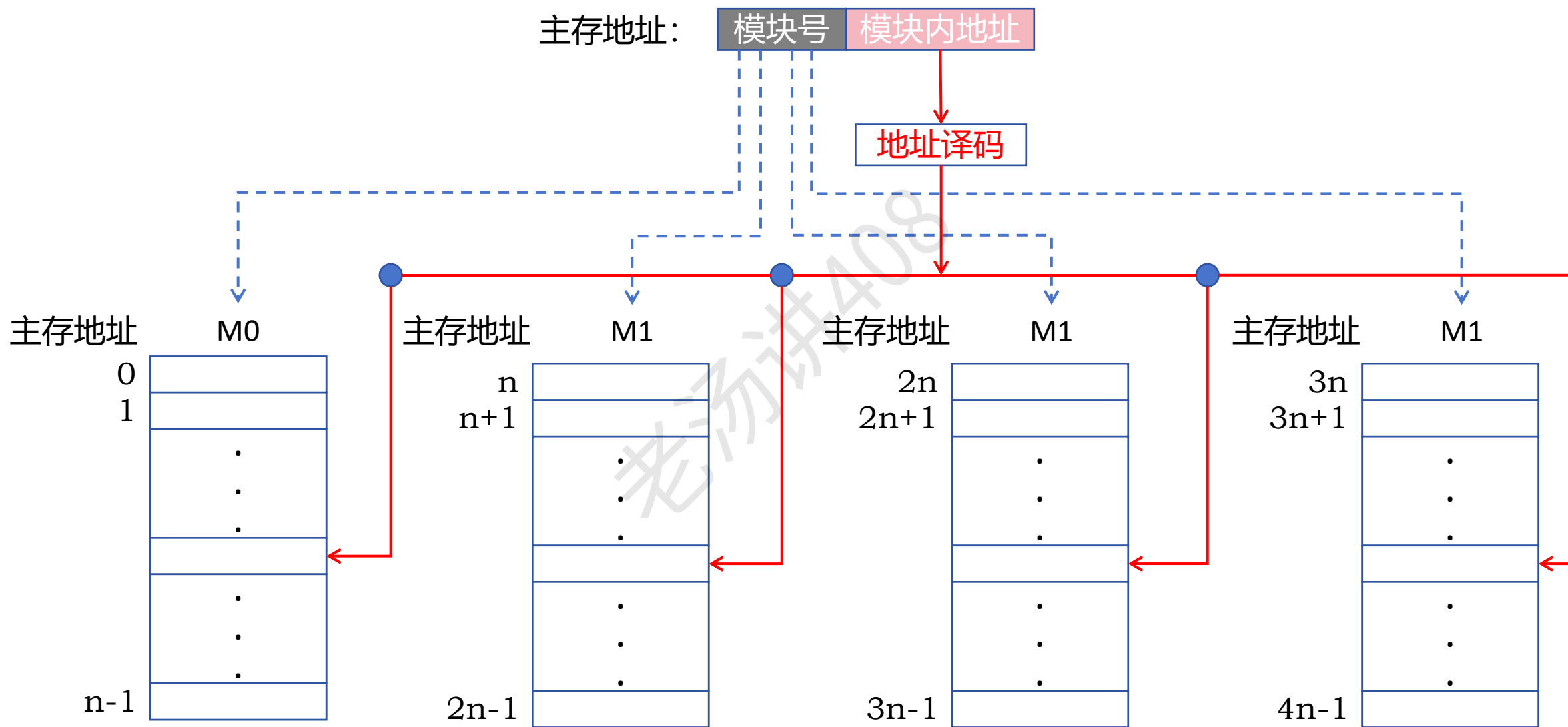


并行存储器结构技术 - 多模块存储器：连续编址方式

主存地址： 1 1 0 0 1 0



并行存储器结构技术 - 多模块存储器：连续编址方式

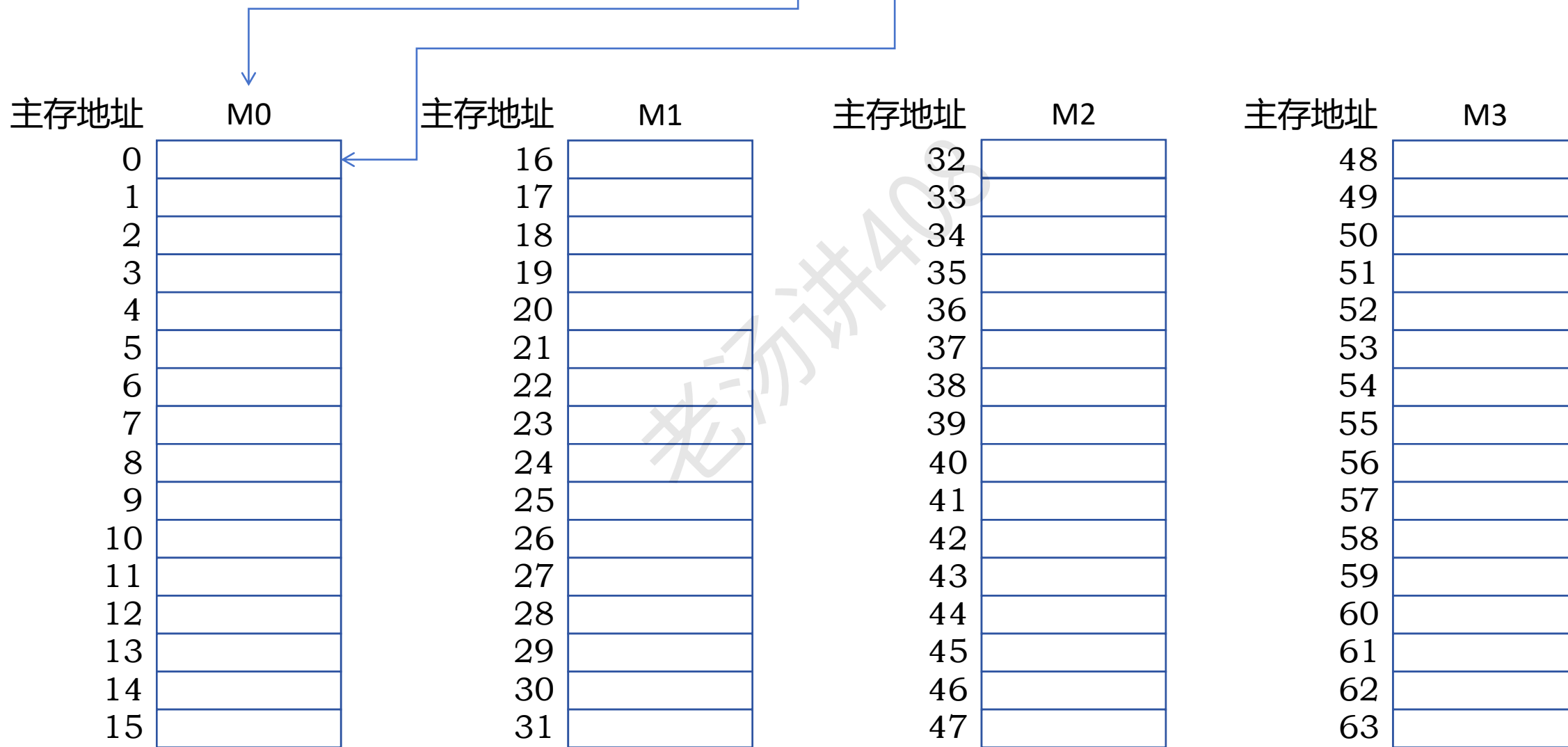


将低位的模块内地址送到由高位模块号确定的存储模块中进行译码

并行存储器结构技术 - 多模块存储器：连续编址方式

主存块的大小是 32B

主存地址： 0 0 0 0 0 0

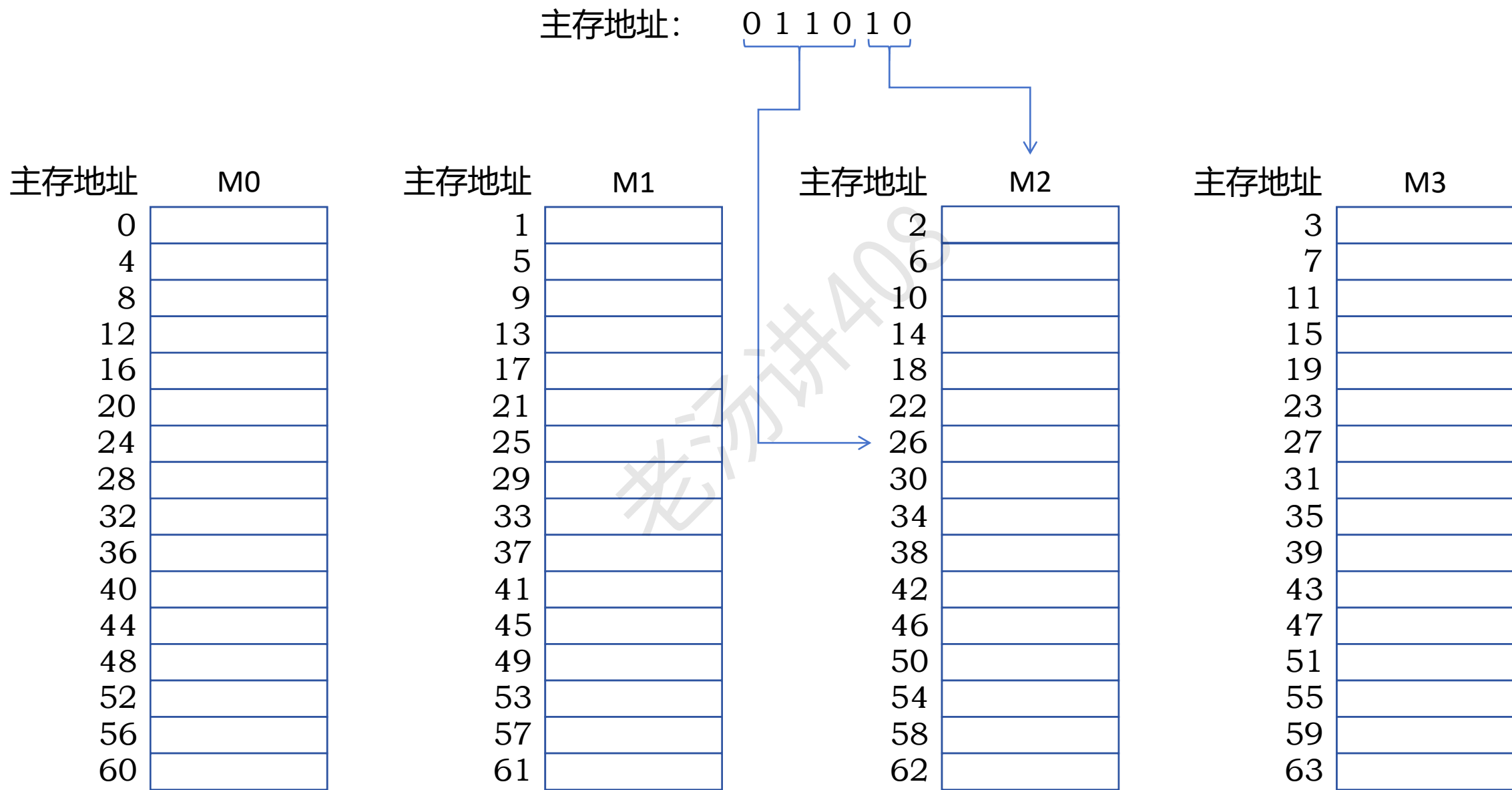


并行存储器结构技术 - 多模块存储器：交叉编址方式

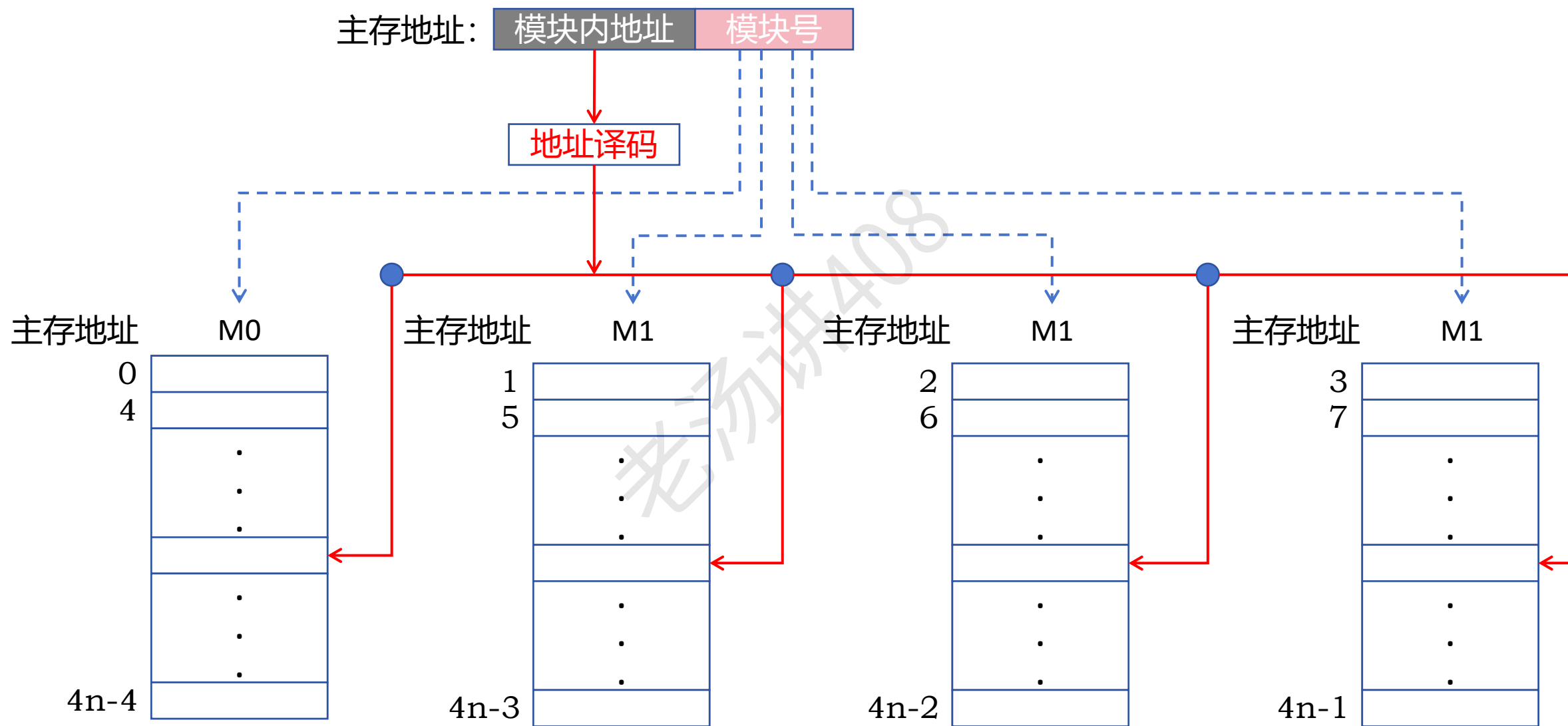
主存块的大小是 32B

主存地址	M0	主存地址	M1	主存地址	M2	主存地址	M3
0		1		2		3	
4		5		6		7	
8		9		10		11	
12		13		14		15	
16		17		18		19	
20		21		22		23	
24		25		26		27	
28		29		30		31	
32		33		34		35	
36		37		38		39	
40		41		42		43	
44		45		46		47	
48		49		50		51	
52		53		54		55	
56		57		58		59	
60		61		62		63	

并行存储器结构技术 - 多模块存储器：交叉编址方式

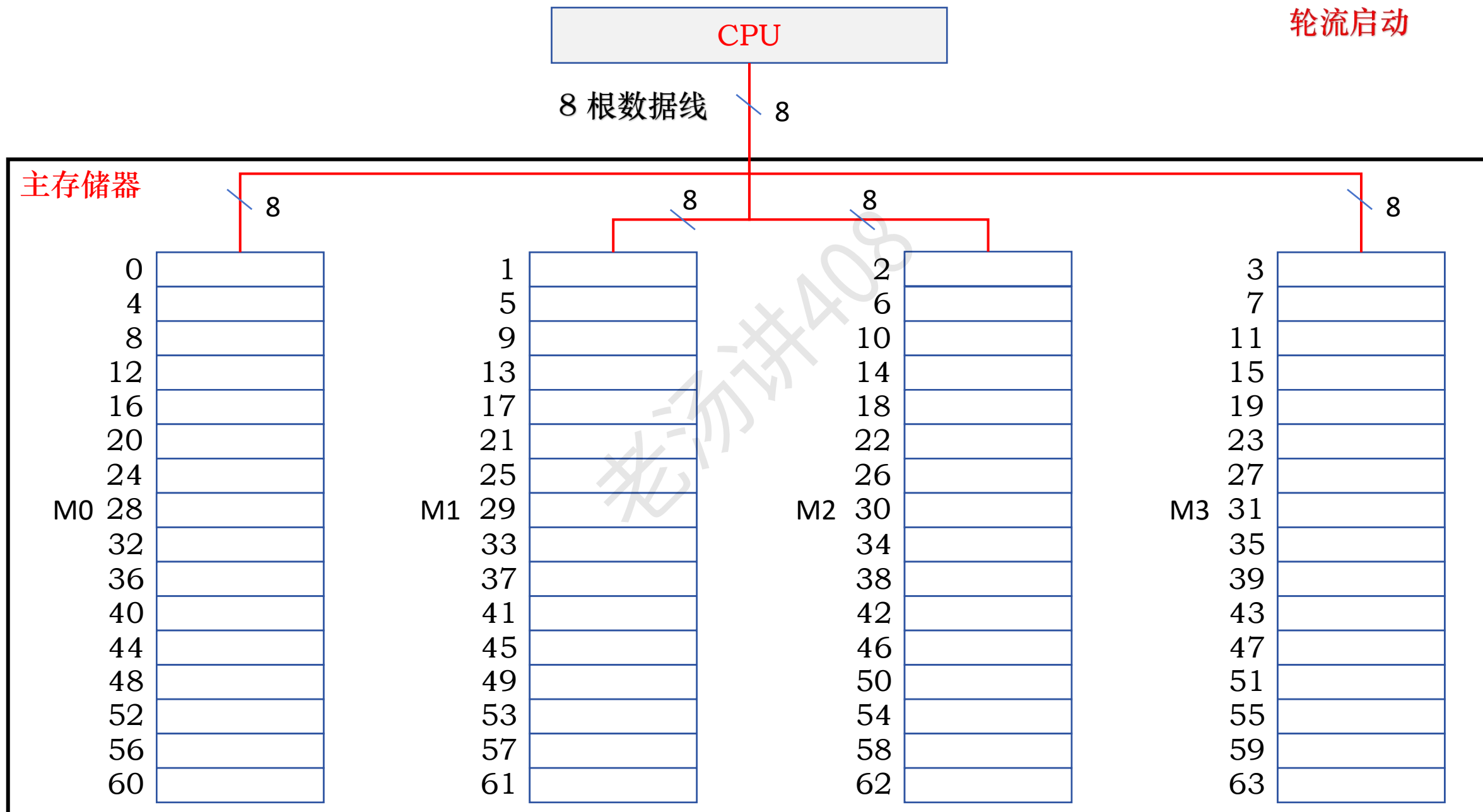


并行存储器结构技术 - 多模块存储器：交叉编址方式

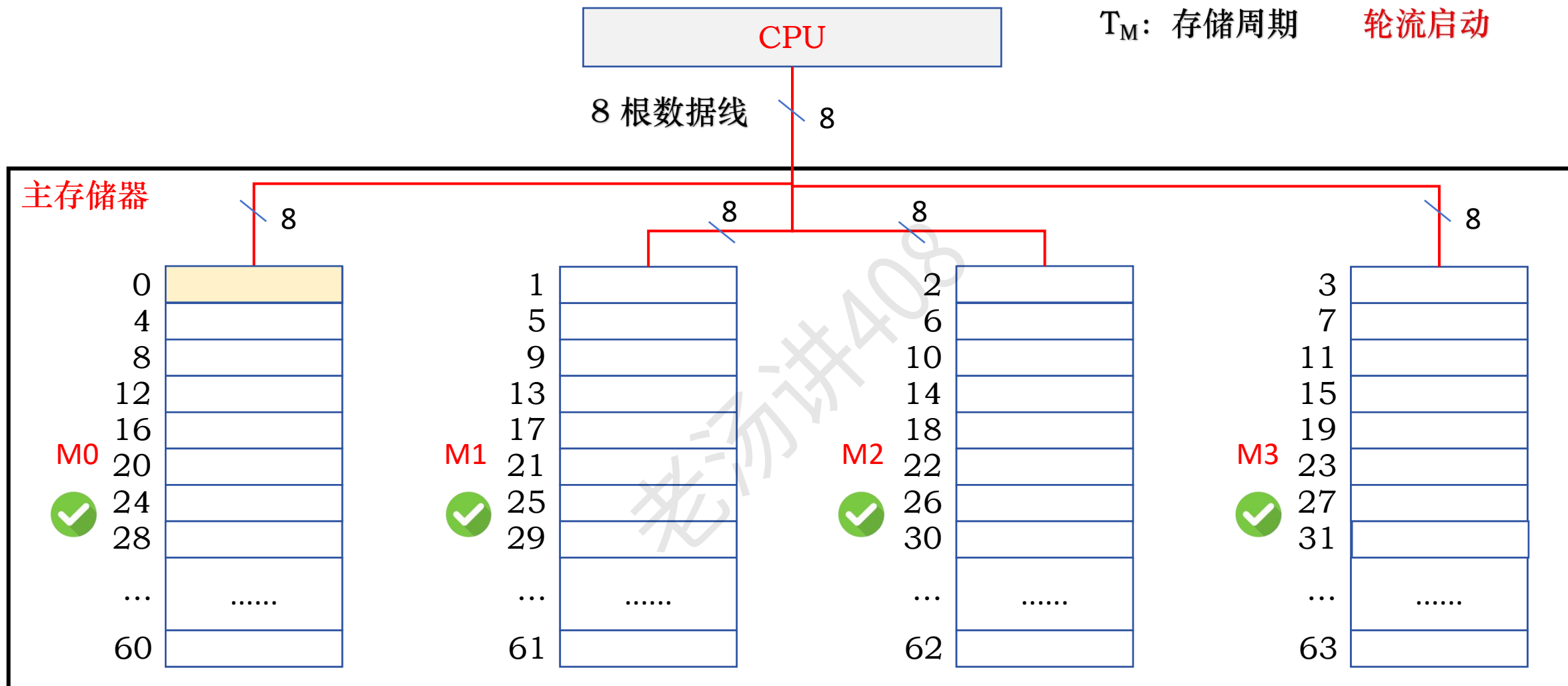


将高位的模块内地址送到由低位模块号确定的存储模块中进行译码

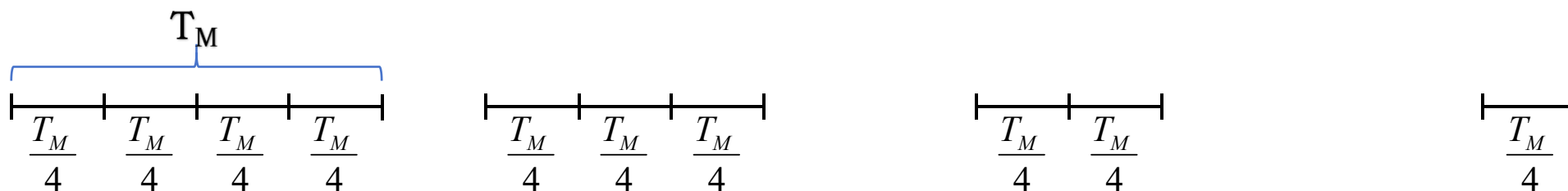
并行存储器结构技术 - 多模块存储器：交叉编址方式



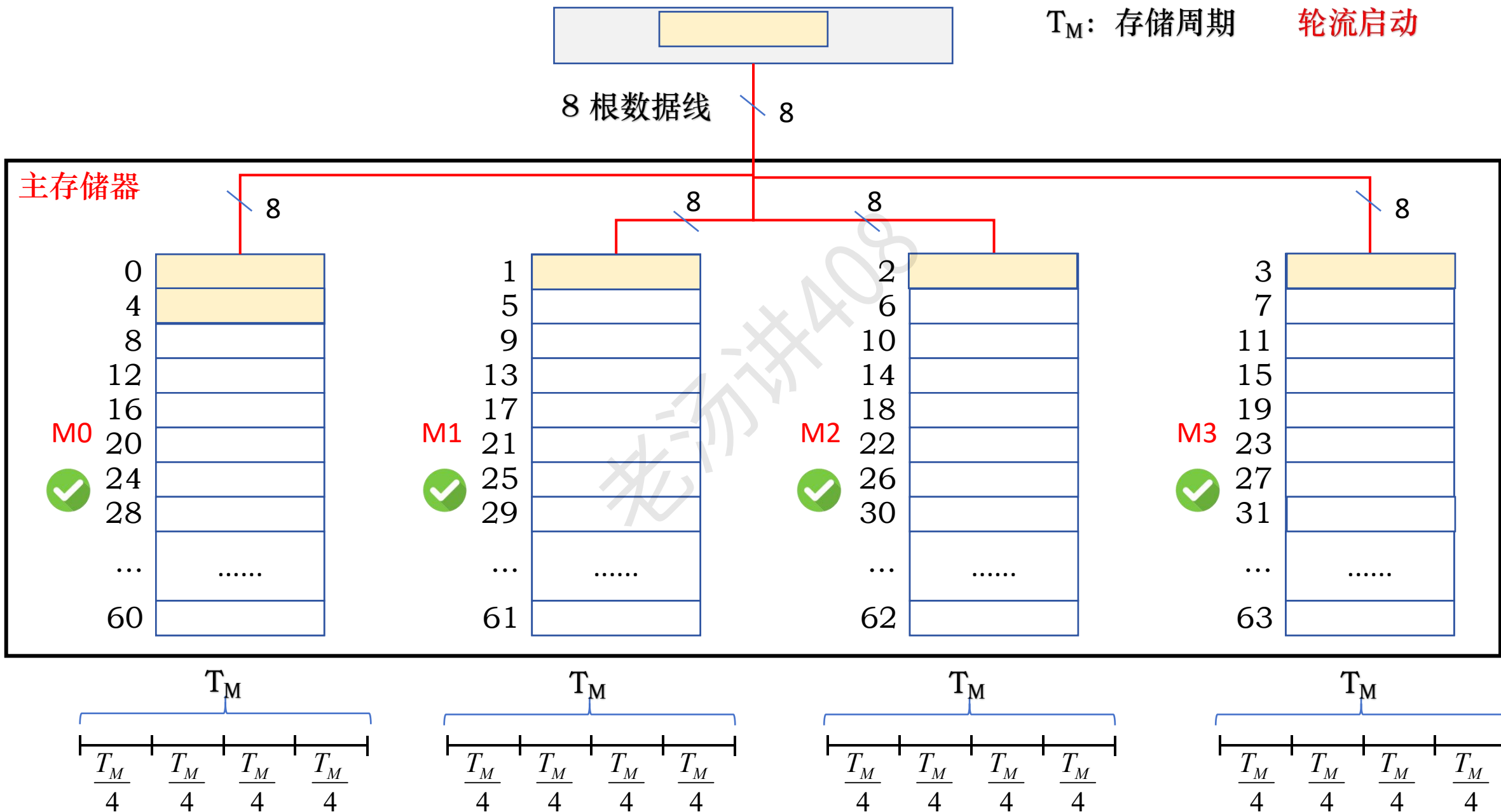
并行存储器结构技术 - 多模块存储器：交叉编址方式



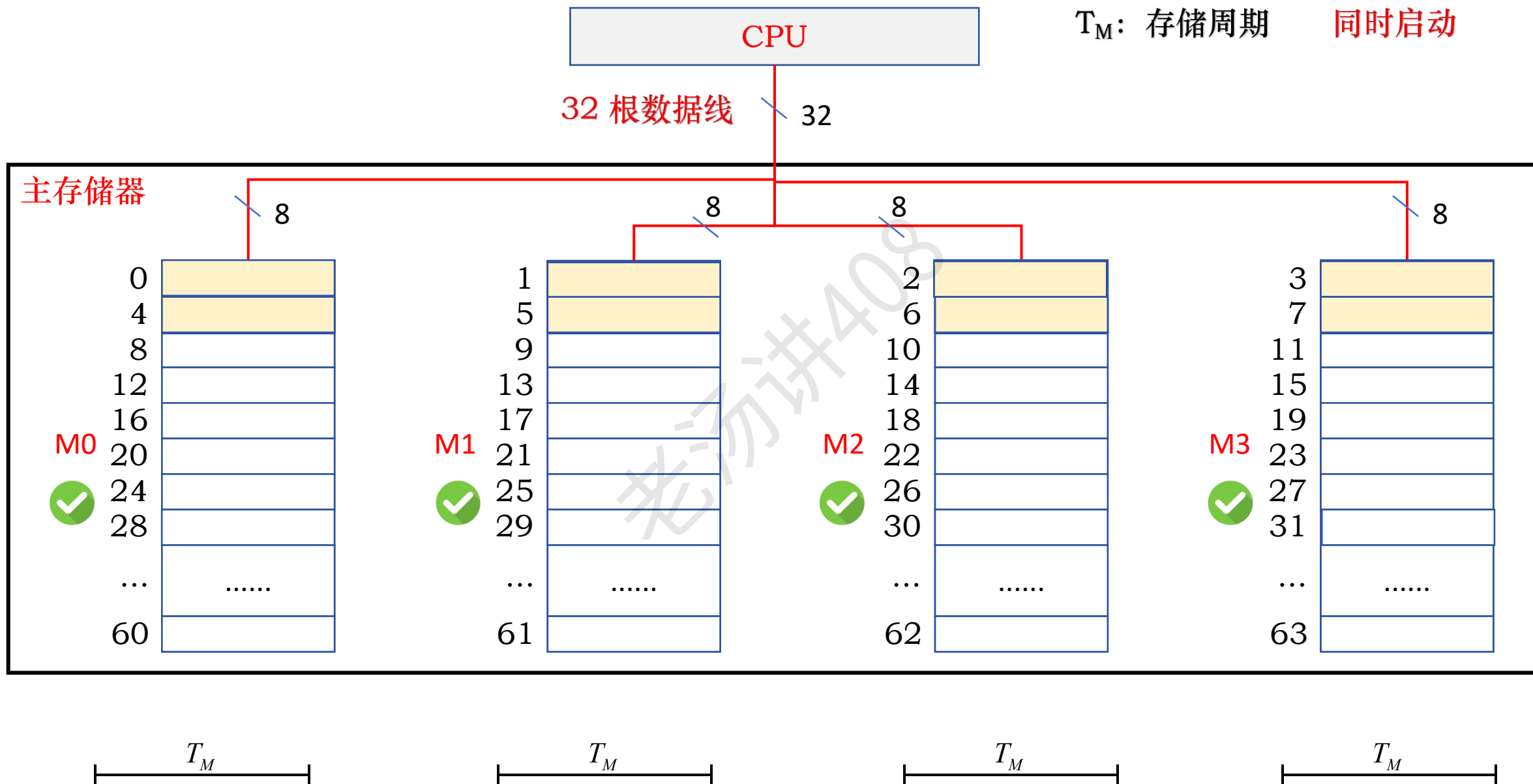
T_M : 存储周期 轮流启动



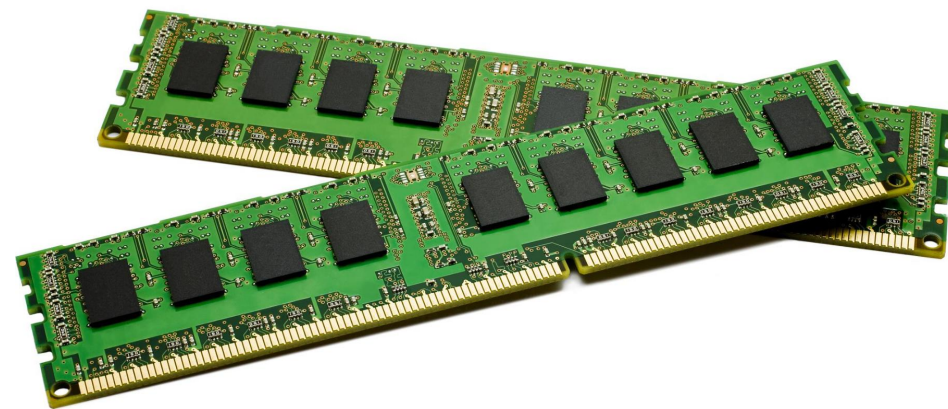
并行存储器结构技术 - 多模块存储器：交叉编址方式



并行存储器结构技术 - 多模块存储器：交叉编址方式



内存条、内存插槽和存储控制器

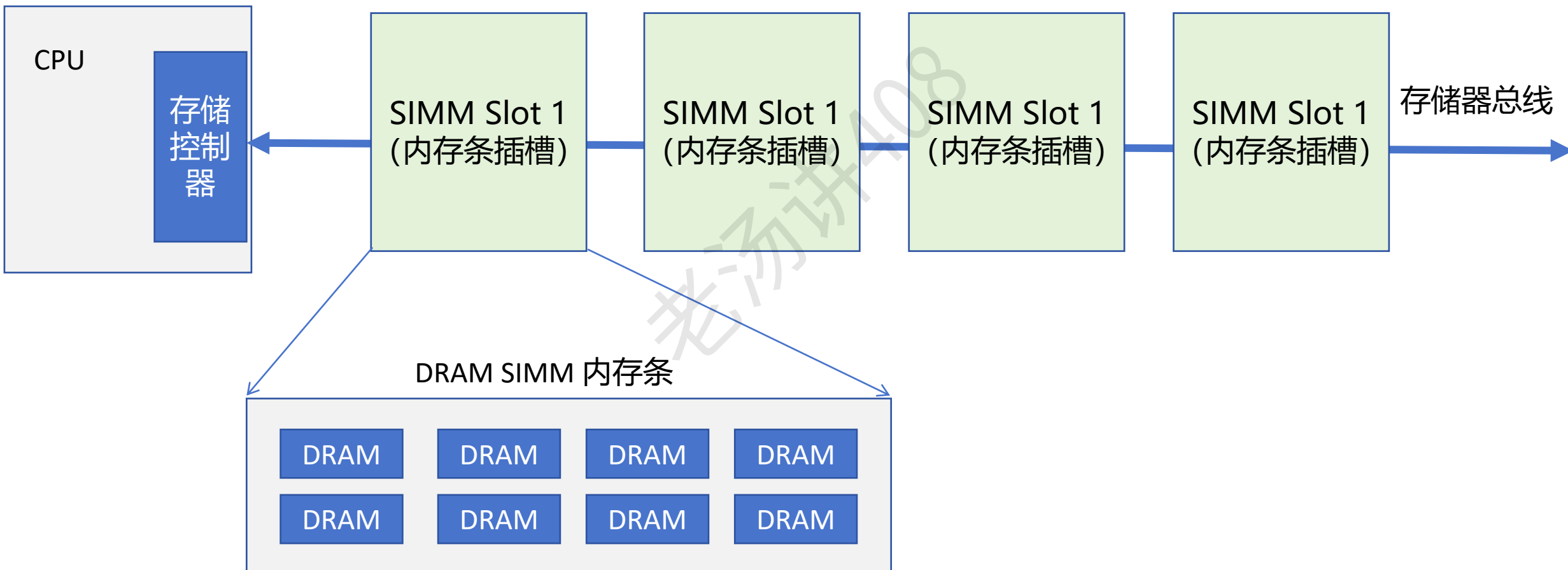


存储芯片

RAM SRAM、**DRAM**

ROM

内存条、内存插槽和存储控制器



数组元素主存地址的计算

① 一维数组

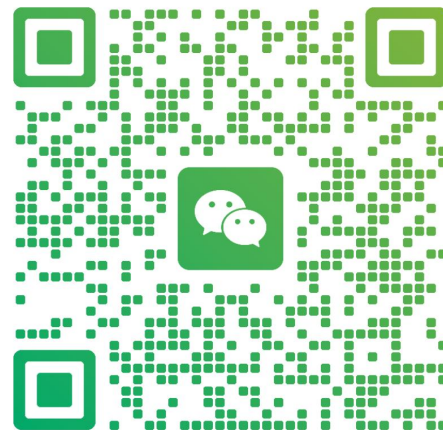
② 二维数组

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

数组元素主存地址的计算：一维数组

存储字大小： 4B

主存储器

按字节编址

```
int a[12] = ...;
```

数组占用的是一块连续的主存空间

72	
68	
64	
60	
56	
52	a[11]
48	a[10]
44	a[9]
40	a[8]
36	a[7]
32	a[6]
28	a[5]
24	a[4]
20	a[3]
16	a[2]
12	a[1]
8	a[0]
4	
0	

数组元素主存地址的计算：一维数组

存储字大小： 4B

主存储器

按字节编址

```
int a[12] = ...;
```

数组占用的是一块连续的主存空间

$a[i]$ 的主存地址 = 首地址 + $i \times 4$

$a[7]$ 的主存地址 = $8 + 7 \times 4 = 36$

$a[11]$ 的主存地址 = $8 + 11 \times 4 = 52$

首地址 →

72	
68	
64	
60	
56	
52	a[11]
48	a[10]
44	a[9]
40	a[8]
36	a[7]
32	a[6]
28	a[5]
24	a[4]
20	a[3]
16	a[2]
12	a[1]
8	a[0]
4	
0	

数组元素主存地址的计算：一维数组

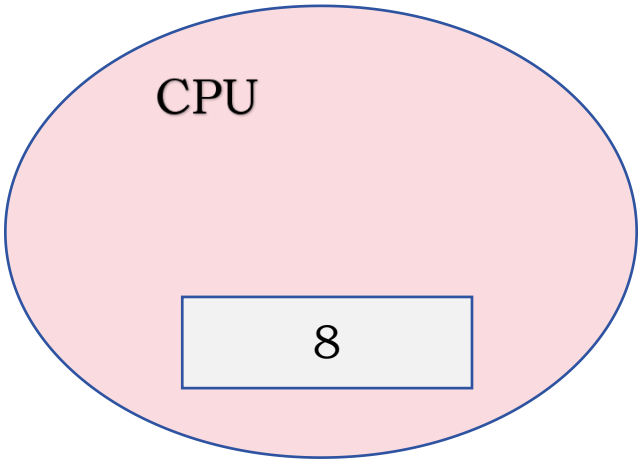
存储字大小： 4B
按字节编址

主存储器

```
int a[12] = ...;
```

数组占用的是一块连续的主存空间

$a[i]$ 的主存地址 = 首地址 + $i \times 4$



首地址 →

72	
68	
64	
60	
56	
52	a[11]
48	a[10]
44	a[9]
40	a[8]
36	a[7]
32	a[6]
28	a[5]
24	a[4]
20	a[3]
16	a[2]
12	a[1]
8	a[0]
4	
0	

数组元素主存地址的计算：一维数组

```
int a[12] = ...;
```

数组占用的是一块连续的主存空间

$a[i]$ 的主存地址 = 首地址 + $i \times 4$

占主存块数 = $12 \times 4B / 16B = 3$

存储字大小： 4B
按字节编址
主存块大小： 16B

首地址

主存储器



数组元素主存地址的计算：一维数组

存储字大小： 4B

按字节编址

主存块大小： 16B

```
int a[12] = ...;
```

数组占用的是一块连续的主存空间

$a[i]$ 的主存地址 = 首地址 + $i \times 4$

占主存块数 = $12 \times 4B / 16B = 3$

$16 / 16 = 1$ 号主存块

$a[7]$ 的主存地址 = $16 + 7 \times 4 = 44$

$44 / 16 = 2$

首地址

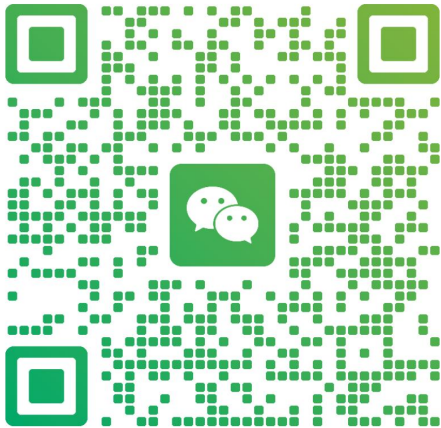


数组元素主存地址的计算：二维数组

存储字大小： 4B
按字节编址
主存块大小： 16B

```
int a[4][5] = ...;
```

按行存储 按列存储
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

老汤讲408

主存储器	
76	a[3][4]
72	a[3][3]
68	a[3][2]
64	a[3][1]
60	a[3][0]
56	a[2][4]
52	a[2][3]
48	a[2][2]
44	a[2][1]
40	a[2][0]
36	a[1][4]
32	a[1][3]
28	a[1][2]
24	a[1][1]
20	a[1][0]
16	a[0][4]
12	a[0][3]
8	a[0][2]
4	a[0][1]
0	a[0][0]

4 号主存块

3 号主存块

2 号主存块

1 号主存块

0 号主存块

首地址 → 0

数组元素主存地址的计算：二维数组

```
int a[4][5] = ...;
```

按行存储

$a[i][j]$ 到 $a[0][0]$ 之间元素数量 $= i \times 5 + j$

元素 $a[i][j]$ 的主存地址 = 首地址 + $(i \times 5 + j) \times 4$

元素 $a[1][4]$ 的主存地址 $= 0 + (1 \times 5 + 4) \times 4 = 36$

存储字大小： 4B
按字节编址
主存块大小： 16B

首地址 → 0



数组元素主存地址的计算：二维数组

```
int a[4][5] = ...;
```

按行存储

元素 $a[i][j]$ 的主存地址 = 首地址 + $(i \times 5 + j) \times 4$

① 按行访问

存储字大小： 4B
按字节编址
主存块大小： 16B

首地址 → 0



数组元素主存地址的计算：二维数组

```
int a[4][5] = ...;
```

按行存储

元素 $a[i][j]$ 的主存地址 = 首地址 + $(i \times 5 + j) \times 4$

① 按行访问

② 按列访问

存储字大小： 4B
按字节编址
主存块大小： 16B

首地址 → 0

